408 详解 · 2014 年选择题 · 升级版

本册每题先给出原卷题干与选项全文（转录自公开渠道真题，来源与核对见《2014-升级日志》），再按 6 节讲解；讲解重心放在方法、推理、易混点上。

升级版说明：每题标题下新增"**题干与选项：**"节（原卷题干与选项全文）；6 节之后新增 7 类增补小节（【图像】【定位】【易混对比】【坑位】【套路】【类比】【口诀】，无适用内容的小节省略）；重点句按全卷统一配色：红＝关键结论、蓝＝定位与套路关键动作、橙＝坑位警告、绿＝口诀；【口诀】附考频标注（该知识点在 2009–2022 年解析中的出现次数＋高/中/低频档位，高频红标）。

第 1 题 · 双层循环的时间复杂度（正确答案：C）

题干与选项：

下列程序段的时间复杂度是 。

count=0;

for(k=1;k<=n;k*=2)

for(j=1;j<=n;j++)

count++;

- A. O(log2n)
- B. O(n)
- C. O(nlog2n)
- D. O(n2)

入手点： 外层和内层分开算，再乘起来：外层跑几轮，内层每轮跑几次，两数相乘就是总量级。

专有名词从零解释：

- 先想“时间复杂度”：衡量程序快慢的一个粗略尺子，只看“问题规模变大时，工作量跟着怎么涨”，不抠常数。
- 再想“大 O 记号”：比如 O(n) 表示工作量跟着 n 线性涨，O(log n) 表示跟着 n 的对数涨，O(n log n) 表示 n 乘上 n 的对数。
- 再想“外层循环、内层循环”：外层跑一轮，内层就要从头到尾跑一遍；所以总次数是外层轮数乘以内层每轮次数。

考查知识点：

- 先想外层跑几轮：外层变量每次翻倍，从 1 开始翻到 n，要翻 log n 次（2 的多少次方等于 n，就是 log n）。
- 再想内层每轮跑几次：内层每次遍历到 n，就是 n 次。
- 再想总量级：外层 log n 轮，每轮内层 n 次，总共 n 乘 log n，就是 O(n log n)，对应 C。
- 拿生活例子验算：外层是“每周翻一倍的周数”，内层是“每天固定做的 n 件事”；周数是 log n，每周 n 件事，总共 n 乘 log n 件事。
- 边界情况：如果外层不是翻倍、而是每次加 1，外层就是 n 轮，总量级就变成 n 乘 n；本题外层是翻倍，所以是 log n，不是 n。
- 易混对子：不要把“外层翻倍”当成“外层只跑几次就不管了”——翻倍也要翻 log n 次；也不要把内层和外层加起来（log n 加 n），循环嵌套是乘法，不是加法。

正确选项为什么对： 外层 log n 轮乘以内层每轮 n 次，等于 n log n，就是 C，选 C。

错误选项为什么错＋举例：

- A（对数阶）：只算了外层的 log n，漏了内层每轮的 n 次。教训：嵌套循环要乘起来，不能只算一层。
- B（线性阶）：只算了内层的一轮 n 次，漏了外层要跑 log n 轮。教训：内层每轮都要跑，外层跑几轮就要乘几倍。
- D（平方阶）：把外层也当成了 n 轮，n 乘 n 得 n 方；外层是翻倍，只有 log n 轮，不是 n 轮。教训：翻倍和加 1 的轮数不一样。

笔记本一句话： 第 1 题＋嵌套循环量级：外层翻倍是 log n 轮，内层每轮 n 次，相乘得 n log n。

【图像】



外层 k 翻倍共 log2n 轮、每轮内层 n 次、总量为 n 乘 log2n 的示意图

【定位】 数据结构 · 第1章 绪论 · 嵌套循环的时间复杂度

【易混对比】

- "外层翻倍"和"外层加1"的区别：翻倍是 k=1,2,4,8…这样跑，共 log2n 轮；加1是 k=1,2,3…这样跑，共 n 轮。轮数不同，相乘后的量级就不同。
- "嵌套循环"和"顺序循环"的区别：嵌套是外层轮数乘以内层每轮次数；顺序是两段各自算完取大者。本题两层嵌套，只能乘，不能加。

外层 log2n 轮，内层每轮 n 次，相乘得 n×log2n，是 O(n log2n)，选 C。

【坑位】

看到"k*=2"先数外层轮数，别凭直觉写 n。

顺手把外层也当成 n 轮，会乘出 n2 误选 D；只数外层又会漏掉内层误选 A。错在没把"轮数"和"每轮次数"分开数。正确做法：先单独数外层轮数（翻倍→log2n），再单独数内层每轮次数（n），最后相乘写量级。

【套路】

- 第一步，数外层轮数：看外层变量怎么变，翻倍就是 log2n 轮。
- 第二步，数内层每轮次数：内层 j 从 1 到 n，就是 n 次。
- 第三步，相乘写量级：轮数×每轮次数=n×log2n。

轮数乘以每轮次数，乘完再写大 O。

【类比】

像办流水席：外层是"翻倍增加的轮数"（log2n 轮），每轮固定上 n 道菜；总共上的菜数是轮数×n。术语贴回去：轮数就是"外层循环次数"，每轮的 n 道菜就是"内层每轮执行次数"，总量级就是两者相乘。

【口诀】 外层翻倍 log 轮，内层 n 次乘起来。

【考频】 「时间复杂度」在 2009–2022 年解析中出现 12 次 · 高频

第 2 题 · 中缀表达式转后缀表达式时栈内元素的变化（正确答案：B）

题干与选项：

假设栈初始为空，将中缀表达式 a/b+(c*d-e*f)/g 转换为等价的后缀表达式的过程中， 当扫描到 f 时，栈中的元素依次是 。

- A. + (* -
- B. + (- *
- C. / + (* - *
- D. / + - *

入手点： 记住运算符栈的几条规矩，再按表达式从左到右走：数字直接输出，运算符按优先级决定进栈还是弹栈，括号单独处理。

专有名词从零解释：

- 先想“中缀表达式”：平时人写的算式，运算符在两个数中间，比如 a 加 b 乘 c。
- 再想“后缀表达式”：运算符写在两个数后面的算式，不用括号也能算，比如 a b c 乘加。
- 再想“运算符栈”：转换过程中临时放运算符的一个栈；栈底是最先放进去的，栈顶是最后放进去的。
- 再想“优先级”：乘除比加减高；左括号进栈后优先级算最低，这样括号里的运算符才能顺利进栈；右括号一来，就把栈里的运算符一直弹到左括号为止。

考查知识点：

- 先想转换的规矩：读到数字，直接写到输出里；读到运算符，看栈顶：如果栈顶运算符优先级不比它低，就先弹出来，再把它压进去；读到左括号，直接压进去；读到右括号，把栈里的运算符一直弹到左括号，左括号也弹掉。
- 再想本题的关键一步：按这套规矩一步步走，扫描到 f 时，栈里从底到顶依次是加号、左括号、减号、乘号，写成 +(-*，就是 B。
- 再想为什么是这个顺序：读到加号时，栈里的斜杠优先级比加号高，先弹到输出里，再由加号进栈，所以加号在最底下；左括号后进，压在加号上面；减号在括号里，压在左括号上面；乘号优先级比减号高，直接压在最上面；所以从底到顶是 +(-*。
- 拿小例子验算：算式 a 加 b 乘 c，读到乘号时栈里是加号，乘号优先级高，直接压进去，栈是加号、乘号；读到右括号时，栈里的运算符要弹到左括号为止；规矩就是这样一步步推出来的。
- 边界情况：左括号进栈后优先级算最低，是为了让括号里的运算符都能进栈；如果左括号优先级不算最低，括号里的低优先级运算符就进不了栈，转换就错了。
- 易混对子：不要把“栈从底到顶”和“栈从顶到底”搞反——题目给的顺序是从底到顶；也不要把“输出的后缀序列”当成“栈里的内容”——输出的是数字和已经弹出的运算符，栈里是还没弹出的运算符。

正确选项为什么对： 按转换规矩一步步走，扫描到 f 时栈从底到顶是 +(-*，就是 B，选 B。

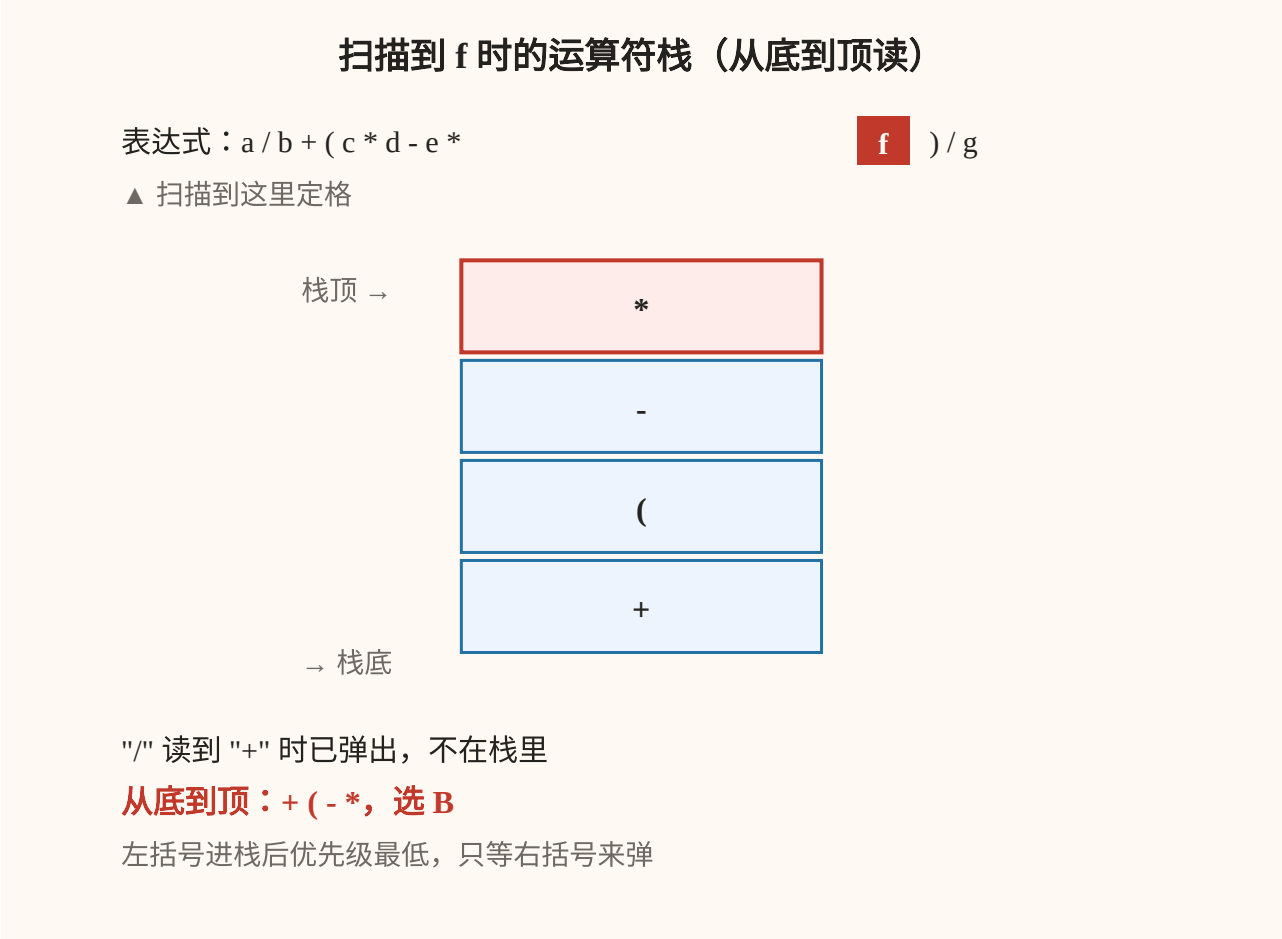
错误选项为什么错＋举例：

- A（+(∗−)：减号和乘号上下写反了。括号里先读到减号、后读到乘号，乘号后进栈、应该在减号上面；A 把乘号写在减号下面，顺序反了。教训：后进栈的在上面，先进栈的在下面，不能凭印象排。

- C（/+(∗−∗)：底下的斜杠不应该还在。读到加号时，栈顶的斜杠优先级比加号高，已经弹到输出里了；加号进栈时栈底是加号，不是斜杠；而且 C 里括号里的顺序也不对（乘号跑到减号下面，还多了一个乘号）。教训：优先级高的先弹出去，弹出去的就不再出现在栈里。
- D（/+−∗)：同样底下的斜杠不应该还在；还把左括号丢了——左括号从读到它起一直在栈里，要等右括号来才弹，扫描到 f 时右括号还没来，左括号不能丢。教训：括号要成对处理，左括号没遇到右括号之前一直在栈里。

笔记本一句话： 第 2 题＋中缀转后缀：运算符按优先级进栈出栈，左括号进栈后优先级最低；扫描到 f 时栈从底到顶是 +(-*。

【图像】



扫描到 f 时运算符栈从底到顶为 +、(、-、* 的栈示意图，斜杠已提前弹出

【定位】 数据结构 · 第3章 栈和队列 · 中缀转后缀过程中运算符栈的状态

【易混对比】

- "左括号进栈"和"普通运算符进栈"的区别：左括号进栈后优先级按最低算，只管占位置、不拦任何运算符进栈；普通运算符进栈要跟栈顶比优先级，不比栈顶低才能直接进。
- "栈从底到顶"和"栈从顶到底"的区别：同一时刻两种读法顺序正好相反；本题选项是按从底到顶给的，先对齐方向再选字母。

扫到 f 的瞬间，栈从底到顶是 +、(、-、*，斜杠在读到 + 时已被弹出，选 B。

【坑位】

括号里的栈顶是左括号，它不参与优先级比较，别拿它跟减号比。

f 之前依次读到 c * d - e * f：乘号要跟减号比优先级，乘号更高所以直接压栈，这一步没问题。容易错的是看到左括号在栈里，就以为"乘号要先跟左括号比"——不用比，左括号进栈后优先级最低，拦不住任何运算符。正确做法：比较优先级只跟左括号上面的运算符比；左括号只等右括号来弹它。

【套路】

- 第一步，写扫描顺序：a / b + (c * d - e * f，从左往右走到 f。
- 第二步，按规矩走栈：数字直接输出；读到 + 时栈顶的斜杠优先级更高，先弹出再压 +；左括号直接压；乘号比减号优先级高，直接压。
- 第三步，定格读栈：扫到 f 的瞬间，从底到顶依次读出 +、(、-、*。

先过一遍扫描顺序，再按规矩走栈，定格时按题目方向读。

【类比】

像安检排队：+、-、* 是旅客，按"优先级"排队进安检门；左括号是隔离带——进了隔离带（括号）后，里面的旅客按自己的优先级排，隔离带本身不跟旅客比高低。术语贴回去：隔离带就是"进栈的左括号"，旅客排队的规矩就是"优先级高的运算符先进栈"。

【口诀】 数字输出符比级，左括进栈最末位；右括一到全弹出。

【考频】 「中缀转后缀」在 2009–2022 年解析中出现 3 次 · 中频

第 3 题 · 循环双端队列的判空与判满条件（正确答案：A）

题干与选项：

循环队列放在一维数组 A[0…M-1]中，end1 指向队头元素，end2 指向队尾元素的后 一个位置。假设队列两端均可进行入队和出队操作，队列中最多能容纳 M-1 个元素。初始 时为空。下列判断队空和队满的条件中，正确 的是 。

- A. 队空：end1 == end2； 队满：end1 == (end2+1)mod M
- B. 队空：end1 == end2； 队满：end2 == (end1+1)mod (M-1)
- C. 队空：end2 == (end1+1)mod M； 队满：end1 == (end2+1)mod M
- D. 队空：end1 == (end2+1)mod M； 队满：end2 == (end1+1)mod (M-1)

入手点： 先把两个指针的含义定下来，再想空和满各是什么意思：空是里面一个元素都没有，满是里面已经装不下新的了。

专有名词从零解释：

- 先想“循环队列”：用一个数组存队列，数组的头和尾连起来，队尾走到数组末尾后，下一个位置又回到数组开头，像一个圈。
- 再想“双端队列”：两头都能进、两头都能出的队列；本题的双端队列用两个指针来管。
- 再想“队头指针 end1、队尾指针 end2”：end1 指着队头元素，end2 指着队尾元素的后一个位置（也就是下一个要放新元素的位置）。
- 再想“牺牲一个位置”：循环队列为了分清“空”和“满”，故意少用一个位置，最多只存 M-1 个元素；不牺牲的话，空和满都是两个指针相等，分不清。

考查知识点：

- 先想判空：队列里一个元素都没有时，队头和队尾后一位置重合，就是 end1 等于 end2。
- 再想判满：队列里已经存了 M-1 个元素，再存一个就满了；这时队尾后一位置再往后移一位，就正好追上队头，就是 end1 等于 end2 后移一位再对 M 取模，写成 end1＝(end2＋1)%M。
- 再想为什么空和满不会撞：空是两个指针相等，满是两个指针差一位（队尾后一位置的下一位是队头）；牺牲一个位置后，空和满的指针关系不一样，分得清。
- 拿生活例子验算：一个圈形停车场有 M 个车位，只停 M-1 辆车；入口指针和出口指针重合，说明一场空；入口指针的下一位是出口指针，说明只剩一个空位不能停了，就是满了。
- 边界情况：初态为空时，两个指针相等；如果数组大小是 M，最多存 M-1 个元素，第 M 个位置永远空着，就是牺牲掉的那一个。
- 易混对子：不要把“队尾指针”当成“指着队尾元素”——本题 end2 指的是队尾元素的后一个位置，不是队尾元素本身；也不要把取模的 M 写成 M-1——数组大小是 M，取模就是 M。

正确选项为什么对： 空是 end1 等于 end2，满是 end1 等于 end2 后移取模 M，就是 A，选 A。

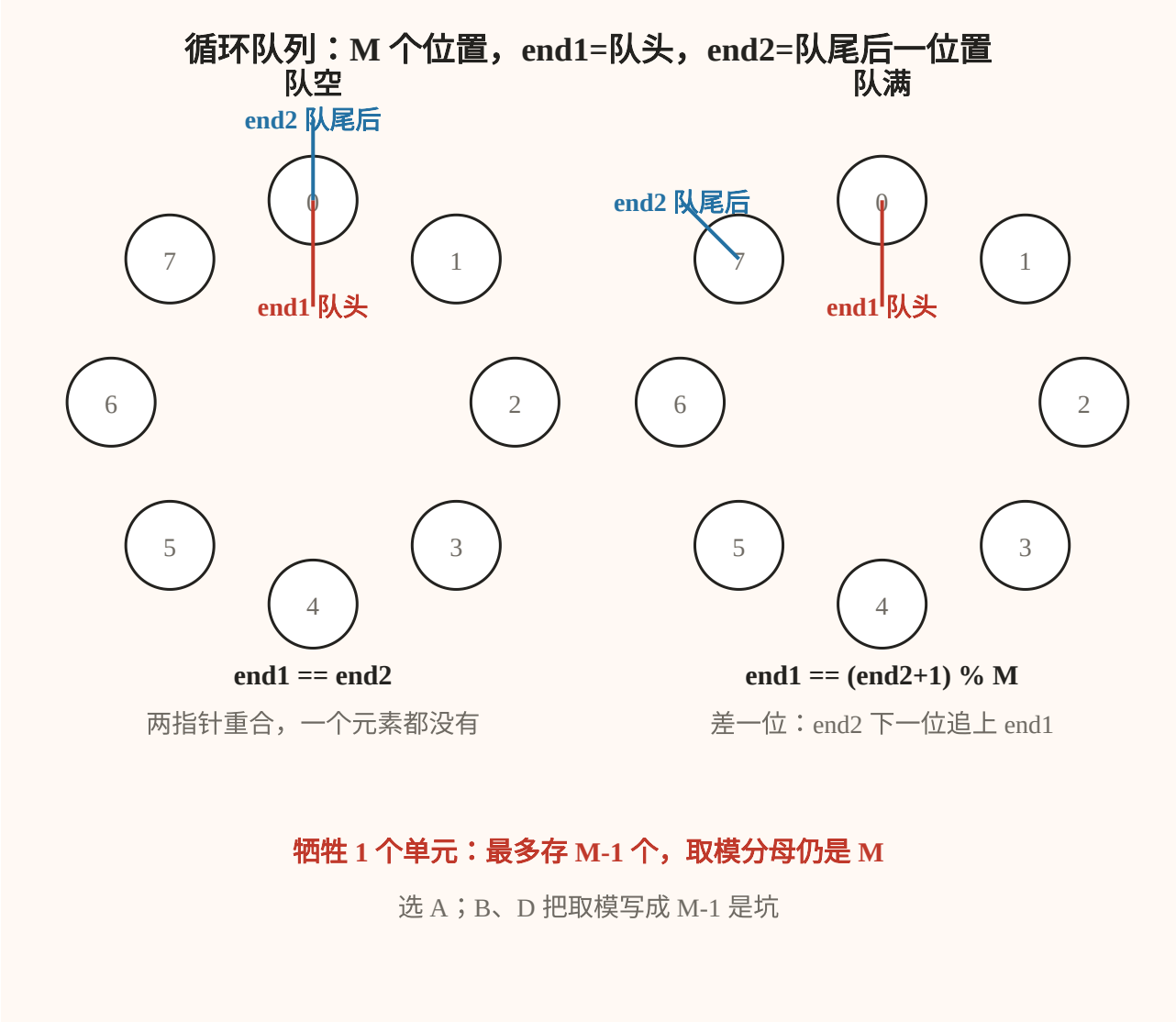
错误选项为什么错＋举例：

- B（空同 A，满为 end2 等于 end1 后移取模 M-1）：满条件把两个指针颠倒了，而且取模写成了 M-1；数组大小是 M，取模应该是 M，指针关系应该是 end1 等于 end2 的下一位。教训：满条件的指针方向和取模数都要对。

- C（空为 end2 等于 end1 后移取模 M，满同 A 的满条件）：空条件写成了满条件——end2 的下一位是 end1，说明队列已经满了，不是空了；空应该是两个指针相等。教训：空和满的式子不能互换。
- D（空为 end1 等于 end2 后移取模 M，满同 B 的满条件）：空条件写成了满条件，满条件又错在取模 M-1 和指针颠倒；两个条件都错了。教训：先定指针含义，再写空和满的式子。

笔记本一句话： 第 3 题＋循环双端队列：end1 队头、end2 队尾后一位置；空是两指针相等，满是 end1＝(end2＋1)%M。

【图像】



M 个槽位的环形队列：end1 指队头、end2 指队尾后一位置，标出队空两指针重合、队满差一位

【定位】 数据结构 · 第3章 栈和队列 · 循环队列的判空与判满

【易混对比】

- "end2 指队尾后一位置"和"end2 指队尾元素"的区别：本题 end2 是下一个要放元素的位置；如果定义成指队尾元素，判满式子会跟着变，读题先看清指针定义再套公式。
- "判空"和"判满"的区别：空是两指针重合；满是 end2 的下一位追上 end1（差一位），牺牲一个单元后两者不撞车。

队空是 end1==end2，队满是 end1==(end2+1)%M，选 A。

【坑位】

取模的分母是数组长度 M，不是"最多存的个数"M-1。

看到"最多容纳 M-1 个元素"，顺手把取模也写成 M-1（B、D 的坑）。错在把"存几个"和"数组多大"混了：取模是对数组长度取，数组有 M 个位置，分母只能是 M；M-1 只是"满"的概念，不影响取模。正确做法：先默写"数组长度 M"，取模分母写 M；再单独想"最多存几个"，那个只用来理解"为什么满是差一位"。

【套路】

- 第一步，定指针含义：end1=队头元素，end2=队尾元素的后一个位置。
- 第二步，写队空：一个元素都没有，两指针重合，end1==end2。
- 第三步，写队满：end2 再往后一位追上 end1，即 end1==(end2+1)%M。

先定指针含义，再写空和满：空重合、满差一、取模看数组长。

【类比】

像环形停车场有 M 个车位、只停 M-1 辆：入口指针和出口指针重合，说明一场空（队空）；入口指针的下一位就是出口指针，说明只剩最后一个空位、不能再进了（队满）。术语贴回去：故意空着的那个车位，就是"牺牲一个单元来区分队空和队满"。

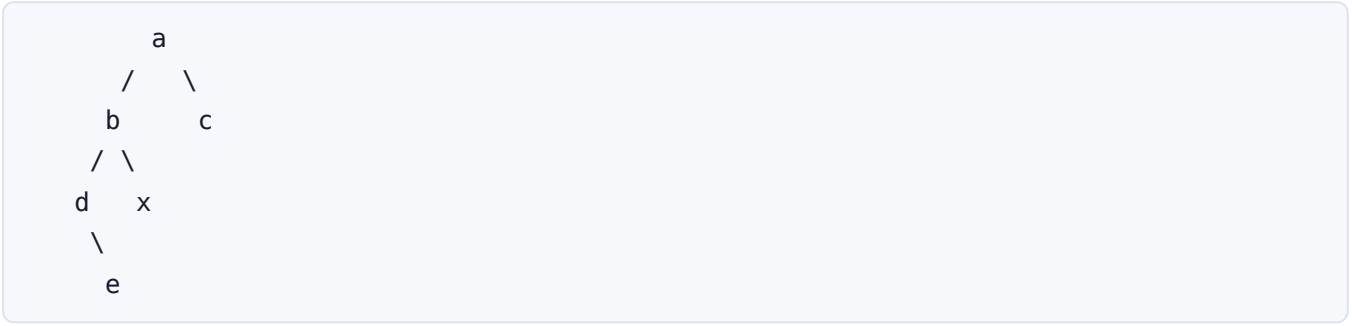
【口诀】 空相等来满差一，取模分母认准 M。

【考频】 「循环队列」在 2009–2022 年解析中出现 2 次 · 低频

第 4 题 · 中序线索二叉树中结点 x 的左、右线索指向（正确答案：D）

题干与选项：

若对如下的二叉树进行中序线索化，则结点 x 的左、右线索指向的结点分别是 。



- A. e、c
- B. e、a
- C. d、c
- D. b、a

入手点： 先写出这棵树的中序遍历序列，再在序列里找 x 的前一个和后一个：前一个就是左线索指的，后一个就是右线索指的。

专有名词从零解释：

- 先想“中序遍历”：按“左子树、根、右子树”的顺序走一棵树，把结点排成一串；这串顺序就是中序序列。
- 再想“线索二叉树”：普通二叉树里，叶子结点的左右指针是空的，白白浪费了；线索二叉树把这些空指针利用起来，让它们指向“中序序列里的前一个结点”或者“后一个结点”，这样找前驱后继就不用再遍历整棵树了。
- 再想“前驱、后继”：中序序列里，某个结点的前一个叫前驱，后一个叫后继。
- 再想“左线索、右线索”：左指针改成线索时，它指向前驱；右指针改成线索时，它指向后继。

考查知识点：

- 先想这棵树的样子：a 是根，左孩子是 b，右孩子是 c；b 的左孩子是 d，右孩子是 x；d 的右孩子是 e。
- 再想中序序列：按左根右走：d 的左子树没有，d 本身，d 的右子树 e，然后 b，然后 x，然后 a，然后 c；合起来是 d、e、b、x、a、c。
- 再想 x 的前驱和后继：序列里 x 的前一个是 b，后一个是 a；所以 x 的左线索指向 b，右线索指向 a。
- 再想为什么 x 的左右指针都是线索：x 没有左孩子，也没有右孩子，两个指针本来都是空的，正好改成线索；如果 x 有孩子，指针就指孩子，不能改成线索。
- 拿生活例子验算：排队时，每个人的左手牵着前一个人、右手牵着后一个人；x 的前一个人是 b，后一个人是 a，所以 x 左手牵 b、右手牵 a。
- 边界情况：如果 x 是中序序列的第一个，它就没有前驱，左线索就是空的；如果 x 是最后一个，它就没有后继，右线索就是空的；本题 x 在中间，前驱后继都有。
- 易混对子：不要把“左孩子”当成“左线索”——左孩子是树结构里的孩子，左线索是中序里的前驱；x 没有左孩子，所以左指针才改成线索指前驱。

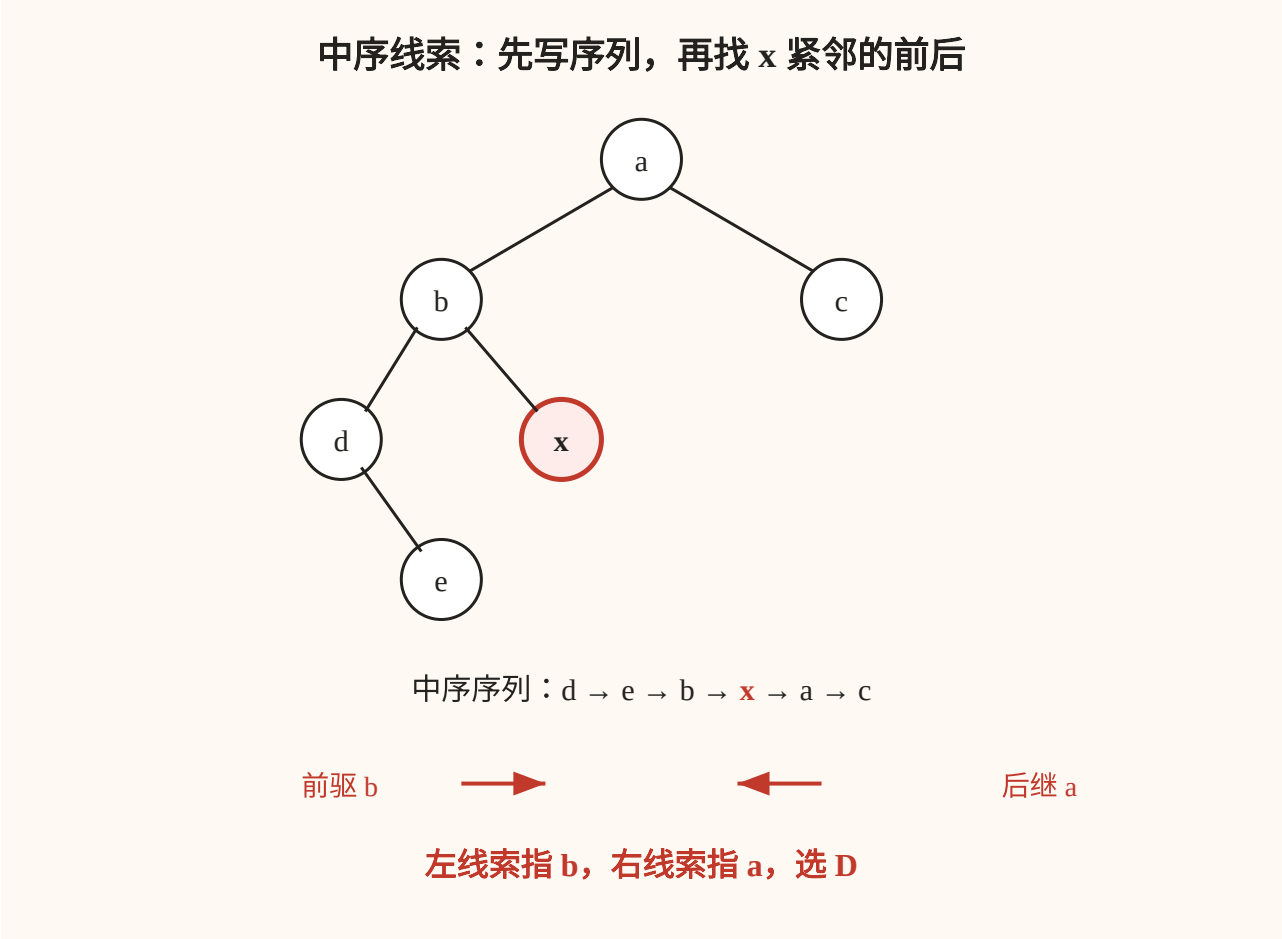
正确选项为什么对： x 的前驱是 b、后继是 a，左线索指向 b、右线索指向 a，就是 D，选 D。

错误选项为什么错＋举例：

- A（左线索指向 e，右线索指向 c）：e 是 d 的右孩子，在中序序列里排在 b 前面，不是 x 的直接前驱；c 是 a 的右孩子，排在 a 后面，不是 x 的直接后继。教训：前驱后继是序列里紧挨着的那个，不能隔着结点。
- B（左线索指向 e，右线索指向 a）：右线索对了，后继确实是 a；但左线索错了，e 不是 x 的前驱，前驱是 b。教训：左右线索要一个一个验，对一个不算对。
- C（左线索指向 d，右线索指向 c）：d 是 b 的左孩子，在序列里排在 e 前面，不是 x 的前驱；c 也不是 x 的后继。教训：前驱是 b、后继是 a，不能拿树里的亲戚关系代替序列里的前后关系。

笔记本一句话： 第 4 题＋中序线索：先写中序序列，再找前驱后继；x 的前驱是 b、后继是 a，左线索指 b、右线索指 a。

【图像】



二叉树的中序序列为 d e b x a c，x 的前驱是 b、后继是 a 的示意图

【定位】 数据结构 · 第5章 树与二叉树 · 中序线索二叉树的线索指向

【易混对比】

- "左线索"和"左孩子"的区别：左孩子是树结构里的亲子关系；左线索是中序序列里紧挨着的前一个结点。x 没有左孩子，左指针才改成线索指前驱。
- "前驱"和"树里看着近的结点"的区别：前驱是中序序列里 x 的前一位（b），不是树图里看着离 x 近的 d 或 e；序列的前后和树图的左右不是一回事。

中序序列是 d、e、b、x、a、c，x 的前驱是 b、后继是 a，左线索指 b、右线索指 a，选 D。

【坑位】

前驱后继认的是序列里"紧挨着"的那一个，隔着结点的不算。

看到 e 在 x 左边附近（d 的右孩子），顺手当成前驱（A、B 的坑）；看到 c 在右边像后继（A 的坑）。错在拿树形位置代替了序列顺序：e 在序列里排在 b 前面，中间隔着 b，不是紧挨着 x。正确做法：先完整写出中序序列，再在序列里找 x 左右紧邻的两个字母，不要看树图猜。

【套路】

- 第一步，写中序序列：按左根右走完全树，本题得 d、e、b、x、a、c。
- 第二步，找紧邻：x 的前一位是前驱 b，后一位是后继 a。
- 第三步，贴标签：左线索→前驱，右线索→后继。

先写序列，再找紧邻，最后贴左右标签。

【类比】

像全班按学号排队：x 的左手牵的是学号紧挨着的前一位同学（b），右手牵的是紧挨着的后一位（a），而不是座位坐得近的同学。术语贴回去：这条队伍就是"中序遍历序列"，牵手就是"线索"。

【口诀】 线索不认树亲戚，只认序列紧邻居。

【考频】 「线索二叉树」在 2009–2022 年解析中出现 3 次 · 中频

第 5 题 · 森林转换为二叉树后叶结点个数的对应关系（正确答案：C）

题干与选项：

将森林 F 转换为对应的二叉树 T，F 中叶结点的个数等于 。

- A. T 中叶结点的个数
- B. T 中度为 1 的结点个数
- C. T 中左孩子指针为空的结点个数
- D. T 中右孩子指针为空的结点个数

入手点： 先把转换的规矩记牢：二叉树里的左孩子是谁、右孩子是谁；再看森林里的叶结点转过去后长什么样。

专有名词从零解释：

- 先想“森林”：好几棵树放在一起，就叫森林；每棵树都有自己的根。
- 再想“孩子-兄弟表示法”：把森林转成二叉树的办法——每个结点的左孩子是它在森林里的第一个孩子，右孩子是它在森林里的下一个兄弟。
- 再想“叶结点”：没有孩子的结点；森林里的叶结点，就是在森林里一个孩子都没有的结点。
- 再想“左孩子指针为空”：某个结点的左孩子位置是空的，说明它没有左孩子。

考查知识点：

- 先想森林里的叶结点转过去后是什么样：叶结点在森林里没有孩子，所以它没有“第一个孩子”，转成二叉树后，它的左孩子位置就是空的。
- 再想反过来：二叉树里左孩子为空的结点，在森林里就没有第一个孩子，也就是没有孩子，就是森林里的叶结点。
- 再想一一对应：森林里的叶结点，和二叉树里左孩子为空的结点，一个对一个，个数相等，就是 C。
- 拿生活例子验算：森林里每个家庭的第一个孩子站左边；没有孩子的家庭，左边就空着；数一数左边空着的家庭数，正好等于没有孩子的家庭数。
- 边界情况：森林里每棵树的根之间是兄弟关系，所以第二棵树的根在二叉树里是第一棵树根的右孩子；根有没有左孩子，只看它在森林里有没有孩子，和它是第几棵树的根没关系。
- 易混对子：不要把“二叉树里的叶结点”和“左孩子为空的结点”当成一回事——二叉树里的叶结点是左右孩子都空的结点，比左孩子为空的结点少；左孩子为空、但右孩子有兄弟的结点，在二叉树里不是叶结点，在森林里却是叶结点。

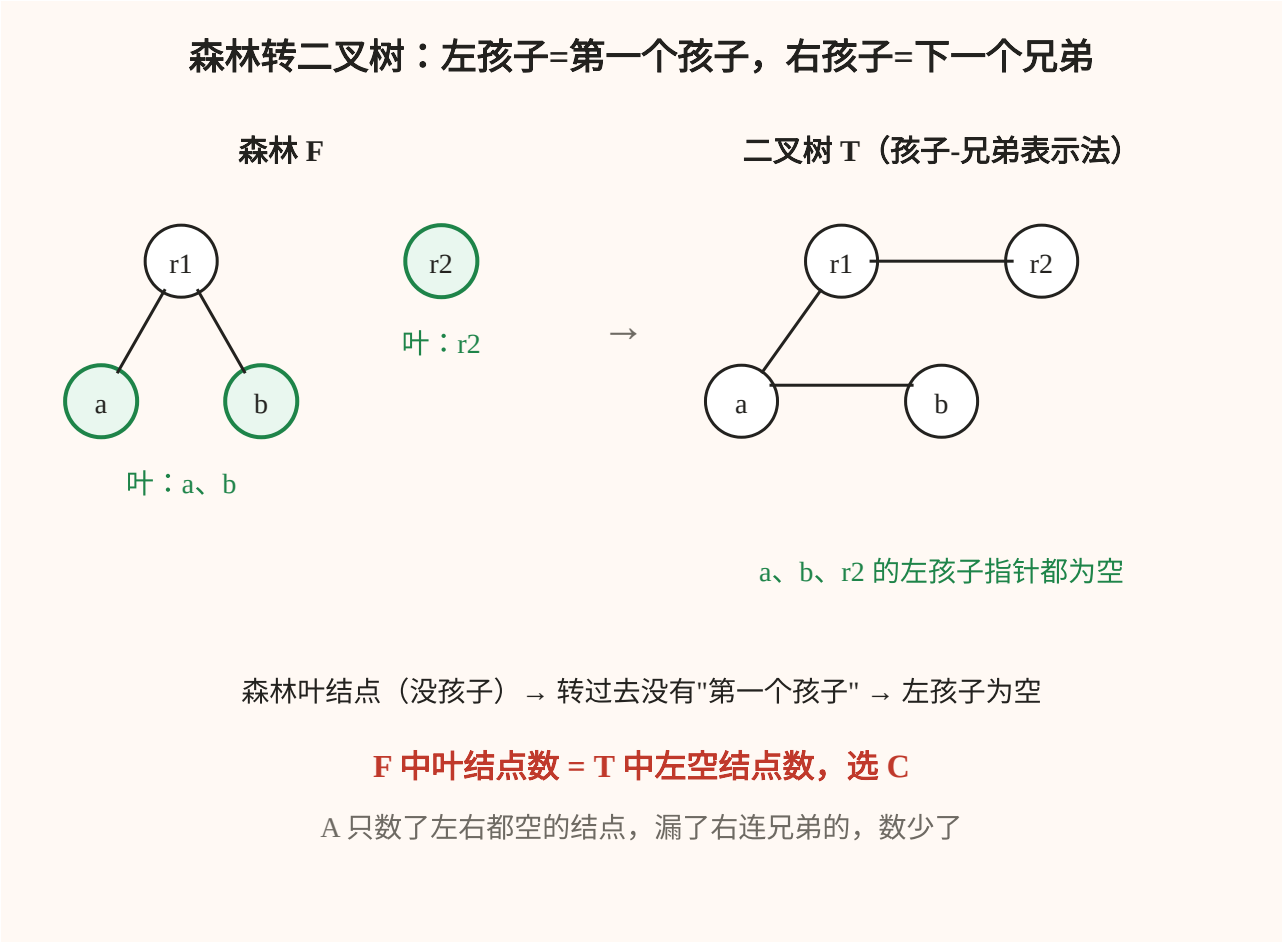
正确选项为什么对： 森林里的叶结点正好对应二叉树里左孩子为空的结点，个数相等，选 C。

错误选项为什么错＋举例：

- A（T 的叶结点数）：T 的叶结点是左右孩子都空的结点；森林里的叶结点转过去后，右孩子可能有兄弟（下一个兄弟），所以不一定是 T 的叶结点；A 数少了。教训：左空和左右都空是两回事。
- B（T 中度为 1 的结点数）：度为 1 的结点是只有一个孩子的结点（左或右）；左孩子为空的结点，右孩子可能有、也可能没有，度可能是 0 也可能是 1；两者不是一回事。教训：度数和左空没有对应关系。
- D（T 中右孩子指针为空的结点数）：右孩子是下一个兄弟；右孩子为空，说明这个结点没有下一个兄弟，和它在森林里有没有孩子没关系。教训：右孩子对应兄弟，不是孩子。

笔记本一句话： 第 5 题＋森林转二叉树：左孩子是第一个孩子；森林叶结点没有孩子，所以左孩子为空，个数就是 T 里左空的结点数。

【图像】



森林转二叉树（孩子-兄弟表示法）：叶结点转过去左孩子指针为空的对应示意图

【定位】 数据结构 · 第5章 树与二叉树 · 森林与二叉树的转换

【易混对比】

- "T 中左孩子为空的结点"和"T 中的叶结点"的区别：叶结点要求左右孩子都空；左空但右孩子连着兄弟的结点是左空结点、不是叶结点，A 数的比 C 少。
- "左孩子"和"右孩子"的区别：左孩子是森林里的第一个孩子，右孩子是下一个兄弟；问"叶"（没孩子）只跟左孩子有关，跟右孩子无关，所以 D 不对。

森林叶结点没有孩子→转成二叉树后没有第一个孩子→左孩子指针为空，两边一一对应，选 C。

【坑位】

"叶结点"有两个定义，别串：森林的叶和二叉树的叶不是同一批。

看到"叶结点"就想"左右都空"，直接选 A（T 中叶结点数）。错在：森林叶结点转过去后，右边可能连着兄弟（右孩子不空），它在 T 里不是叶，但在森林里是叶。正确做法：题目问的是森林的叶，先翻译成"在森林里没孩子"，再用转换规则翻译成"在 T 里左孩子空"，别跳过中间这一步。

【套路】

- 第一步，定转换规则：左孩子=第一个孩子，右孩子=下一个兄弟。
- 第二步，翻译"叶"：森林叶=没孩子=没第一个孩子=左孩子空。
- 第三步，数对应：两边一个对一个，个数相等。

左孩子是第一个孩子；没孩子就是左空，个数对过去。

【类比】

像每家派第一个孩子站左边集合：没孩子的家庭左边没人站；数左边空着的家庭数，正好等于没孩子的家庭数。术语贴回去：站左边就是"左孩子指针"，没孩子就是"森林的叶结点"。

【口诀】 森林叶结点，转树看左空。

【考频】 「森林转二叉树」在 2009–2022 年解析中出现 5 次 · 中频

第 6 题 · 前缀码的判定（正确答案：D）

题干与选项：

5 个字符有如下 4 种编码方案，不是 前缀编码的是 。

- A. 01,0000,0001,001,1
- B. 011,000,001,010,1
- C. 000,001,010,011,100
- D. 0,100,110,1110,1100

入手点： 一个一个组查：组里有没有一个码是另一个码的前面一段；只要找到一对这样的，就不是前缀码。

专有名词从零解释：

- 先想“编码”：给每个字符编一个二进制串，比如 a 编成 0，b 编成 10。
- 再想“前缀”：一个串是另一个串的前面一段，比如 110 是 1100 的前面三位，110 就是 1100 的前缀。
- 再想“前缀编码”：一组编码里，没有任何一个码是另一个码的前缀；这样解码时不用分隔符，读到一个码就能确定是一个字符，不会含糊。

考查知识点：

- 先想 A 组（01、0000、0001、001、1）：01 和 001 前两位不一样（01 和 00），不是前缀关系；0000 和 0001 前三位相同、第四位不同，不是前缀关系；1 和其他码的开头都不一样（其他码都以 0 开头）；所以 A 组是前缀码。
- 再想 B 组（011、000、001、010、1）：011 和 010 前两位相同、第三位不同，不是前缀关系；000 和 001 第三位不同，不是前缀关系；1 和其他码开头不一样；所以 B 组是前缀码。
- 再想 C 组（000、001、010、011、100）：五个码都是 3 位，各不相同；长度相等的不同码，不可能是对方的前缀；所以 C 组是前缀码。
- 再想 D 组（0、100、110、1110、1100）：110 是 1100 的前面三位，110 就是 1100 的前缀；所以 D 组不是前缀码，就是答案。
- 拿生活例子验算：电话号码里，如果一个人的号码是另一个人号码的前面一段，打电话时就分不清打给谁；前缀编码就是要求没有这种号码。
- 边界情况：长度相等的码，只要不相等，就不可能是前缀关系；只有长度不等的码，才需要检查短的是不是长的前面一段。
- 易混对子：不要把“开头相同”当成“前缀关系”——开头相同只是前几位一样，要短的那个全部等于长的那个的前面一段，才是前缀关系；比如 011 和 010 开头相同，但不是前缀关系。

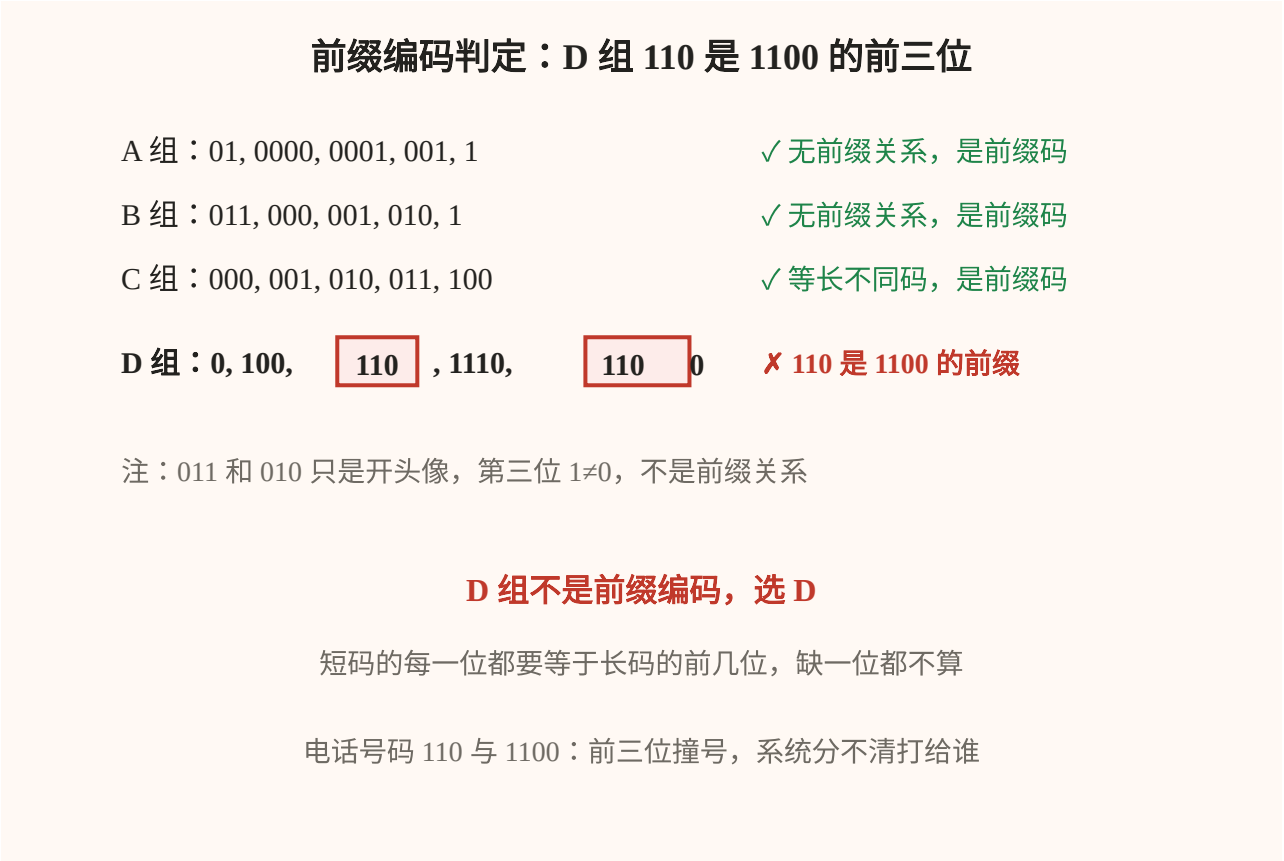
正确选项为什么对： D 组里 110 是 1100 的前缀，所以 D 组不是前缀码，题目问“不是”的那一组，选 D。

错误选项为什么错＋举例：

- A（01、0000、0001、001、1）：逐对查，没有一个码是另一个码的前缀，是前缀码；题目问“不是”，所以 A 不对。教训：要一对一对查，不能凭感觉。
- B（011、000、001、010、1）：逐对查，也没有前缀关系，是前缀码；所以 B 不对。教训：开头相同的码不一定是前缀关系。
- C（000、001、010、011、100）：五个码等长、各不相同，一定是前缀码；所以 C 不对。教训：等长不同码不可能互为前缀。

笔记本一句话： 第 6 题＋前缀码：组里没有一个码是另一个码的前面一段，才是前缀码；D 组里 110 是 1100 的前缀，所以不是。

【图像】



D 组编码中 110 是 1100 的前缀、其余三组无前缀关系的对比示意图

【定位】 数据结构 · 第5章 树与二叉树 · 前缀编码的判定

【易混对比】

- "开头相同"和"前缀关系"的区别：011 和 010 前两位相同，但要短码的每一位都等于长码的前几位才算前缀；只看"开头像"会误判（B 的坑）。
- "等长码"和"不等长码"的区别：长度相等的两个不同码不可能互为前缀（C 组一眼过）；只有长短不一的组，才需要逐位比。

D 组里 110 正好是 1100 的前三位，110 是 1100 的前缀，D 不是前缀编码，选 D。

【坑位】

检查前缀要"短码的每一位都对上长码的前几位"，缺一位都不算。

看到 011、010 开头都是 01 就以为是前缀关系——其实要 011 的三位逐位等于 010 的前三位，第三位 1 不等于 0，不算。错在把"开头两像"当成了"全部对上"。正确做法：先按长度分组，等长的不同码直接跳过；只对长短不一的组，拿短码去对长码的前几位逐位比对。

【套路】

- 第一步，按长度分组：等长不同码不可能互为前缀，直接过。
- 第二步，短对长逐位比：短码的每一位都要等于长码对应位置。
- 第三步，找反例：找到一对就算"不是"，D 组 110/1100 中招。

等长先过，短长逐位比，找到一对就出局。

【类比】

像电话号码：110 和 1100，前三位完全一样，拨 110 时系统分不清你是要打给 110 还是打给 1100 开头的号码；前缀编码就是不允许这种"一个号码是另一个号码的前半段"。术语贴回去：号码就是"编码"，前半段撞号就是"前缀关系"。

【口诀】 短码若为长码头，此组必定非前缀。

【考频】 「前缀编码」在 2009–2022 年解析中出现 5 次 · 中频

第 7 题 · 有向图的拓扑排序（正确答案：D）

题干与选项：

对如下所示的有向图进行拓扑排序，得到的拓扑序列可能是 。

> 注：有向图以原卷为准（结点 1–6；原卷图略）

- A. 3,1,2,4,5,6
- B. 3,1,2,4,6,5
- C. 3,1,4,2,5,6
- D. 3,1,4,2,6,5

入手点： 一条边一条边验：每条边的起点必须排在终点前面；只要有一条边违反，整个序列就不行。

专有名词从零解释：

- 先想“有向图”：顶点之间用带箭头的边连起来，箭头从起点指向终点，表示“起点要在终点前面”。
- 再想“拓扑序列”：把所有顶点排成一排，使得每条边的起点都排在终点前面；这样的排法就叫拓扑序列。
- 再想“可能”：题目问“可能”，不是“一定”——只要这个序列满足所有边的先后要求，就算可能；不用管它是不是按某种特定算法排出来的。

考查知识点：

- 先想边集（8 条）：3→1、1→2、1→4、4→2、2→5、3→6、4→6、6→5。
- 再想 A（3,1,2,4,5,6）为什么不行：查 4→2，4 排在第 4 位，2 排在第 3 位，4 在 2 后面，违反；再查 6→5，6 排在第 6 位，5 排在第 5 位，6 在 5 后面，也违反；所以 A 不行。
- 再想 B（3,1,2,4,6,5）为什么不行：查 4→2，4 在 2 后面，违反；所以 B 不行。
- 再想 C（3,1,4,2,5,6）为什么不行：查 6→5，6 在 5 后面，违反；所以 C 不行。
- 再想 D（3,1,4,2,6,5）为什么行：逐条验——3→1（3 在 1 前面），1→2（1 在 2 前面），1→4（1 在 4 前面），4→2（4 在 2 前面），2→5（2 在 5 前面），3→6（3 在 6 前面），4→6（4 在 6 前面），6→5（6 在 5 前面），8 条边全满足；所以 D 行。
- 拿生活例子验算：排队时，每条边都是一条“谁要在谁前面”的规矩；一个序列要满足所有规矩，才算排对了；违反一条规矩，整个序列就不行。
- 边界情况：拓扑序列可能不止一个；本题四个选项里，只有一个满足所有边，所以答案唯一；如果两个选项都满足，题目就不会这样出了。
- 易混对子：不要只看“第一个是不是 3”——四个选项都以 3 开头，第一个对了不算对；要 8 条边一条一条验，漏一条就可能把错的当成对的。

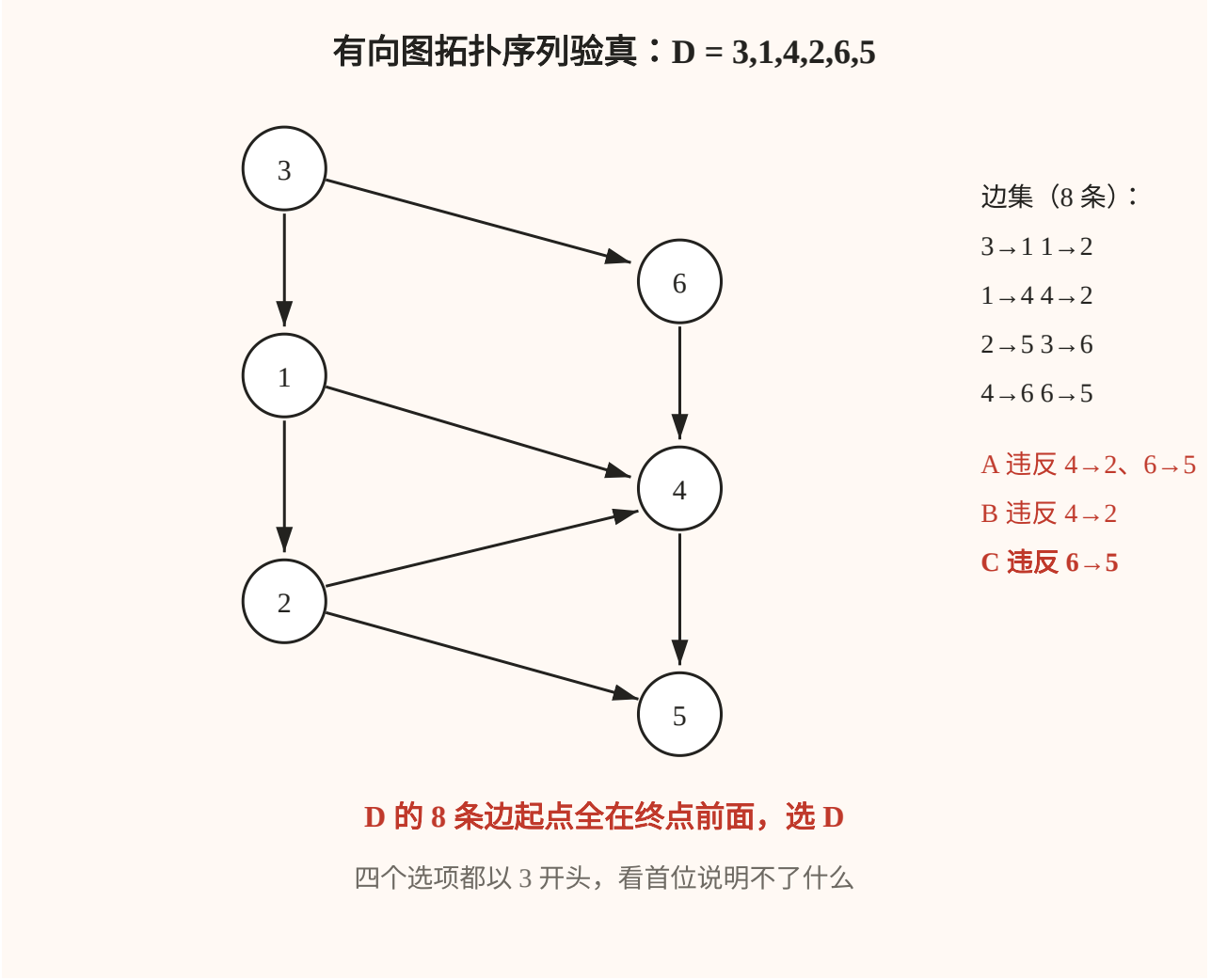
正确选项为什么对： D 的序列满足全部 8 条边的先后要求，是唯一合法的拓扑序列，选 D。

错误选项为什么错＋举例：

- A（3,1,2,4,5,6）：违反 4→2 和 6→5 两条边；教训：每条边都要验，违反一条就不行。
- B（3,1,2,4,6,5）：违反 4→2；教训：2 的前面必须有 4，2 排在 4 前面就是错的。
- C（3,1,4,2,5,6）：违反 6→5；教训：5 的前面必须有 6，5 排在 6 前面就是错的。

笔记本一句话： 第 7 题＋拓扑序列验真：每条边的起点必须在终点前面，一条一条验，违反一条就不行。

【图像】



6 结点 8 条边的有向图，D 序列 3,1,4,2,6,5 使每条边的起点都排在终点前面

【定位】 数据结构 · 第6章 图 · 拓扑序列的合法性验证

【易混对比】

- "可能"和"按某种算法排出"的区别：题目只问"可能是"，只要满足所有边的先后就是合法序列，不用管它是不是教材算法的输出。
- "看首位"和"逐边验"的区别：四个选项都以 3 开头，首位对了说明不了什么；合法性要靠 8 条边一条一条验。

D 的序列让 8 条边的起点全排在终点前面，是四个选项里唯一合法的，选 D。

【坑位】

验边时先列出边集再查，别在脑子里"顺一遍"。

A 看着 3,1,2,4…挺顺，顺手就选了，但 4→2 这条边里 4 被排到了 2 后面；人脑顺读容易漏掉"回头"的边。错在用"读着顺"代替了"逐条验"。正确做法：先把 8 条边列成清单（3→1、1→2、1→4、4→2、2→5、3→6、4→6、6→5），拿着清单一条一条对序列，违反一条就地淘汰。

【套路】

- 第一步，列边集：把图里的有向边全部写出来。
- 第二步，逐条验：每条边查起点是否排在终点前面。
- 第三步，淘汰：违反一条就划掉整个选项。

列边集，逐条验，一违即废。

【类比】

像排演出顺序：每条边是一条"谁必须在谁前面"的死规矩（比如 4 必须在 2 前面）；一个节目单要同时满足所有死规矩，违反一条就得重排。术语贴回去：节目单就是"拓扑序列"，死规矩就是"有向边"。

【口诀】 逐边验真，一违即废。

【考频】 「拓扑排序」在 2009–2022 年解析中出现 7 次 · 中频

第 8 题 · 散列冲突堆积现象对查找的影响（正确答案：D）

题干与选项：

用哈希（散列）方法处理冲突（碰撞）时可能出现堆积（聚集）现象，下列选项中， 会受堆积现象直接影响的是。

- A. 存储效率
- B. 散列函数
- C. 装填(装载)因子
- D. 平均查找长度

入手点： 先想堆积是什么意思，再想它直接拖慢了哪一步：是找数据变慢了，还是别的指标变了。

专有名词从零解释：

- 先想“散列法”：也叫哈希法，给每个关键字算一个位置，存到表里；找的时候再算一次位置，直接去那个位置拿。
- 再想“冲突”：两个不同的关键字算出来同一个位置，只能存一个，另一个要另找地方。
- 再想“堆积”：也叫聚集，冲突多了，不同关键字的存放位置挤成一团；后来再有冲突，挤的地方越来越多，查找时要比较的次数也越来越多。
- 再想“平均查找长度”：找一个关键字，平均要比较几次；比较次数越多，查找越慢。

考查知识点：

- 先想堆积为什么影响平均查找长度：堆积让冲突变多，查找时要顺着挤成一团的位置一个一个比，比较次数变多，平均查找长度就变长了。
- 再想为什么不是存储效率：存储效率是“表里有多少位置被用上了”，由装填因子（元素个数除以表长）决定；堆积不改变元素个数，也不改变表长，所以不直接影响存储效率。
- 再想为什么不是散列函数：散列函数是事先定好的算法，堆积是存数据过程中出现的现象，不会反过来改变散列函数本身。
- 再想为什么不是装填因子：装填因子是元素个数和表长的比值；堆积不改变元素个数，也不改变表长，所以装填因子不变。
- 拿生活例子验算：停车场里，车都挤在入口附近的一片，找车位时要一辆一辆往后看，找的时间变长了；但停车场的总车位数和总车位个数都没变，变的只是找车要比较几次。
- 边界情况：堆积分两种——同义词堆积（同一个散列值的关键字挤在一起）和非同义词堆积（不同散列值的关键字因为处理办法挤在一起）；不管哪种，拖慢的都是查找时的比较次数。
- 易混对子：不要把“装填因子”和“堆积”当成一回事——装填因子是表满不满的程度，堆积是元素分布挤不挤；表不满也可能堆积，表满了也可能分布均匀。

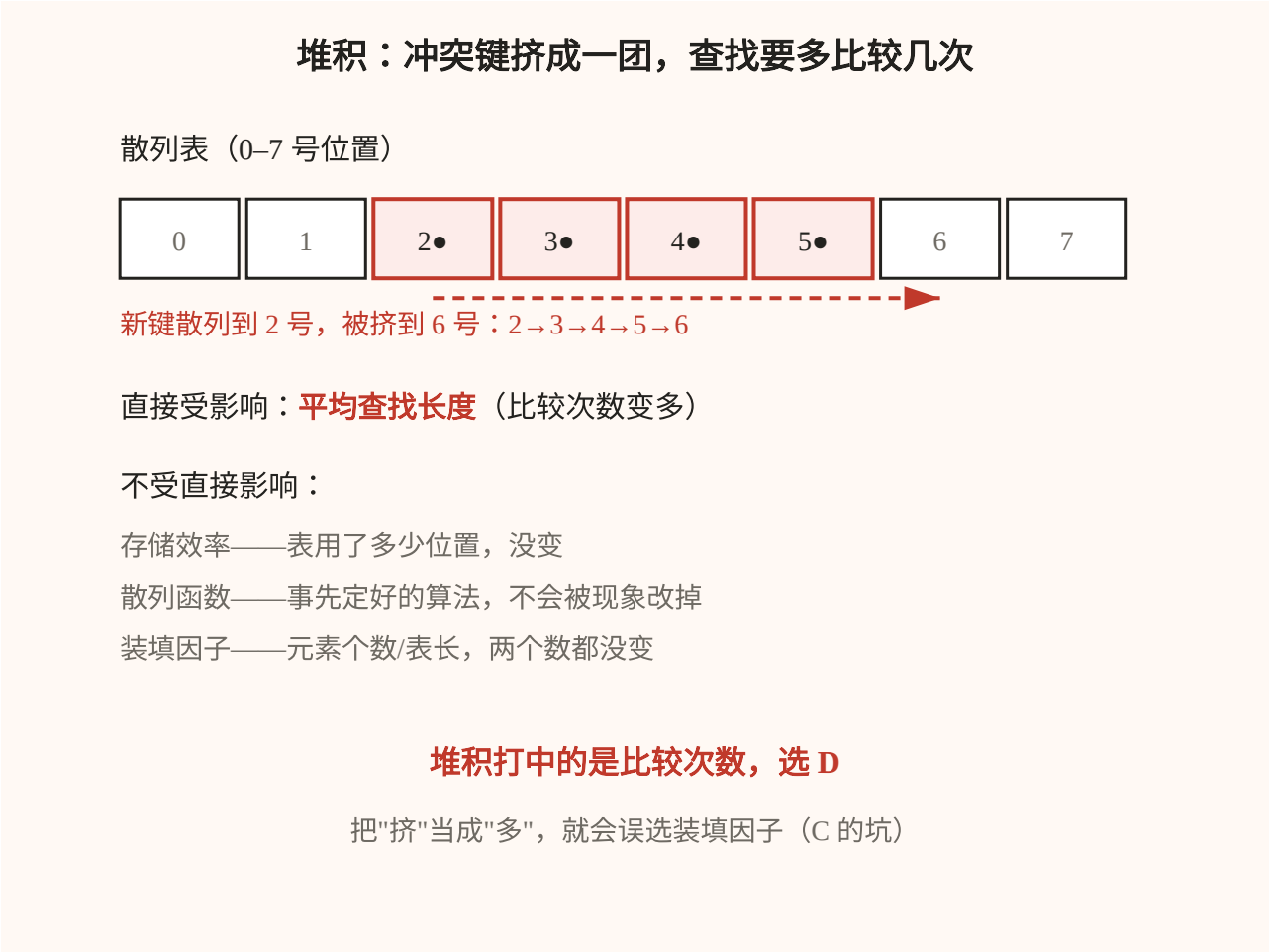
正确选项为什么对： 堆积让查找时比较次数变多，平均查找长度变长，选 D。

错误选项为什么错＋举例：

- A（存储效率）：存储效率看的是表用了多少，和堆积没关系；教训：堆积影响的是找的速度，不是用的空间。
- B（散列函数）：散列函数是事先定的算法，堆积不会改变它；教训：现象不会反过来改变算法。
- C（装填因子）：装填因子是元素个数除以表长，堆积不改变这两个数；教训：装填因子和堆积是两个不同的指标。

笔记本一句话： 第 8 题＋堆积：冲突挤成一团，查找比较次数变多，平均查找长度变长；存储效率、散列函数、装填因子都不直接受影响。

【图像】



散列表堆积示意：冲突键挤成一团，查找要多比较几次；存储效率、散列函数、装填因子均不受影响

【定位】 数据结构 · 第7章 查找 · 散列冲突的堆积现象

【易混对比】

- "堆积"和"装填因子"的区别：装填因子是元素个数/表长，衡量表满不满；堆积是元素挤不挤，衡量分布匀不匀；表不满也可能挤成一团。
- "平均查找长度"和"存储效率"的区别：前者是"找一次平均比几次"，堆积直接拉长它；后者是"表用了多少位置"，由装填因子决定，堆积碰不到它。

堆积让查找时比较次数变多，直接受影响的是平均查找长度，选 D。

【坑位】

堆积不改变"表里有几个元素"和"表有多长"，这两个数不动，装填因子和存储效率就都不动。

看到"冲突多了"就觉得表变满了、装填因子变了（C 的坑）。错在把"挤"当成了"多"：堆积只是位置挤成一团，元素个数和表长都没变，比值当然不变。正确做法：问"直接影响"，先想这个指标的定义里有没有"比较次数"——平均查找长度的定义就是平均比较次数，堆积正好打中它。

【类比】

像停车场：车都挤在入口附近，找空位要一辆一辆往后看（比较次数变多）；但停车场的总车位数和来的车数都没变，"停满程度"（装填因子）和"用了多少位"（存储效率）都不变。术语贴回去：找空位多看几辆，就是"平均查找长度变长"。

【口诀】 堆积挤成团，查找步数添；表长个数不变，因子存储两不沾。

【考频】 「散列查找」在 2009–2022 年解析中出现 1 次 · 低频

第 9 题 · 4 阶 B 树中含关键字结点的个数上限（正确答案：D）

题干与选项：

在一棵具有 15 个关键字的 4 阶 B 树中，含关键字的结点个数最多是 。

- A. 5
- B. 6
- C. 10
- D. 15

入手点： 要结点最多，就让每个结点装的关键字最少；先算每个结点最少能装几个，再看 15 个关键字能分到几个结点。

专有名词从零解释：

- 先想“B 树”：一种多路平衡查找树，每个结点可以存好几个关键字、带好几个孩子；所有叶子结点都在同一层，树是平衡的。
- 再想“阶”：B 树的阶就是一个结点最多有几个孩子；4 阶就是最多 4 个孩子。
- 再想“关键字”：存在结点里的数据；4 阶 B 树的结点最多存 3 个关键字（孩子数减 1），最少存 1 个关键字（根和非根结点都一样，根最少 1 个，其他结点最少是阶的一半向上取整再减 1，也就是 1 个）。
- 再想“含关键字的结点”：就是存了至少一个关键字的结点；空结点不算。

考查知识点：

- 先想每个结点最少几个关键字：4 阶 B 树，非根结点最少 2 个孩子，对应 1 个关键字；根最少 1 个关键字；所以每个结点最少 1 个关键字。
- 再想 15 个关键字最多分到几个结点：每个结点 1 个关键字，15 个关键字正好分到 15 个结点。
- 再想这样的树合不合法：15 个结点排成满二叉树的样子——第 1 层 1 个，第 2 层 2 个，第 3 层 4 个，第 4 层 8 个，一共 15 个；每个内部结点 2 个孩子（满足最少 2 个、不超过 4 个），所有叶子都在第 4 层，平衡；所以合法。
- 拿生活例子验算：15 个人，每间房最少住 1 个人，最多能住 15 间房；如果每间房住 3 个人，就只要 5 间房；要房间最多，就每间房只住 1 个人。
- 边界情况：如果问的是“结点最少几个”，那就是每个结点装满 3 个关键字，15 除以 3 得 5 个；本题问“最多”，所以反过来，每个结点装最少。
- 易混对子：不要把“最多”和“最少”搞反——最多结点对应每结点最少关键字，最少结点对应每结点最多关键字。

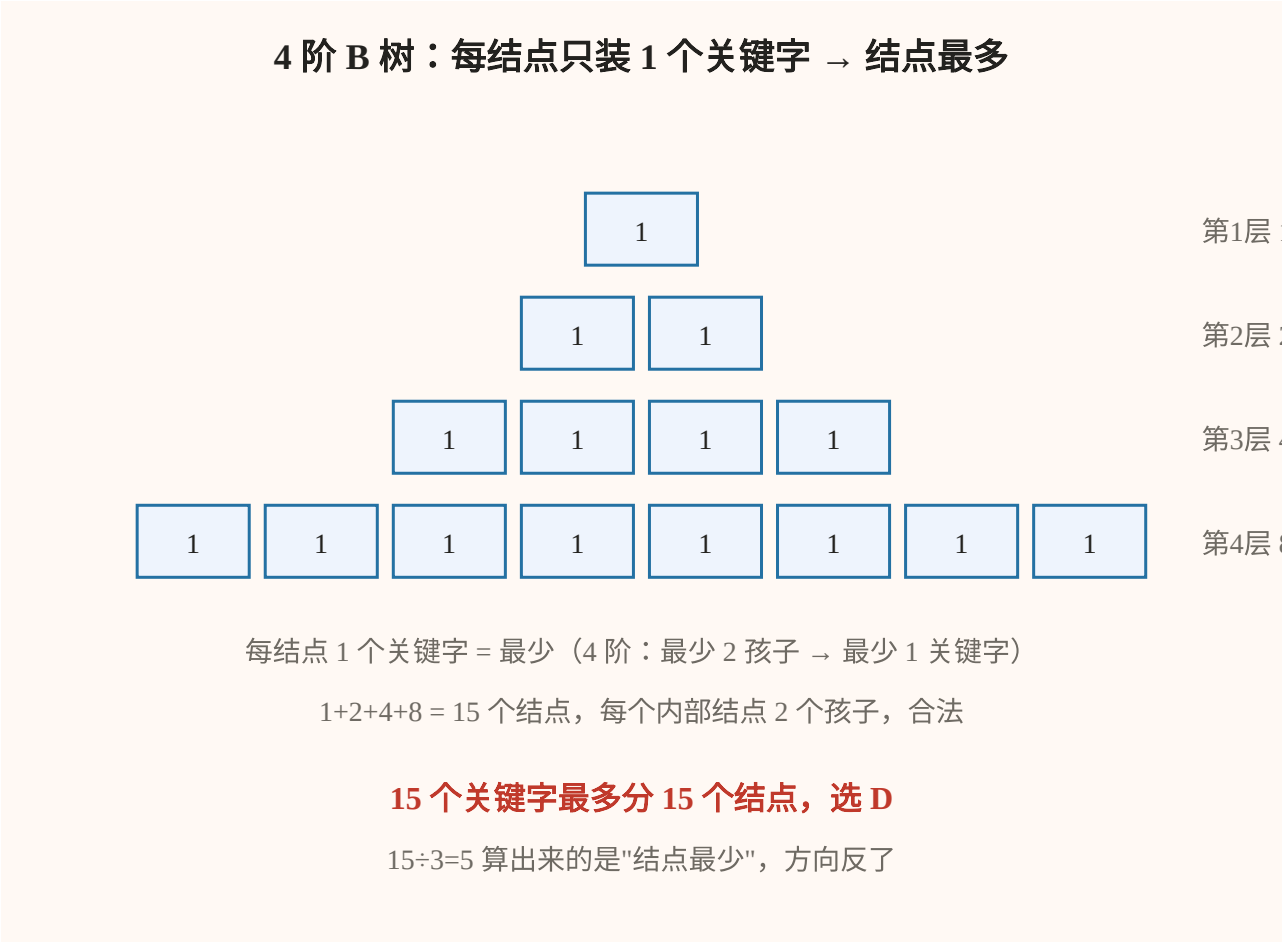
正确选项为什么对： 每个结点最少 1 个关键字，15 个关键字分到 15 个结点，就是 D，选 D。

错误选项为什么错＋举例：

- A（5）：那是每个结点装满 3 个关键字时的结点数，是最少，不是最多。教训：最多和最少要分清。
- B（6）：不是最多也不是最少的极端值；15 个关键字分到 6 个结点，平均每个结点 2.5 个关键字，不是最少的情况。教训：要最多，就要每个结点都取最少。
- C（10）：也不是极端值；分到 10 个结点，平均每个结点 1.5 个关键字，还能再拆。教训：只要还有结点装了 2 个以上关键字，结点数就还能再多。

笔记本一句话： 第 9 题＋B 树结点数最多：每结点关键字取最少（4 阶是 1 个），15 个关键字就是 15 个结点。

【图像】



15 个结点每个只装 1 个关键字、排成 4 层满二叉形的 4 阶 B 树示意图

【定位】 数据结构 · 第7章 查找 · B 树结点数的极值

【易混对比】

- "结点最多"和"结点最少"的区别：最多→每结点关键字取最少；最少→每结点关键字取最满。问反了，答案就反了。
- "4 阶"和"每结点最少 1 个关键字"的区别：4 阶是"最多 4 个孩子"；最少关键字是"最少孩子数减 1"（最少 2 个孩子→最少 1 个关键字）。阶数不能直接拿来除。

4 阶 B 树每结点最少 1 个关键字，15 个关键字最多分到 15 个结点，选 D。

【坑位】

"结点最多"先想"每结点最少几个关键字"，别直接拿 15 除以 3。

顺手用"每结点最多 3 个关键字"去算，15÷3=5，算出来的是"结点最少"，跟题目问的正好反了。错在方向定反了：问最多要用下限，问最少要用上限。正确做法：先定方向（最多→取最少），再算下限（4 阶→最少 1 个关键字），最后分配（15÷1=15）。

【套路】

- 第一步，定方向：问结点最多→每结点关键字取最少。
- 第二步，算下限：4 阶→非根最少 2 个孩子→最少 1 个关键字。
- 第三步，分配：15 个关键字÷每结点 1 个=15 个结点。

问最多取下限，问最少取上限；先定方向再算数。

【类比】

像分房间：15 个人，每间房最少住 1 人，问最多能占几间房——每间只住 1 人，占 15 间；要是问最少几间，才是每间住满 3 人、占 5 间。术语贴回去：房间就是"结点"，每间住的人数就是"结点里的关键字数"。

【口诀】 结点要最多，每点只装一；结点要最少，每点装到顶。

【考频】 「B 树」在 2009–2022 年解析中出现 8 次 · 中频

第 10 题 · 希尔排序增量的判定（正确答案：B）

题干与选项：

用希尔排序方法对一个数据序列进行排序时，若第 1 趟排序结果为 9,1,4,13,7,8,20,23,15，则该趟排序采用的增量（间隔）可能是 。

- A. 2
- B. 3
- C. 4
- D. 5

入手点： 一趟希尔排序后，每个分组内部一定是有序的；把这串数字按增量分组，看哪个增量下每个分组都排好了。

专有名词从零解释：

- 先想“希尔排序”：也叫缩小增量排序；先取一个比较大的增量，把序列分成几组（间隔相同的元素一组），每组内部排序；再把增量缩小，重复，直到增量变成 1。
- 再想“增量”：分组的步长；增量是 3，就是把第 1、4、7 个元素分一组，第 2、5、8 个分一组，第 3、6、9 个分一组。
- 再想“一趟”：按某个增量排一遍；一趟排完，每个分组内部都应该是排好的。

考查知识点：

- 先想给出的序列：9，1，4，13，7，8，20，23，15（9 个数）。
- 再想增量 3 时的分组：第 1、4、7 个是 9、13、20，是有序的；第 2、5、8 个是 1、7、23，是有序的；第 3、6、9 个是 4、8、15，是有序的；三组都排好了，所以增量 3 可能。
- 再想增量 2 时的分组：第 1、3、5、7、9 个是 9、4、7、20、15，9 比 4 大，无序；所以增量 2 不可能。
- 再想增量 4 时的分组：第 1、5、9 个是 9、7、15，9 比 7 大，无序；所以增量 4 不可能。
- 再想增量 5 时的分组：第 1、6 个是 9、8，9 比 8 大，无序；所以增量 5 不可能。
- 拿生活例子验算：一趟排完，每组内部必须排好；检查增量时，只要找到一组内部是乱的，这个增量就不可能。
- 边界情况：增量 3 是“可能”，不是“一定”——题目问“可能值”，只要这个增量下分组都有序，就算可能；本题四个选项里，只有一个增量满足条件。
- 易混对子：不要把“整个序列有序”当成检查标准——一趟希尔排序后，整个序列不一定有序，只有每个分组内部有序；也不要按增量分组时数错位置，第几个元素分到哪组，要按步长数。

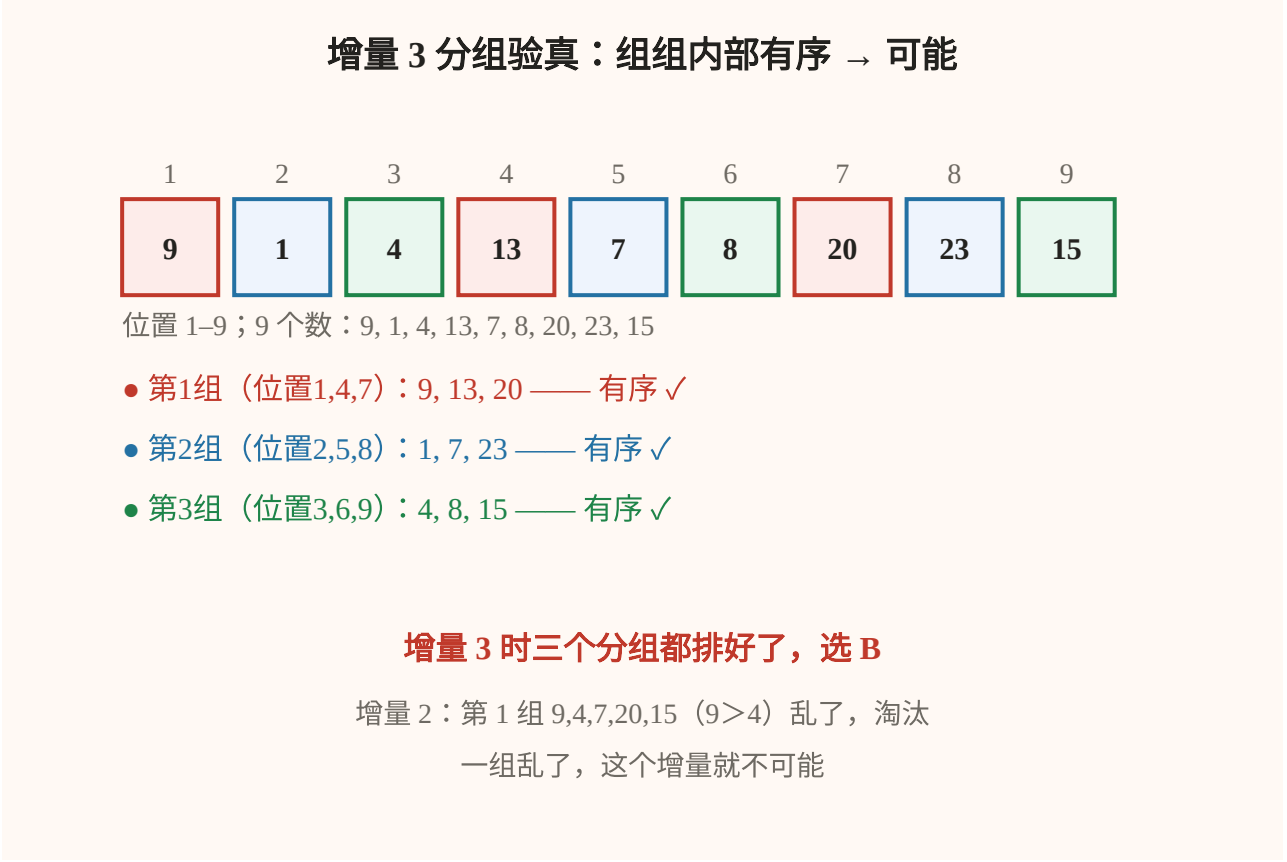
正确选项为什么对： 增量 3 时三个分组都排好了，其他增量都有分组是乱的，所以增量可能是 3，选 B。

错误选项为什么错＋举例：

- A（2）：分组 9、4、7、20、15 里 9 比 4 大，无序；教训：一组乱了，整个增量就不可能。
- C（4）：分组 9、7、15 里 9 比 7 大，无序；教训：分组要按步长数，不能数错。
- D（5）：分组 9、8 里 9 比 8 大，无序；教训：两个元素的组也要查，一个逆序也不行。

笔记本一句话： 第 10 题＋希尔增量验真：一趟排完每个分组内部必须有序；按增量分组，一组乱了就不可能。

【图像】



9 个数按增量 3 分成 3 组、每组内部有序的分组示意图

【定位】 数据结构 · 第8章 排序 · 希尔排序增量的判定

【易混对比】

- "分组内有序"和"整个序列有序"的区别：一趟希尔排序后只保证每个分组内部有序，整体不一定有序；拿"整体有序"当检查标准会误杀。
- "可能"和"一定"的区别：增量 3 下分组都有序，只能说"可能是 3"，不能反推"第 1 趟用的就是 3"；题目问可能，满足条件就够。

增量 3 时三组（9,13,20）、（1,7,23）、（4,8,15）都内部有序，其他增量都有乱组，选 B。

【坑位】

分组时按"步长"数位置，别按"连续"数。

增量 3 的第 1 组是第 1、4、7 个元素（9、13、20），顺手拿成第 1、2、3 个（9、1、4）去验，组都分错了，验了也白验。错在把"间隔 d"数成了"连续 d 个"。正确做法：先给 9 个数标位置 1–9，再按步长挑：第 i 组是位置 i, i+d, i+2d…。

【套路】

- 第一步，写位置编号：给 9 个数标上 1–9。
- 第二步，按增量分组：第 i 组取位置 i, i+d, i+2d…。
- 第三步，查组内有序：一组乱了，这个增量就淘汰。

按步长分组，组组查有序，一组乱了即淘汰。

【类比】

像按步长点名排队：增量 3 就是"第 1、4、7 个出列站一排，第 2、5、8 个站一排……"；一趟排完，每一排内部必须从小到大站好，哪一排乱了，这个步长就不可能是当时的口令。术语贴回去：步长就是"增量"，每一排就是"一个分组"。

【口诀】 步长分组查组内，一组乱了即淘汰。

【考频】 「希尔排序」在 2009–2022 年解析中出现 7 次 · 中频

第 11 题 · 快速排序第二趟结果的判定（正确答案：C）

题干与选项：

下列选项中，不可能是快速排序第 2 趟排序结果的是 。

- A. 2,3,5,4,6,7,9
- B. 2,7,5,6,4,3,9
- C. 3,2,5,4,7,6,9
- D. 4,2,3,5,7,6,9

入手点： 快速排序每趟确定一个基准的最终位置；两趟下来，至少有两个元素已经站在排好序后的最终位置上；数一数每个序列里有几个元素在最终位置，少于两个的就不可能。

专有名词从零解释：

- 先想“快速排序”：挑一个基准，把比基准小的放左边、比基准大的放右边，基准就站到了最终位置；再对左右两边分别重复。
- 再想“一趟”：一次划分的过程；一趟结束，基准站在最终位置，左右两边内部还是乱的。
- 再想“最终位置”：把所有元素排好序后，某个元素应该在的位置；比如元素集排好序是 2、3、4、5、6、7、9，那么 2 的最终位置是第 1 位，9 的最终位置是第 7 位。
- 再想“基准”：也叫枢轴，每趟挑出来的那个元素；一趟结束，基准一定在最终位置。

考查知识点：

- 先想元素集排好序的样子：2、3、4、5、6、7、9；第 1 位是 2，第 2 位是 3，第 3 位是 4，第 4 位是 5，第 5 位是 6，第 6 位是 7，第 7 位是 9。
- 再想 A（2,3,5,4,6,7,9）：第 1 位 2 对了，第 2 位 3 对了，第 5 位 6 对了，第 6 位 7 对了，第 7 位 9 对了，一共 5 个在最终位置，不少于 2 个，可能。
- 再想 B（2,7,5,6,4,3,9）：第 1 位 2 对了，第 7 位 9 对了，一共 2 个在最终位置，不少于 2 个，可能。
- 再想 C（3,2,5,4,7,6,9）：只有第 7 位 9 对了，其他位都不对，一共 1 个在最终位置，少于 2 个，不可能。
- 再想 D（4,2,3,5,7,6,9）：第 4 位 5 对了，第 7 位 9 对了，一共 2 个在最终位置，不少于 2 个，可能。
- 拿生活例子验算：两趟排完，至少有两个“钉子户”已经站到了最终位置、再也不动了；一个序列里只有一个钉子户，说明它最多只排了一趟，不可能是两趟的结果。
- 边界情况：如果两趟挑的基准是同一个元素（不可能，基准一旦定下来就不再参与划分），钉子户只会更多，不会更少；所以“至少两个”是下限，不是上限。
- 易混对子：不要以为“序列里有一部分是排好的”就算可能——要数的是“站在最终位置”的元素，不是“局部有序”的元素；比如 5、4 这样的小段，看着像排好的，但位置不对就不算。

正确选项为什么对： C 里只有一个元素在最终位置，少于两趟应有的至少两个，所以 C 不可能是第 2 趟结果，选 C。

错误选项为什么错＋举例：

- A（2,3,5,4,6,7,9）：有 5 个元素在最终位置，不少于 2 个，可能；所以 A 不对。教训：数最终位置的元素个数，不是数局部有序的段数。
- B（2,7,5,6,4,3,9）：有 2 个元素在最终位置，刚好达到下限，可能；所以 B 不对。教训：刚好两个也算可能。

- D（4,2,3,5,7,6,9）：有 2 个元素在最终位置，可能；所以 D 不对。教训：中间乱不要紧，关键看最终位置的元素够不够两个。

笔记本一句话： 第 11 题＋快排第 n 趟：每趟定一个基准的最终位置；2 趟后至少 2 个元素在最终位置，少于 2 个就不可能。

【图像】



快排两趟钉子户示意图：以排好序的 2,3,4,5,6,7,9 为参照，C 选项中只有 9 站在最终位置

【定位】 数据结构 · 第8章 排序 · 快速排序"第 n 趟结果"判定

【易混对比】

- "第 n 趟"和"第 n 层递归"是两回事：一趟就是一次划分，每趟钉死一个基准；n 趟下来至少 n 个元素站在最终位置。别把趟数当成递归深度去数。
- "站在最终位置"和"局部看着有序"是两回事：2,3 挨着看着顺眼，但判定标准是"位置和数值都对上排好序的序列"；只数钉子户，不数顺眼段。
- "至少 n 个"和"恰好 n 个"是两回事：两趟可能钉死 5 个（A 选项），钉子户只多不少；下限不是上限。

两趟划分至少钉死两个基准；C 里只有 9 站在最终位置，少于两个，不可能，选 C。

【坑位】

题没给初始序列，就别脑补"第一趟选了谁当基准"。

看到"第 2 趟"就想去猜第一趟的基准是谁、怎么划分的，这是死路：初始序列未知，基准选择未知，任何具体的划分过程都推不出来。为什么错：猜基准等于在未知数上盖房子，换一种初始序列结论就塌。正确做法：用不变量——不管怎么划分，每趟必钉死一个基准；先写出排好序的全序列，逐位比对数钉子户个数，个数不够下限的直接判死。

【套路】

- 第一步，写出排好序的全序列并标位：2,3,4,5,6,7,9，第 1 位该是 2……第 7 位该是 9。
- 第二步，逐位比对数钉子户：位置和数值都对上的才算一个。

- 第三步，拿个数比下限：第 n 趟至少 n 个钉子户；少于 n 的不可能。

先写有序序列，再数钉子户，个数比趟数。

【类比】

像往墙上钉钉子挂画：每趟（每次划分）钉死一颗钉子（基准），画框（其他元素）还在晃。两趟下来，墙上至少有两颗钉死的钉子。判定时数的是钉死的钉子，不是还在晃的画框。术语贴回去：钉死的钉子就是"已在最终位置的元素"，晃的画框就是"尚未确定位置的元素"。

【口诀】 快排一趟钉一桩，几趟下来几个钉子户打底。

【考频】 「快速排序」在 2009–2022 年解析中出现 9 次 · 中频

第 12 题 · 指令数与 CPI 变化后的程序执行时间（正确答案：D）

题干与选项：

程序 P 在机器 M 上的执行时间是 20 秒，编译优化后，P 执行的指令数减少到原来 的 70%，而 CPI 增加到原来的 1.2 倍，则 P 在 M 上的执行时间是 。

- A. 8.4 秒
- B. 11.7 秒
- C. 14 秒
- D. 16.8 秒

入手点： 执行时间等于指令数乘以每条指令的平均时钟周期数，再乘以每个时钟周期的时间；时钟没变，只看指令数和平均周期数怎么变。

专有名词从零解释：

- 先想“指令数”：程序一共要执行多少条指令。
- 再想“CPI”：每条指令平均要几个时钟周期；比如 CPI 是 2，意思是一条指令平均花 2 个时钟周期。
- 再想“执行时间”：指令数乘以 CPI，再乘以每个时钟周期的时间（时钟周期的倒数就是主频）；时钟没变时，执行时间只跟着指令数和 CPI 变。

考查知识点：

- 先想原来的执行时间：20 秒。
- 再想指令数变了多少：变成原来的 70%，就是乘以 0.7。
- 再想 CPI 变了多少：变成原来的 1.2 倍，就是乘以 1.2。
- 再想新的执行时间：20 乘 0.7 再乘 1.2，等于 16.8 秒，就是 D。
- 拿生活例子验算：原来要做 100 件事，每件事花 2 小时，一共 200 小时；现在事变成 70 件，每件事花 2.4 小时，一共 168 小时；算法就是原来的时间乘以两个变化率。
- 边界情况：如果时钟频率也变了，还要再乘以时钟周期的变化率；本题时钟没变，只乘指令数和 CPI 的变化率。
- 易混对子：不要把“指令数降到 70%”理解成“时间降到 70%”——CPI 还涨了 1.2 倍，两个变化要一起乘；也不要把 0.7 和 1.2 加起来（0.7 加 1.2 是 1.9，乘出来是 38 秒，完全不对）。

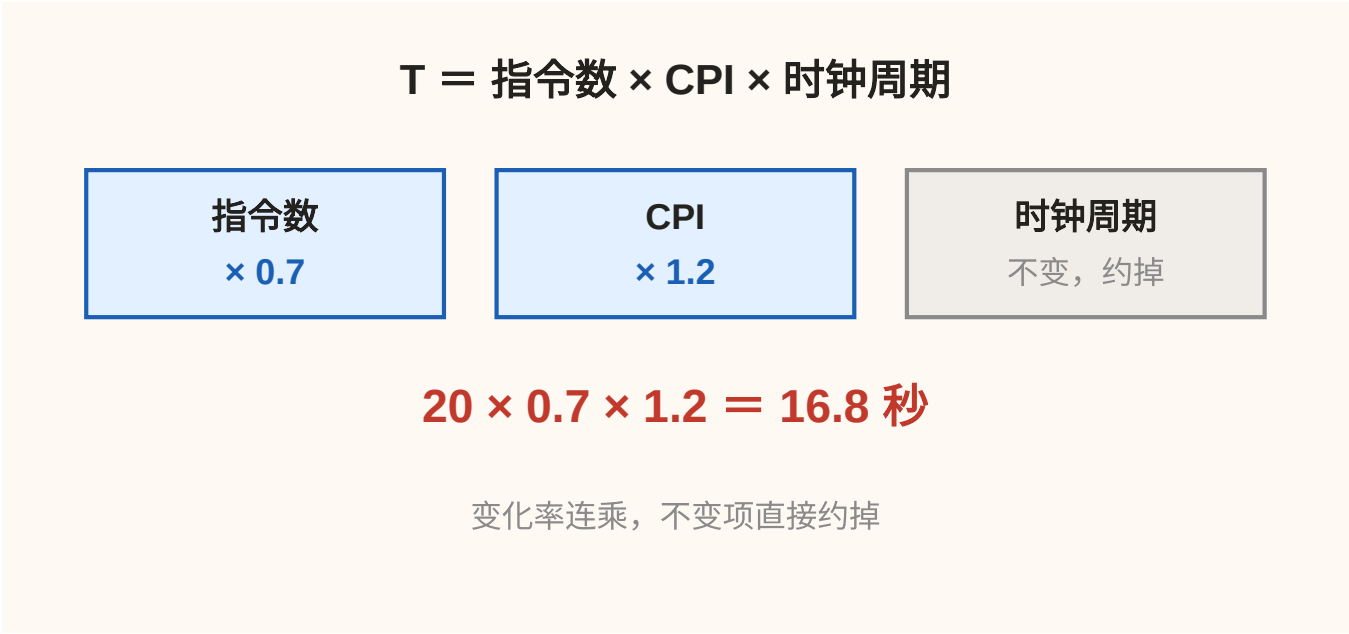
正确选项为什么对： 20×0.7×1.2＝16.8 秒，就是 D，选 D。

错误选项为什么错＋举例：

- A（8.4 秒）：把 CPI 的 1.2 倍算反了方向（当成 0.6 倍），20×0.7×0.6＝8.4；教训：CPI 是涨了 1.2 倍，不是降了。
- B（11.7 秒）：两个变化率的乘法算错了；教训：0.7 乘 1.2 是 0.84，20 乘 0.84 是 16.8。
- C（14 秒）：只乘了指令数的 0.7 倍，漏了 CPI 的 1.2 倍，20×0.7＝14；教训：两个变化都要乘，不能只乘一个。

笔记本一句话： 第 12 题＋执行时间：指令数变化率乘以 CPI 变化率，再乘原来时间；本题 20×0.7×1.2＝16.8 秒。

【图像】



CPU 执行时间三因子示意图：指令数乘 0.7、CPI 乘 1.2、时钟周期不变，结果 16.8 秒

【定位】 计算机组成原理 · 第1章 计算机系统概述 · CPU 执行时间

【易混对比】

- "指令数降到 70%"和"执行时间降到 70%"是两回事：指令数只是三个因子之一，CPI 还涨了 1.2 倍；局部变化率不能直接当整体变化率。
- "增加到 1.2 倍"和"增加了 1.2 倍"是两回事：前者是乘 1.2，后者是乘 2.2；读题先看清是"到"还是"了"。

T＝指令数×CPI×时钟周期；时钟不变约掉，新时间＝20×0.7×1.2＝16.8 秒，选 D。

【坑位】

别把 0.7 和 1.2 加起来。

0.7＋1.2＝1.9，再乘 20 得 38 秒，看着像算了，实则全错。为什么错：指令数和 CPI 是相乘关系（T＝IC×CPI×Tc），变化率也要连乘；加法只在"同一量纲的增量"里成立，这里不成立。正确做法：列出完整公式，不变的项（时钟周期）直接约掉，变化率连乘：20×0.7×1.2。

【套路】

- 第一步，列公式：T＝指令数×CPI×时钟周期时间。
- 第二步，约掉不变项：时钟没变，直接消去。
- 第三步，连乘变化率：20×0.7×1.2＝16.8。

时间三因子，变几个乘几个，不变就约掉。

【类比】

原来 100 件货、每件搬 2 趟，总共 200 趟；现在货剩 70 件、每件要搬 2.4 趟，总共 70×2.4＝168 趟。货量和每件趟数是乘出来的，不是加出来的。术语贴回去：货量就是"指令数"，每件趟数就是"CPI"，总趟数就是"执行时间"。

【口诀】 时间等于三连乘，变化率也连乘，不变项直接约。

【考频】 「CPU执行时间」在 2009–2022 年解析中出现 9 次 · 中频

第 13 题 · 8 位定点补码运算的溢出判定（正确答案：C）

题干与选项：

若 x=103，y=-25，则下列表达式采用 8 位定点补码运算实现时，会发生溢出的 是 。

- A. x+y
- B. -x+y
- C. x-y
- D. -x-y

入手点： 8 位补码能表示的范围是负 128 到正 127；四个式子各算出结果，看哪个结果超出了这个范围。

专有名词从零解释：

- 先想“定点补码”：计算机里表示带符号整数的办法；8 位补码的最高位是符号位，0 是非负数，1 是负数。
- 再想“表示范围”：8 位补码能表示的整数，从负 128 到正 127；比 127 大，或者比负 128 小，都表示不出来。
- 再想“溢出”：运算结果超出了表示范围，计算机存不下，结果就是错的。

考查知识点：

- 先想 x＋y：103 加负 25，等于 78，在负 128 到正 127 之间，不溢出。
- 再想负 x 加 y：负 103 加负 25，等于负 128，恰好是范围的下限，能表示，不溢出。
- 再想 x 减 y：103 减负 25，等于 103 加 25，等于 128，比 127 大 1，超出了范围，溢出。
- 再想负 x 减 y：负 103 减负 25，等于负 103 加 25，等于负 78，在范围内，不溢出。
- 拿生活例子验算：一个只能装负 128 到正 127 的盒子，128 就装不下了，负 128 刚好能装；边界上的数要一个一个验，不能凭感觉。
- 边界情况：负 128 是 8 位补码的最小值，本身能表示；正 128 是表示不出来的，第一个溢出的正数就是 128。
- 易混对子：不要把“减负数”当成“变小”——减一个负数等于加上它的绝对值，结果变大，更容易往上溢出；也不要把“负 128”当成溢出——负 128 是范围内，不是范围外。

正确选项为什么对： x 减 y 等于 128，超出了正 127 的上限，溢出，选 C。

错误选项为什么错＋举例：

- A（x＋y）：结果 78，在范围内，不溢出；教训：先算出结果，再看范围。
- B（负 x 加 y）：结果负 128，恰好是下限，能表示，不溢出；教训：边界值本身不算溢出。
- D（负 x 减 y）：结果负 78，在范围内，不溢出；教训：减负数是加绝对值，但这道题加完还在范围内。

笔记本一句话： 第 13 题＋8 位补码溢出：范围是负 128 到正 127；x-y＝128 超了上限，溢出；负 128 本身不溢出。

【图像】



8 位补码数轴示意图：范围负 128 到 127，128 越过上界标红为溢出

【定位】 计算机组成原理 · 第2章 数据的表示和运算 · 定点补码溢出判定

【易混对比】

- "减负数"和"变小"是两回事：x−y＝x＋25，结果变大，往上溢；看到减负数先变加法再看方向。
- "−128"和"溢出"是两回事：−128 是 8 位补码的下限本身，合法；溢出的是 128（比上限多 1）。
- "列竖式算补码"和"判溢出"是两回事：这题不用真去算补码加法，先算十进制结果再卡范围更快。

8 位补码范围 −128～127；x−y＝128 超出上限，只有 C 溢出，选 C。

【坑位】

别被"−x＋y＝−128"吓到判溢出。

−128 长得像"超了"，但它是下限本身，能表示，不溢出。为什么错：把"边界值"当成了"界外值"；溢出是"装不下"，−128 正好是盒子底，是盒子的一部分。正确做法：四个式子先算十进制结果（78、−128、128、−78），再逐个卡 −128～127，边界值单独拎出来验。

【套路】

- 第一步，化成加法：减负数变加正数，x−y＝x＋25。
- 第二步，算十进制结果：78、−128、128、−78。
- 第三步，卡范围：−128～127，超出的判溢出。

先算十进制，再卡 −128～127，边界单独验。

【类比】

盒子只能装 −128 到 127。128 是多出来的第 1 个，盖不上盖；−128 是盒子底，本身就是盒子的一部分。术语贴回去：盒子就是"8 位补码的表示范围"，盖不上盖就是"溢出"。

【口诀】 八位补码下限负一二八、上限正一二七，减负变加往上冒。

【考频】 「补码溢出」在 2009–2022 年解析中出现 6 次 · 中频

第 14 题 · IEEE754 单精度浮点数的大小与符号比较（正确答案：A）

题干与选项：

float 型数据常用 IEEE754 单精度浮点格式表示。假设两个 float 型变量 x 和 y 分别存放在 32 位寄存器 f1 和 f2 中，若(f1)=CC90 0000H，(f2)=B0C0 0000H，则 x 和 y 之间的 关系为 。

- A. x<y 且符号相同
- B. x<y 且符号不同
- C. x>y 且符号相同
- D. x>y 且符号不同

入手点： 把 32 位拆成三段：符号位、阶码、尾数；先比符号，再比阶码；符号相同比大小时，负数是绝对值越大、数值越小。

专有名词从零解释：

- 先想“IEEE754 单精度”：计算机里存小数的标准格式，32 位：最高 1 位是符号位，接着 8 位是阶码，最后 23 位是尾数。
- 再想“符号位”：0 是正数，1 是负数。
- 再想“阶码”：8 位存的是移码，要减去 127，才是真实的指数；指数越大，数的数量级越大。
- 再想“尾数”：小数部分，前面默认还有个 1；尾数越大，数在同一个数量级里越大。

考查知识点：

- 先想 f1（CC90 0000H）：写成二进制，最高位是 1，符号是负；阶码是 10011001，等于 153，减去 127 得指数 26；数值大约是负 8.6 乘 10 的 7 次方，是个绝对值很大的负数。
- 再想 f2（B0C0 0000H）：最高位也是 1，符号也是负；阶码是 01100001，等于 97，减去 127 得指数负 30；数值大约是负 1.6 乘 10 的负 9 次方，是个绝对值很小的负数。
- 再想符号异同：两个符号位都是 1，都是负数，符号相同。
- 再想大小关系：两个都是负数，绝对值大的那个数值反而小；x 的绝对值是 10 的 7 次方量级，y 的绝对值是 10 的负 9 次方量级，x 的绝对值大得多，所以 x 小于 y。
- 拿生活例子验算：欠 8000 万和欠 1 分钱，都是欠钱（符号相同），欠得多的那个更小（更负）；所以 x 小于 y。
- 边界情况：比大小时，先比符号（正数一定比负数大），符号相同再比阶码（指数大的绝对值大），阶码相同再比尾数；本题两个都是负数，指数一比就分出来了，不用比尾数。
- 易混对子：不要把“指数大”直接当成“数值大”——负数是指数越大、绝对值越大、数值越小；也不要把“符号位都是 1”当成“符号不同”——符号位相同就是符号相同。

正确选项为什么对： 两个都是负数，符号相同；x 的绝对值大得多，负数绝对值越大数值越小，所以 x 小于 y，选 A。

错误选项为什么错＋举例：

- B（x 小于 y 且符号不同）：大小关系对了，但符号说错了；两个符号位都是 1，符号相同，不是不同。教训：先看符号位，再比大小。
- C（x 大于 y 且符号相同）：符号对了，但大小关系反了；负数是绝对值越大越小，x 的绝对值大，所以 x 更小。教训：负数比大小和正数反着。
- D（x 大于 y 且符号不同）：大小和符号都说错了；教训：两个条件要一个一个验。

笔记本一句话： 第 14 题＋浮点数比大小：先看符号位，再看阶码；两个都是负数，绝对值大的反而小。

【图像】



IEEE754 单精度拆分对比图：f1、f2 符号位都是 1，阶码 153 对 97

【定位】 计算机组成原理 · 第2章 数据的表示和运算 · IEEE754 浮点数比较

【易混对比】

- "符号位相同"和"大小关系确定"是两回事：符号位都是 1 只说明都是负数；负数比大小看绝对值，绝对值大的反而小。
- "阶码大"和"数值大"是两回事：阶码大＝绝对值大；正数绝对值大则数值大，负数绝对值大则数值小；先定正负再谈阶码。
- "十六进制直比"和"按字段比"是两回事：CC90H＞B0C0H 没错，但那是位串大小；数值大小必须拆三段，符号位一出来结论就反。

两数符号位都是 1，符号相同；x 阶码 153（指数 26）、y 阶码 97（指数 −30），x 绝对值大得多，负数绝对值越大越小，所以 x＜y，选 A。

【坑位】

别直接比十六进制下结论。

CC90 0000H 看着比 B0C0 0000H 大，顺手就选"x＞y 且符号相同"，这是最顺手的错法。为什么错：十六进制比的是位串的字典序，浮点数的数值含义藏在"符号＋阶码＋尾数"的结构里；符号位都是 1 时，位串大＝绝对值大＝数值小。正确做法：先拆符号位定正负，再比阶码定绝对值，阶码分出胜负就不用看尾数。

【套路】

- 第一步，看符号位：最高位，0 正 1 负，先定正负；一正一负直接分胜负。
- 第二步，比阶码：同号时阶码大＝绝对值大；正数取绝对值大的，负数取绝对值小的。
- 第三步，阶码相同才比尾数。

先符号，再阶码，后尾数；负数反着比。

【类比】

欠钱：欠 8000 万和欠 1 分钱，都是"欠"（符号相同），欠得多的那个更穷（数值更小）。术语贴回去："欠"就是"符号位为 1 的负数"，欠多少就是"绝对值"，更穷就是"数值更小"。

【口诀】 先看符号再看阶，同号阶大绝对值大，负数反着比大小。

【考频】 「IEEE754」在 2009–2022 年解析中出现 1 次 · 低频

第 15 题 · DRAM 芯片地址引脚与数据引脚的总数（正确答案：A）

题干与选项：

某容量为 256MB 的存储器由若干 4M×8 位的 DRAM 芯片构成，该 DRAM 芯片的 地址引脚和数据引脚总数是 。

- A. 19
- B. 22
- C. 30
- D. 36

入手点： 地址引脚和数据引脚分开算：数据引脚看芯片的位宽，地址引脚看芯片的容量，再想想 DRAM 地址线是复用的。

专有名词从零解释：

- 先想“DRAM”：动态随机存储器，靠电容上的电荷存信息；电容会漏电，所以要周期性刷新。
- 再想“4M 乘 8 位”：一片芯片有 4M 个存储单元，每个单元 8 位（一个字节）；4M 就是 2 的 22 次方。
- 再想“地址引脚”：芯片上用来接收地址信号的针脚；地址有多少位，原则上就要多少根地址线。
- 再想“地址复用”：DRAM 为了省引脚，把地址分两次送：先送行地址，再送列地址，两次共用同一组引脚；所以地址引脚数是地址位数的一半。
- 再想“数据引脚”：芯片上用来传数据的针脚；位宽是 8 位，就有 8 根数据引脚。

考查知识点：

- 先想地址要几位：4M 个单元，地址要 22 位（2 的 22 次方等于 4M）。
- 再想地址引脚几根：地址复用，行地址和列地址各 11 位，共用一组引脚，所以地址引脚是 11 根。
- 再想数据引脚几根：位宽 8 位，所以数据引脚是 8 根。
- 再想总数：11 加 8，等于 19，就是 A。
- 拿生活例子验算：一个仓库有 4M 个格子，编号要 22 位；为了省门，只开 11 个门，行号和列号分两次报；出货一次出 8 箱，8 个出货口；门和出货口加起来是 19 个。
- 边界情况：如果是不复用的存储器（比如 SRAM），地址引脚就是 22 根；DRAM 才复用，所以是 11 根；题目问的是 DRAM，不能按不复用算。
- 易混对子：不要把“整个 256MB 存储器的地址位数”当成“芯片的地址引脚数”——256MB 是整个存储器的容量，不是芯片的容量；芯片是 4M 的，地址是 22 位，复用后是 11 根。

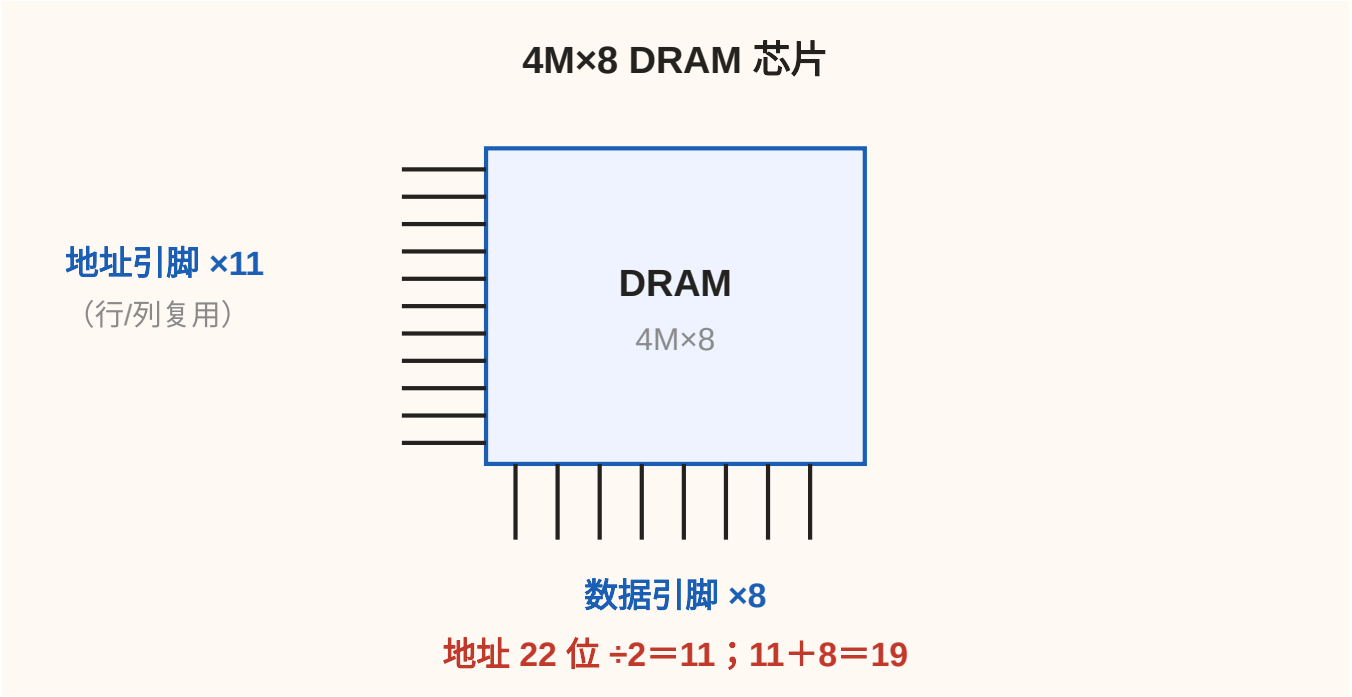
正确选项为什么对： 地址引脚 11 根加数据引脚 8 根，等于 19，选 A。

错误选项为什么错＋举例：

- B（22）：把 22 位地址直接当成了引脚总数，漏了数据引脚，也没算地址复用；教训：地址位数、地址引脚、数据引脚是三回事。
- C（30）：地址引脚按不复用的 22 根算，再加数据引脚 8 根，得 30；漏了 DRAM 地址复用减半这一步。教训：DRAM 地址线是复用的，要除以 2。
- D（36）：按整个 256MB 算地址位数（256M 是 2 的 28 次方，28 根），再加数据引脚 8 根，得 36；错在拿整个存储器的容量代替芯片的容量。教训：先看芯片是 4M 的，不是 256MB 的。

笔记本一句话： 第 15 题＋DRAM 引脚：地址位数除以 2（复用）加数据位宽；本题 11＋8＝19。

【图像】



DRAM 芯片引脚示意图：22 位地址经行列复用变成 11 根地址引脚，加 8 根数据引脚共 19 根

【定位】 计算机组成原理 · 第3章 存储系统 · DRAM 芯片引脚数

【易混对比】

- "芯片容量"和"存储器总容量"是两回事：引脚数看单片 4M×8；256MB 只决定用几片，不管单片引脚。
- "地址位数"和"地址引脚数"是两回事：地址 22 位，但 DRAM 行列复用，引脚只有 11 根；位数除以 2 才是引脚数。
- "地址引脚"和"数据引脚"是两回事：11 根地址＋8 根数据＝19；别漏数据引脚，也别把 8 位位宽当地址。

4M＝2 的 22 次方，地址 22 位复用成 11 根，数据 8 根，共 19 根，选 A。

【坑位】

别一看到 256MB 就去算 28 位地址。

256MB 是整个存储器的容量，题目问的是"该 DRAM 芯片"的引脚；拿系统级容量算芯片级引脚，地址就多算了 6 位。为什么错：引脚长在芯片上，只跟芯片的 4M×8 有关；总容量只决定"用几片"，不改变"每片长什么样"。正确做法：先圈定层级——问芯片就只看芯片容量，再算地址位数、复用、加位宽。

【套路】

- 第一步，定芯片容量：4M×8，地址 22 位，数据 8 位。
- 第二步，地址复用除以 2：22÷2＝11 根地址引脚。
- 第三步，相加：11＋8＝19。

芯片级容量起步，地址位数砍半，再加数据位宽。

【类比】

仓库有 4M 个格子，编号要 22 位；但只开 11 个窗口，行号列号分两次报；出货一次 8 箱，8 个出货口；窗口加出货口一共 19 个。术语贴回去：窗口就是"地址引脚"，分两次报就是"地址复用"，出货口就是"数据引脚"。

【口诀】 DRAM 地址线复用，位数砍半再加位宽。

【考频】 「DRAM」在 2009–2022 年解析中出现 7 次 · 中频

第 16 题 · 指令 Cache 与数据 Cache 分离的主要目的（正确答案：D）

题干与选项：

采用指令 Cache 与数据 Cache 分离的主要目的是 。

- A. 降低 Cache 的缺失损失
- B. 提高 Cache 的命中率
- C. 降低 CPU 平均访存时间
- D. 减少指令流水线资源冲突

入手点： 先想不分开时会出什么问题：取指令和取数据抢同一个 Cache 口，流水线就要停下来等；分开了，各走各的路，就不抢了。

专有名词从零解释：

- 先想“Cache”：夹在 CPU 和主存之间的小而快的存储器；CPU 要的数据如果在 Cache 里，就不用去慢的主存拿。
- 再想“指令 Cache、数据 Cache”：指令 Cache 只存指令，数据 Cache 只存数据；分开存，就叫分离。
- 再想“哈佛结构”：指令和数据分开存放的结构，和分离 Cache 是一个思路。
- 再想“流水线”：把一条指令的执行分成好几个阶段，不同指令的不同阶段同时进行，像工厂流水线。
- 再想“结构冲突”：也叫资源冲突，流水线的两个阶段同一时间要同一个硬件，抢起来了，只好让一个阶段等一等。

考查知识点：

- 先想不分开时的问题：取指阶段要访问 Cache 拿指令，访存阶段要访问 Cache 拿数据；两个阶段同一时间都要 Cache，同一个 Cache 口只能服务一个，另一个就要等，这就是结构冲突，流水线停顿。
- 再想分开后的好处：取指走指令 Cache，访存走数据 Cache，各走各的路，不抢了，流水线不用停顿。
- 再想为什么是“主要目的”：分离 Cache 解决的就是“取指和访存抢 Cache”这个冲突问题；其他好处（比如命中率、缺失损失）不是分离带来的。
- 拿生活例子验算：一个窗口既卖票又取票，排队的人抢一个窗口，就要等；分成卖票窗口和取票窗口，各排各的队，就不抢了。
- 边界情况：分离 Cache 后，指令和数据各占一块 Cache，总容量不变时，每块比原来小，命中率可能反而低一点；但流水线不冲突的好处更大，所以还是分开。
- 易混对子：不要把“Cache 的一般好处”和“分离 Cache 的好处”搞混——Cache 本身是为了解决速度不匹配，分离 Cache 是为了解决取指和访存抢资源。

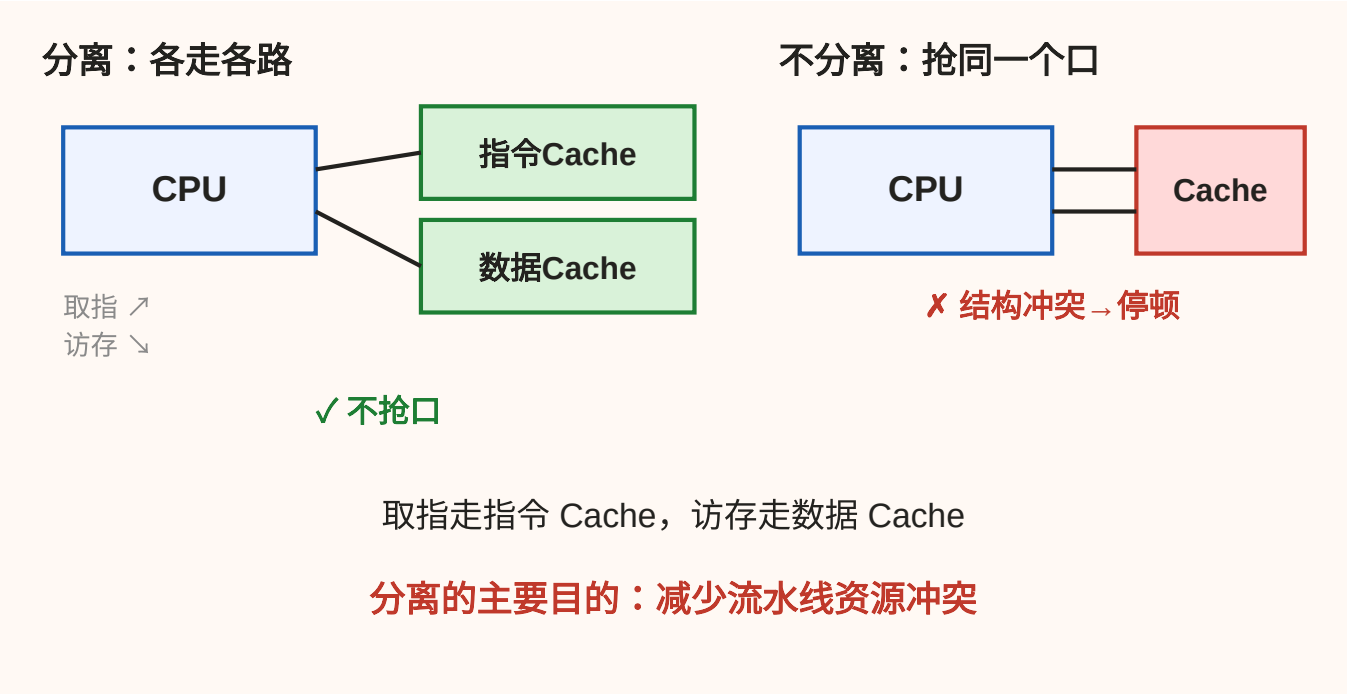
正确选项为什么对： 分离后取指和访存各走各的 Cache，不抢资源，流水线结构冲突少了，选 D。

错误选项为什么错＋举例：

- A（降低缺失损失）：缺失损失是 Cache 里没有、要去主存拿的代价，由 Cache 大小和主存速度决定；分离不直接降低它。教训：缺失损失和资源冲突是两回事。
- B（提高命中率）：命中率看的是程序局部性和 Cache 容量；分离后每块 Cache 反而小了，命中率不一定提高。教训：分离解决的是冲突，不是命中率。
- C（降低 CPU 平均访存时间）：平均访存时间等于命中时间加缺失率乘缺失损失；分离主要解决的是冲突停顿，不是平均访存时间本身。教训：主要目的和附带效果要分清。

笔记本一句话： 第 16 题＋指令数据 Cache 分离：取指和访存各走各的路，不抢 Cache 口，流水线结构冲突少了。

【图像】



Cache 分离示意图：取指走指令 Cache、访存走数据 Cache，各走各路不抢口

【定位】 计算机组成原理 · 第3章 存储系统 · 指令 Cache 与数据 Cache 分离

【易混对比】

- "Cache 本身的好处"和"分离 Cache 的好处"是两回事：Cache 解决 CPU 与主存的速度不匹配；分离解决"取指和访存抢同一个 Cache 口"的结构冲突；别把前者的答案套到后者的题上。
- "结构冲突"和"数据冲突"是两回事：分离治的是两个访存部件抢硬件资源，不是数据相关。
- "命中率"和"冲突"是两回事：分离后每块 Cache 变小，命中率可能降；目的不是提命中率。

取指走指令 Cache、访存走数据 Cache，各走各的路，流水线结构冲突减少，选 D。

【坑位】

别选"降低 CPU 平均访存时间"。

平均访存时间＝命中时间＋缺失率×缺失损失，分离主要砍的是"流水线等 Cache 的停顿"，不是公式里的项。为什么错：把"不冲突"偷换成了"访存更快"；分离后单块 Cache 变小，缺失率甚至可能升，平均访存时间未必降。正确做法：题干出现"分离"就想"谁和谁抢资源"，答案必是冲突类表述。

【类比】

一个窗口又卖票又取票，两队人抢一个窗口；分成卖票窗和取票窗，各排各的队。术语贴回去：窗口就是"Cache 访问口"，两队人就是"取指部件和访存部件"，抢窗口就是"结构冲突"。

【口诀】 指令数据各走各路，流水线不抢 Cache 口。

【考频】 「Cache分离」在 2009–2022 年解析中出现 1 次 · 低频

第 17 题 · Store 指令中偏移量的取值范围（正确答案：A）

题干与选项：

某计算机有 16 个通用寄存器，采用 32 位定长指令字，操作码字段（含寻址方式位） 为 8 位，Store 指令的源操作数和目的操作数分别采用寄存器直接寻址和基址寻址方式。若 基址寄存器可使用任一通用寄存器，且偏移量用补码表示，则 Store 指令中偏移量的取值范 围是 。

- A. -32768 ~ +32767
- B. -32767 ~ +32768
- C. -65536 ~ +65535
- D. -65535 ~ +65536

入手点： 指令字一共 32 位，扣掉操作码、源寄存器号、基址寄存器号，剩下的位全是偏移量；偏移量用补码表示，范围就按补码算。

专有名词从零解释：

- 先想“定长指令字”：每条指令都一样长，本题是 32 位。
- 再想“操作码”：指令里表示“干什么”的字段，本题占 8 位。
- 再想“寄存器直接寻址”：操作数就在某个寄存器里，指令里只要写寄存器号；16 个通用寄存器，寄存器号要 4 位。
- 再想“基址寻址”：操作数的地址等于基址寄存器的值加上偏移量；指令里要写基址寄存器号和偏移量。
- 再想“补码表示”：偏移量可正可负，用补码存；n 位补码的范围是负 2 的 n-1 次方到正 2 的 n-1 次方减 1。

考查知识点：

- 先想寄存器号要几位：16 个通用寄存器，编号要 4 位。
- 再想偏移量占几位：32 位指令字，减去操作码 8 位，减去源寄存器号 4 位，减去基址寄存器号 4 位，剩下 16 位。
- 再想 16 位补码的范围：负 32768 到正 32767，就是 A。
- 拿生活例子验算：一条 32 格的纸条，操作码占 8 格，两个寄存器号各占 4 格，剩下 16 格写偏移量；16 格补码能写负 32768 到正 32767。
- 边界情况：如果偏移量不用补码、只表示非负数，范围就是 0 到 65535；本题说偏移量用补码表示，所以是负 32768 到正 32767。
- 易混对子：不要把“基址寄存器可用任一通用寄存器”理解成“基址寄存器号不用占位”——任一通用寄存器也要 4 位编号；也不要把补码范围记成对称的（正负一样大），补码负数比正数多一个。

正确选项为什么对： 偏移量 16 位，补码范围负 32768 到正 32767，选 A。

错误选项为什么错＋举例：

- B（负 32767 到正 32768）：把补码范围记反了；补码是负数多一个，不是正数多一个。教训：补码范围是负 2 的 n-1 次方到正 2 的 n-1 次方减 1。
- C（负 65536 到正 65535）：那是 17 位补码的范围；本题偏移量是 16 位，不是 17 位。教训：先算偏移量占几位，再算范围。
- D（负 65535 到正 65536）：也是 17 位的范围，而且正负还反了；教训：位数和正负方向都要对。

笔记本一句话： 第 17 题＋指令字里的偏移量：32 减操作码减两个寄存器号，剩下 16 位；16 位补码是负 32768 到正 32767。

【图像】



32 位 Store 指令字段划分图：8 位操作码、4 位源寄存器、4 位基址寄存器、16 位偏移量

【定位】 计算机组成原理 · 第4章 指令系统 · 指令格式与偏移量范围

【易混对比】

- "操作码字段（含寻址方式位）"和"纯操作码"是两回事：8 位里已经含了寻址方式位，别再单独扣一次。
- "基址寄存器可任选"和"不占位"是两回事：16 个里任选一个，编号照样占 4 位；"任选"不等于"免票"。
- "补码范围"和"对称范围"是两回事：16 位补码是 −32768～32767，负数多一个；别写成正负 32767 对称。

32−8−4−4＝16 位偏移量，补码范围 −32768～32767，选 A。

【坑位】

别把两个寄存器号数漏了。

源操作数寄存器直接寻址占 4 位，基址寻址的基址寄存器号也占 4 位，一共扣 8 位；只扣一个 4 位，偏移量就多算成 20 位。为什么错：基址寻址的"基址寄存器号"藏在题干后半句里，不画字段格容易漏。正确做法：画 32 格纸条，操作码 8 格、源 4 格、基址 4 格，剩下的格子全归偏移量。

【套路】

- 第一步，画字段格：操作码 8＋源寄存器 4＋基址寄存器 4。
- 第二步，算剩余位：32−16＝16。
- 第三步，套补码范围：−2 的 15 次方～2 的 15 次方−1。

指令字长减法定，剩下几位补码定范围。

【类比】

32 格纸条分蛋糕：操作码切 8 格，两个寄存器号各切 4 格，剩下的 16 格全归偏移量。术语贴回去：格子就是"二进制位"，偏移量的格子数就是"补码位数"，决定"取值范围"。

【口诀】 指令字长做减法，剩下几位定补码范围。

【考频】 「指令格式」在 2009–2022 年解析中出现 14 次 · 高频

第 18 题 · 微程序控制器中断定方式下下地址字段的最少位数（正确答案：C）

题干与选项：

某计算机采用微程序控制器，共有 32 条指令，公共的取指令微程序包含 2 条微指 令，各指令对应的微程序平均由 4 条微指令组成，采用断定法（下地址字段法）确定下条微 指令地址，则微指令中下址字段的位数至少是 。

- A. 5
- B. 6
- C. 8
- D. 9

入手点： 下地址字段要能区分所有的微指令；先数微指令一共有几条，再看几位二进制数能区分这么多条。

专有名词从零解释：

- 先想“微程序”：一条机器指令不是一步做完的，要拆成一串更小的操作，每一步叫一条微指令；一串微指令合起来，就是这条机器指令的微程序。
- 再想“控制存储器”：专门存微程序的地方；每条微指令存在一个地址里。
- 再想“断定法”：也叫下地址字段法，微指令里直接写着下一条微指令的地址；执行完这条，就按写好的地址去取下一条。
- 再想“公共取指微程序”：每条指令开始都要先取指令，这部分微程序是公共的，不用每条指令各写一份。
- 再想“下地址字段”：微指令里用来写下一条微指令地址的那几位；位数越多，能区分的地址越多。

考查知识点：

- 先想微指令总数：公共取指 2 条，加上 32 条指令、每条平均 4 条，就是 2 加 128，等于 130 条。
- 再想几位能区分 130 个地址：7 位二进制数能区分 128 个地址，128 比 130 少，不够；8 位能区分 256 个地址，够了；所以最少 8 位。
- 拿生活例子验算：130 间房要编号，7 位编号只能编 128 间，不够；8 位能编 256 间，够了；所以最少要 8 位。
- 边界情况：如果微指令总数刚好是 128 条，7 位就够了；本题是 130 条，多了 2 条，就必须进到 8 位。
- 易混对子：不要把“32 条指令”直接当成微指令总数——每条指令还有平均 4 条微指令，再加公共取指的 2 条，总数是 130，不是 32。

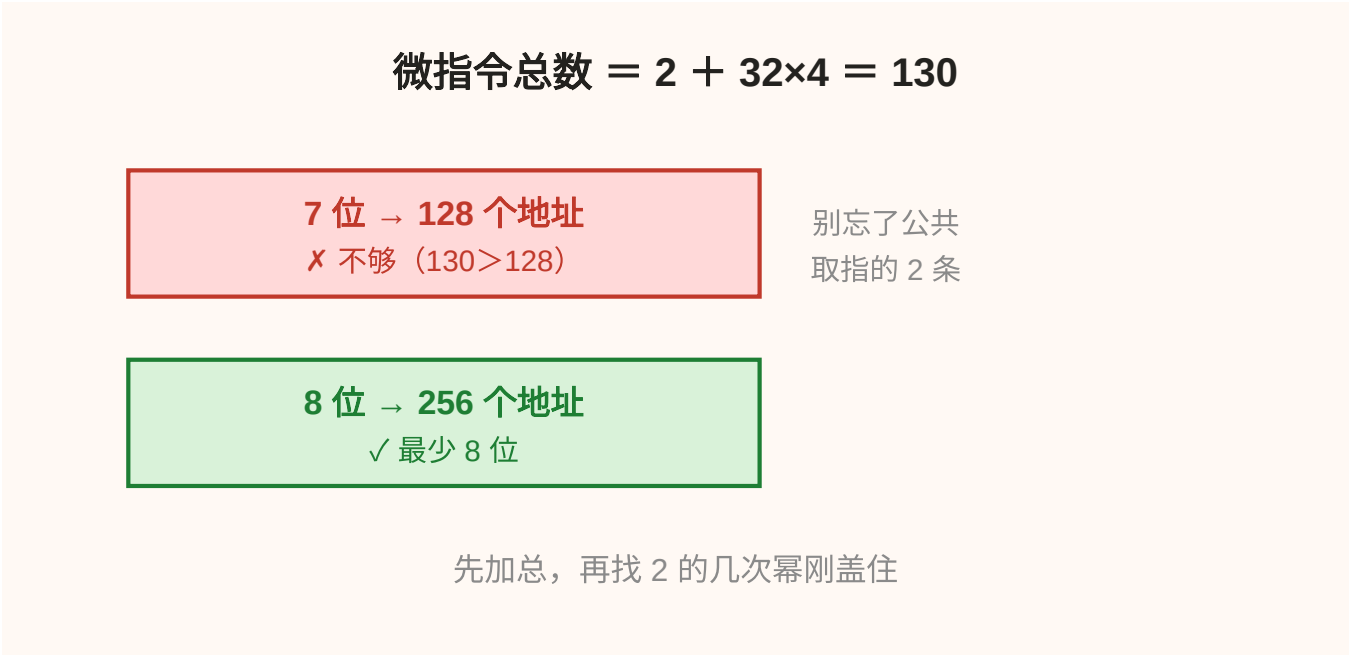
正确选项为什么对： 微指令一共 130 条，7 位不够、8 位够，所以最少 8 位，选 C。

错误选项为什么错＋举例：

- A（5）：5 位只能区分 32 个地址，只够区分 32 条指令，漏了每条指令的微指令数和公共取指。教训：先算微指令总数。
- B（6）：6 位只能区分 64 个地址，还不够 130 条。教训：2 的 6 次方是 64，不是 130。
- D（9）：9 位能区分 512 个地址，够是够，但不是最少；8 位已经够了，多一位就浪费了。教训：题目问“最少”，够了就行，不能多算。

笔记本一句话： 第 18 题＋下地址字段位数：先算微指令总数（本题 130 条），再找够用的最小位数（8 位）。

【图像】



微指令总数与地址位数示意图：130 条微指令，7 位只够 128，8 位最少够用

【定位】 计算机组成原理 · 第5章 中央处理器 · 微程序控制器下地址字段

【易混对比】

- "指令条数"和"微指令条数"是两回事：32 是机器指令条数，下地址要区分的是微指令（130 条）；别拿 32 直接算位数。
- "平均 4 条"和"每条 4 条"是两回事：平均值直接乘总数就行，题目给"平均"就是让你乘的。
- "至少几位"和"够用几位"是两回事：130 条，7 位只够 128，不够；8 位是"刚好够"的最小值。

微指令共 2＋32×4＝130 条，7 位不够，8 位最少，选 C。

【坑位】

别忘了公共取指的 2 条。

32×4＝128，看着"7 位刚好够"，加上公共取指的 2 条变成 130，7 位就爆了。为什么错：公共取指微程序藏在题干里，不起眼；128 是 2 的 7 次方这个"整"数太有迷惑性，让人停在 7 位。正确做法：先加后算——总数＝公共取指＋各指令微指令，先求和（130）再找 2 的幂。

【套路】

- 第一步，求微指令总数：公共取指 2＋32×4＝130。
- 第二步，找最小的 n 使 2 的 n 次方≥总数：2 的 7 次方＝128＜130，2 的 8 次方＝256≥130。

微指令数先加总，2 的几次幂刚盖住。

【类比】

130 间房编号：7 位门牌只能编 128 间，第 129、130 间没号；8 位门牌能编 256 间，够了。术语贴回去：门牌位数就是"下地址字段位数"，房间就是"微指令"。

【口诀】 总数加齐再找幂，七位一百二八不够用。

【考频】 「微程序」在 2009–2022 年解析中出现 6 次 · 中频

第 19 题 · 同步总线的最大数据传输率（正确答案：C）

题干与选项：

某同步总线采用数据线和地址线复用方式，其中地址/数据线有 32 根，总线时钟频 率为 66MHz，每个时钟周期传送两次数据(上升沿和下降沿各传送一次数据)，该总线的最大 数据传输率(总线带宽)是 。

- A. 132 MB/s
- B. 264 MB/s
- C. 528 MB/s
- D. 1056 MB/s

入手点： 每秒传多少次，再乘以每次传多少字节：每秒的周期数乘以每周期传的次数，再乘以每次传的字节数。

专有名词从零解释：

- 先想“同步总线”：大家跟着同一个时钟走的总线；时钟一到，数据就有效。
- 再想“地址线与数据线复用”：同一组线分时用，有时传地址，有时传数据；传数据时，32 根线都用来传数据。
- 再想“上升沿、下降沿”：时钟信号从低变高叫上升沿，从高变低叫下降沿；一个时钟周期里，上升沿和下降沿各有一次。
- 再想“传输率”：每秒能传多少字节，单位是 MB/s。

考查知识点：

- 先想每次传多少：32 根线就是 32 位，等于 4 个字节。
- 再想每周期传几次：上升沿传一次，下降沿传一次，一共 2 次；每周期传 8 个字节。
- 再想每秒传多少：时钟 66MHz，就是每秒 6600 万个周期；每周期 8 个字节，一共 528MB/s，就是 C。
- 拿生活例子验算：一条 4 车道的路，每分钟过 6600 批车，每批分两拨走（上坡一拨、下坡一拨），每拨 4 辆车；一分钟就是 6600 乘 8 辆车。
- 边界情况：如果只算上升沿，每周期只传 4 个字节，速率就是 264MB/s；本题上升沿和下降沿都传，所以要乘以 2。
- 易混对子：不要把“32 根线”当成“32 个字节”——32 根线是 32 位，等于 4 个字节；也不要把“MB/s”和“Mb/s”搞混——字节和比特差 8 倍。

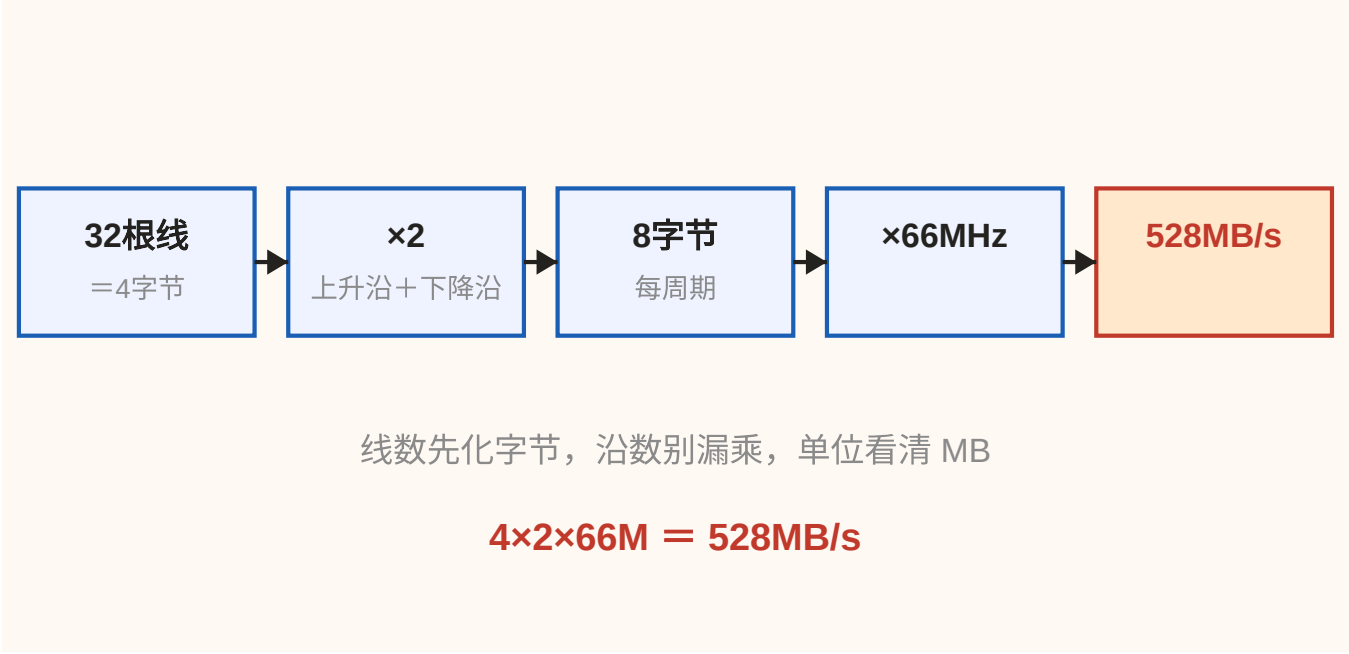
正确选项为什么对： 每周期 8 个字节，乘以 66MHz，等于 528MB/s，选 C。

错误选项为什么错＋举例：

- A（132MB/s）：把线数或时钟算少了（比如把 32 位按 8 位算，或者时钟只算了四分之一）；教训：32 根线是 4 个字节，不是 1 个字节。
- B（264MB/s）：只算了上升沿，没算下降沿，少乘了一个 2；教训：上升沿和下降沿各传一次，要乘以 2。
- D（1056MB/s）：多乘了一个 2（比如把“每周期 2 次”重复算了一遍）；教训：上升沿加下降沿一共是 2 次，不是 4 次。

笔记本一句话： 第 19 题＋总线传输率：每周期字节数乘以时钟频率；本题每周期 8 字节，8×66M＝528MB/s。

【图像】



总线带宽计算示意图：32 根线即 4 字节，每周期传 2 次，8 字节乘 66MHz 得 528MB/s

【定位】 计算机组成原理 · 第6章 总线 · 总线带宽计算

【易混对比】

- "32 根线"和"32 字节"是两回事：32 根线＝32 位＝4 字节；线数先除以 8 再算。
- "上升沿下降沿各一次"和"一次"是两回事：一个周期传 2 次，别漏乘 2；也别乘两遍变成 4。
- "地址数据复用"和"带宽打折"是两回事：复用只说明"传数据时 32 根全是数据线"，带宽按满 32 根算；别因为"复用"二字打折。

每周期 4 字节×2 次＝8 字节，8×66M＝528MB/s，选 C。

【坑位】

别把 Mb/s 和 MB/s 混成一锅。

66M×32 位＝2112Mb/s，看着像答案，其实是"比特每秒"；除以 8 才是 264MB/s（单沿），双沿再×2＝528MB/s。为什么错：选项里 264 和 528 长得像，就差一个 2，单位一混就掉进 B。正确做法：先把"位"换成"字节"，再乘频率，最后核对单位是大 B 还是小 b。

【套路】

- 第一步，一次传几字节：32 位÷8＝4 字节。
- 第二步，一周期传几次：上升＋下降＝2 次，共 8 字节。
- 第三步，乘频率并核单位：8×66M＝528MB/s。

线数先化字节，周期几次别漏乘，单位大小写核对。

【类比】

4 车道公路，每分钟过 6600 批车，每批分上坡下坡两拨走，一分钟 6600×8 辆。术语贴回去：车道就是"数据线"，一批车就是"一个时钟周期"，上坡下坡就是"上升沿下降沿"。

【口诀】 线数化字节，沿数别漏乘，频率最后乘。

【考频】 「总线带宽」在 2009–2022 年解析中出现 5 次 · 中频

第 20 题 · 总线突发传输方式的识别（正确答案：C）

题干与选项：

一次总线事务中，主设备只需给出一个首地址，从设备就能从首地址开始的若干连 续单元读出或写入多个数据。这种总线事务方式称为 。

- A. 并行传输
- B. 串行传输
- C. 突发传输
- D. 同步传输

入手点： 抓住关键词：一个首地址、连续多个数据；哪种传输方式就是“给一个地址、连续传一批”？

专有名词从零解释：

- 先想“总线事务”：一次完整的数据传送过程，从主设备发出请求、到从设备响应、再到数据传完。
- 再想“首地址”：一批数据的第一个数据的地址；给了首地址，后面的数据地址可以一个一个往下推，不用再给。
- 再想“突发传输”：也叫猝发传输，一次给一个首地址，连续传多个数据；不用每个数据都给一次地址，省时间。
- 再想“并行传输”：多根线同时传，每根线传一位，一次传好几位。
- 再想“串行传输”：一根线一位一位传，一次只传一位。
- 再想“同步传输”：跟着时钟节拍传，时钟到了数据就有效；这是“按什么节拍传”，不是“一次传几个数据”。

考查知识点：

- 先想题目的关键特征：只给一个首地址，连续读写多个数据；这就是突发传输的定义。
- 再想为什么不是并行传输：并行传输说的是“多根线同时传”，不是“一个地址连续多个数据”；题目没提几根线同时传。
- 再想为什么不是串行传输：串行传输说的是“一位一位传”，和“连续多个数据”不是一回事。
- 再想为什么不是同步传输：同步传输说的是“跟着时钟传”，任何传输都可以是同步的；题目问的是“这种方式”的名字，不是节拍。
- 拿生活例子验算：去仓库取货，报一个货架号，仓库管理员按顺序把后面几箱货连续搬出来，不用每箱都报一次号；这就是突发传输。
- 边界情况：突发传输里，首地址后面的数据地址是自动加 1 推出来的；如果每个数据都要单独给地址，那就不是突发传输，是普通的单次传输。
- 易混对子：不要把“连续”理解成“串行”——连续说的是多个数据一批传，串行说的是位一位一位传；也不要把“同步”当成方式的名字——同步是节拍，不是方式。

正确选项为什么对： 一个首地址、连续多个数据，就是突发传输的定义，选 C。

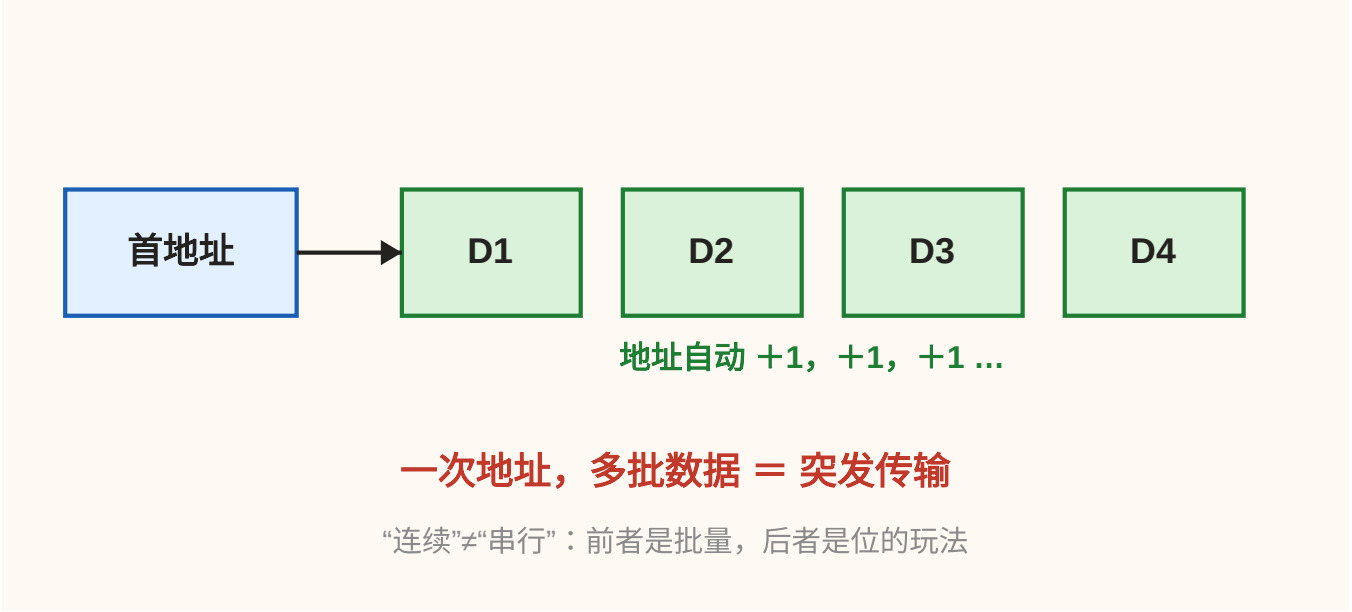
错误选项为什么错＋举例：

- A（并行传输）：并行说的是多根线同时传位，和“一个地址连续多个数据”没关系。教训：并行和串行说的是位的传法，不是数据的批法。
- B（串行传输）：串行说的是位一位一位传，和“连续多个数据”没关系。教训：连续多个数据不是串行。

- D（同步传输）：同步说的是跟着时钟节拍，和一次传几个数据没关系。教训：节拍和方式是两个维度。

笔记本一句话： 第 20 题＋突发传输：给一个首地址，连续读写多个数据，不用每个数据都给地址。

【图像】



突发传输时序示意图：只发一个首地址，后续数据块连续传送、地址自动加 1

【定位】 计算机组成原理 · 第6章 总线 · 总线事务方式

【易混对比】

- "突发（连续多个数据）"和"串行（一位一位）"是两回事：突发说的是"一批数据只给一个地址"，串行说的是"位"的传法；两个维度，别串。
- "突发"和"同步"是两回事：同步是"跟着时钟节拍"，任何方式都可同步；题干特征是一个首地址＋连续，这是方式名。
- "首地址"和"每个数据都给地址"是两回事：突发只给首地址，后面地址自动加 1；每个都给就不是突发。

一个首地址、连续读写多个数据＝突发传输，选 C。

【坑位】

别被"连续"二字勾去选串行。

"连续多个数据"是批量的事，串行是位的玩法；"连续"≠"串行"。为什么错：题干"连续"和选项"串行"共用一个"连/串"的语感，顺手就勾了。正确做法：抓题干的"地址给几次"——只给一次首地址，后面连续传，就是突发；问"位怎么传"时才轮到串行/并行。

【类比】

仓库取货：报一个货架号，管理员按顺序把后面几箱连续搬出来，不用每箱报一次号。术语贴回去：货架号就是"首地址"，连续搬出来的几箱就是"突发传送的一批数据"。

【口诀】 一地址带一串，突发传输不用逐个喊。

【考频】 「突发传输」在 2009–2022 年解析中出现 5 次 · 中频

第 21 题 · I/O 接口相关叙述的正误，问错误的一项（正确答案：D）

题干与选项：

下列有关 I/O 接口的叙述中，错误 的是 。

- A. 状态端口和控制端口可以合用同一个寄存器
- B. I/O 接口中 CPU 可访问的寄存器称为 I/O 端口
- C. 采用独立编址方式时，I/O 端口地址和主存地址可能相同
- D. 采用统一编址方式时，CPU 不能用访存指令访问 I/O 端口

入手点： 先把“独立编址”和“统一编址”分清：独立是两套地址空间，统一是一套地址空间；再看每条说法在哪种编址下才成立。

专有名词从零解释：

- 先想“I/O 接口”：夹在 CPU 和外设之间的电路；CPU 通过接口和外设打交道。
- 再想“I/O 端口”：接口里 CPU 能访问的寄存器；CPU 读写这些寄存器，就是在和外设打交道。
- 再想“状态端口”：读它能知道设备现在的状态（比如忙不忙、准备好没有）。
- 再想“控制端口”：写它能给设备发命令（比如开始打印、停止）。
- 再想“独立编址”：I/O 端口的地址和主存的地址各是一套，互不干扰；访问 I/O 端口要用专门的 I/O 指令。
- 再想“统一编址”：I/O 端口的地址和主存的地址是同一套，I/O 端口就编在主存地址空间里；访问 I/O 端口就用普通的访存指令。

考查知识点：

- 先想 A 为什么对：状态端口和控制端口可以合用同一个寄存器；读这个寄存器时，读到的是状态；写这个寄存器时，写进去的是控制命令；一个寄存器两种用法，省地方。
- 再想 B 为什么对：I/O 端口的定义就是“I/O 接口中 CPU 能访问的寄存器”；CPU 访问不了的寄存器（比如接口内部的工作寄存器），不叫端口。
- 再想 C 为什么对：独立编址是两套地址空间，I/O 端口地址和主存地址数值相同也没关系，因为访问时用的指令不一样（I/O 指令和访存指令），不会找错地方。
- 再想 D 为什么错：统一编址就是把 I/O 端口编进主存地址空间，CPU 正是用访存指令访问 I/O 端口的；说“统一编址时 CPU 不能用访存指令访问 I/O 端口”，正好说反了。
- 拿生活例子验算：独立编址像两个小区各有一套门牌号，1 号楼在两个小区都有，靠“小区名”区分；统一编址像一个小区一套门牌号，I/O 端口就是这个小区里的几户，找它们就用普通的找地址办法。
- 边界情况：独立编址下，CPU 访问 I/O 端口要用专门的 I/O 指令，不能用访存指令；统一编址下，CPU 访问 I/O 端口就用访存指令，不用专门的 I/O 指令；两种编址的访问办法正好反过来。
- 易混对子：不要把“独立编址”和“统一编址”的访问办法搞反——独立用 I/O 指令，统一用访存指令。

正确选项为什么对： 统一编址正是用访存指令访问 I/O 端口，D 说不能用，说反了，所以 D 是错误项，选 D。

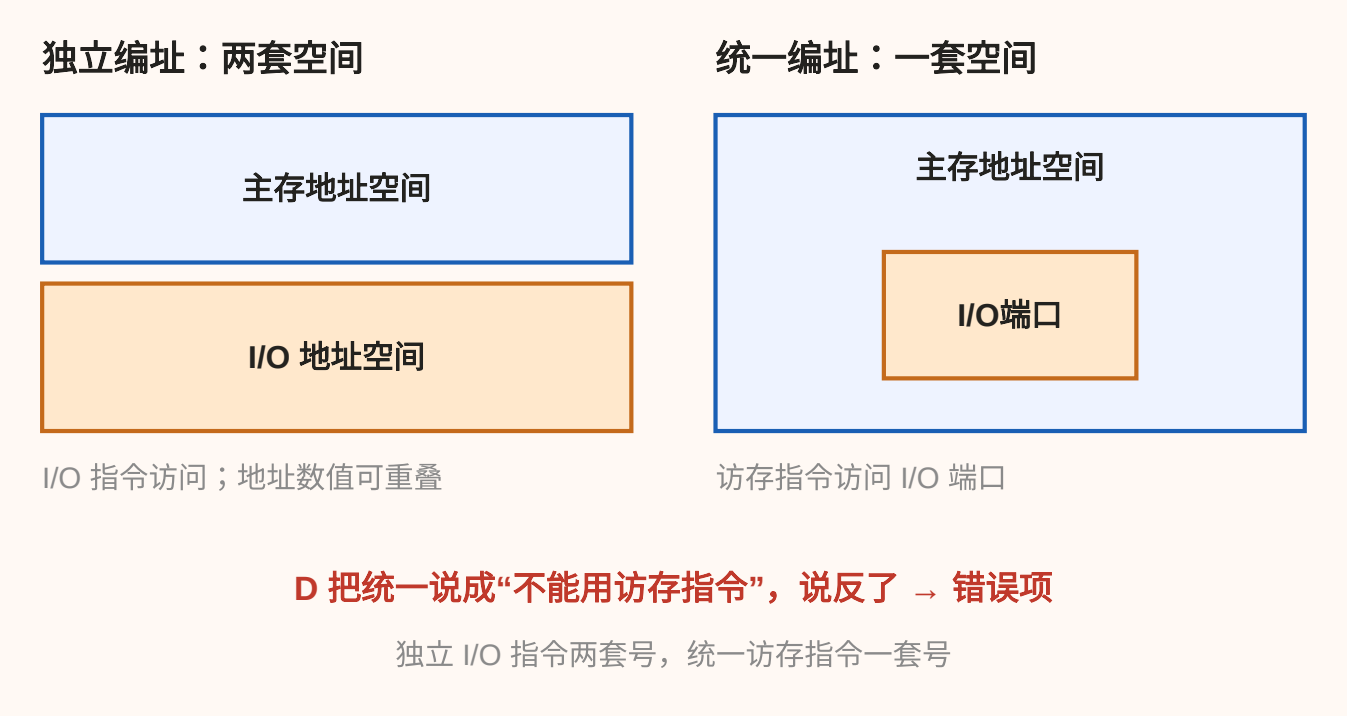
错误选项为什么错＋举例：

- A（状态端口与控制端口可合用同一寄存器）：读是状态、写是控制，一个寄存器两种用法，是对的；题目问错误项，所以 A 不是答案。教训：问错误项时，对的说法要排除。
- B（I/O 接口中 CPU 可访问的寄存器称 I/O 端口）：这就是端口的定义，是对的，不是答案。教训：定义题按定义答。

- C（独立编址时 I/O 端口地址与主存地址可能相同）：两套地址空间，数值相同不冲突，是对的，不是答案。教训：独立编址的地址空间是分开的。

笔记本一句话： 第 21 题＋I/O 编址：独立用 I/O 指令、地址可重叠；统一用访存指令；状态和控制可合用一个寄存器。

【图像】



I/O 编址方式对比图：独立编址两套空间用 I/O 指令，统一编址一套空间用访存指令

【定位】 计算机组成原理 · 第7章 输入输出系统 · I/O 接口与编址方式

【易混对比】

- "独立编址"和"统一编址"的访问指令是两回事：独立用专门的 I/O 指令，统一用访存指令；D 错就错在把统一的说成了"不能用访存指令"。
- "地址数值相同"和"地址冲突"是两回事：独立编址是两套空间，数值相同不冲突；别一看到"相同"就判错。
- "状态/控制合用一寄存器"和"读写打架"是两回事：读出是状态、写入是控制，方向不同不打架。

统一编址就是把 I/O 端口编进主存空间，CPU 用访存指令访问；D 说反了，是错误项，选 D。

【坑位】

问"错误的是"，别顺手选了"对的"。

A、B、C 逐条看都对，看到 C 对就想选 C，这是"找对的"惯性。为什么错：题干问的是"错误的"，判对三条不是目的，找出说反的那条才是。正确做法：四条逐条打勾打叉——A 合用可（勾）、B 端口定义对（勾）、C 独立可重叠（勾）、D 统一恰恰用访存指令、说反了（叉），叉的就是答案。

【类比】

独立编址像两个小区各一套门牌号：1 号楼两边都有，靠"小区名"（指令类型）区分；统一编址像一个小区一套门牌号，I/O 端口就是小区里的几户，找它们用普通的找地址办法（访存指令）。术语贴回去：小区名就是"编址方式"，找地址的办法就是"访问指令"。

【口诀】 独立 I/O 指令两套号，统一访存指令一套号。

【考频】 「I/O接口」在 2009–2022 年解析中出现 1 次 · 低频

第 22 题 · 中断处理占用 CPU 时间的比例（正确答案：B）

题干与选项：

若某设备中断请求的响应和处理时间为 100ns，每 400ns 发出一次中断请求，中断 响应所允许的最长延迟时间为 50ns，则在该设备持续工作过程中，CPU 用于该设备的 I/O 时间占整个 CPU 时间的百分比至少是 。

- A. 12.5%
- B. 25%
- C. 37.5%
- D. 50%

入手点： 每 400ns 一个周期里，CPU 有多少时间花在处理上；处理时间除以周期，就是占比。

专有名词从零解释：

- 先想“中断”：设备有事找 CPU，CPU 停下手头的事，先去处理设备的事。
- 再想“中断处理时间”：CPU 真正处理设备的事花的时间，本题是 100ns。
- 再想“中断请求周期”：设备两次发出请求之间隔多久，本题是 400ns。
- 再想“中断响应延迟”：从设备发出请求，到 CPU 开始处理，中间等的时间；本题允许最长等 50ns。
- 再想“时间占比”：CPU 花在某件事上的时间，除以总时间。

考查知识点：

- 先想一个周期里处理花多久：每 400ns 来一次请求，每次处理花 100ns；所以每 400ns 里，CPU 至少有 100ns 在处理。
- 再想占比：100 除以 400，等于 25%，就是 B。
- 再想 50ns 是干什么的：50ns 是响应延迟的上限——从请求发出到开始处理，最多只能等 50ns；这是保证请求不丢失的约束条件，不是处理时间；算占比时不把它加进去。
- 再想为什么是“至少”：处理时间是固定的 100ns，这是下限；如果处理时还被别的事打断，占比还会更高；所以 25% 是至少的比例。
- 拿生活例子验算：每 400 分钟来一个电话，每个电话打 100 分钟；打电话的时间占总时间的四分之一；电话响了之后最多等 50 分钟才接，这个等待时间不算打电话的时间。
- 边界情况：如果处理时间比请求周期还长（比如处理 500ns、周期 400ns），请求就会积压，系统就跟不上了；本题处理 100ns、周期 400ns，再加 50ns 延迟，一共 150ns，小于 400ns，系统跟得上。
- 易混对子：不要把“响应延迟”当成“处理时间”——延迟是等的时间，处理是干活的时间；算 CPU 花在 I/O 上的比例，只算干活的时间。

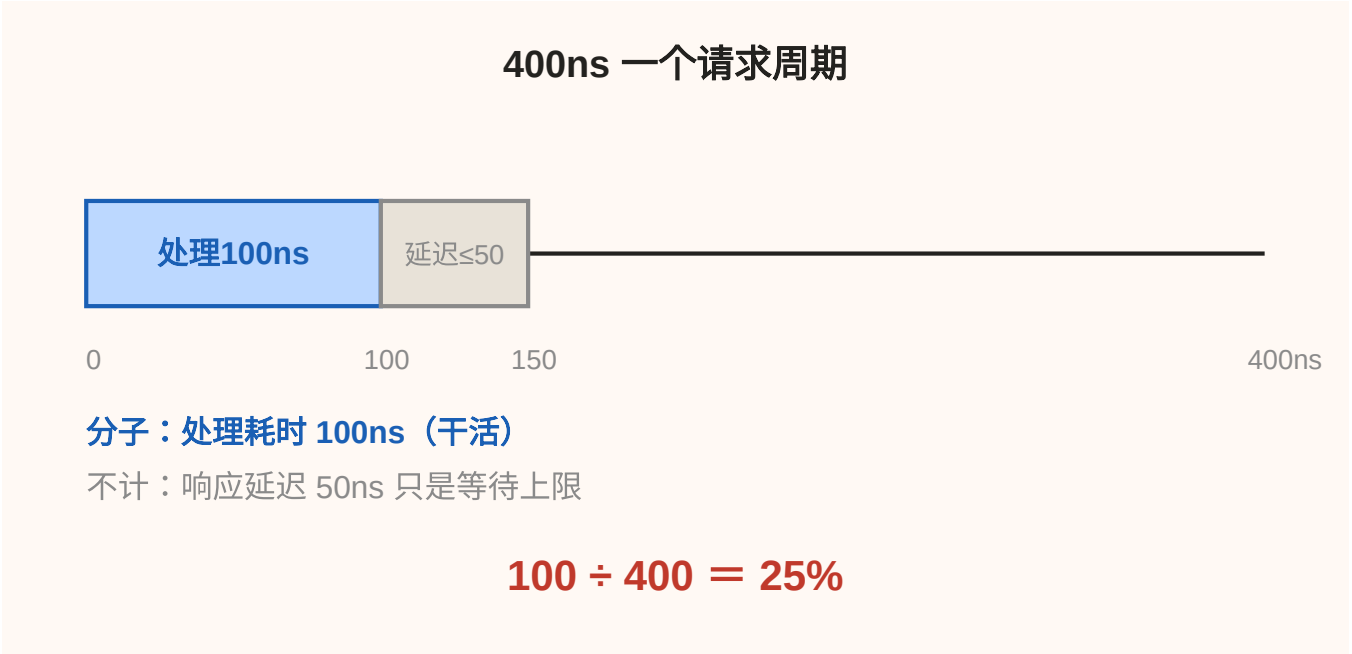
正确选项为什么对： 每 400ns 里处理花 100ns，100 除以 400 等于 25%，选 B。

错误选项为什么错＋举例：

- A（12.5%）：100 除以 800，把周期算成了两倍；教训：周期就是 400ns，不是 800ns。
- C（37.5%）：把 50ns 响应延迟也加进了处理时间，(100＋50)除以 400；教训：延迟是等的时间，不是干活的时间，不算进占比。
- D（50%）：200 除以 400，把处理时间多算了一倍；教训：每次处理就是 100ns，不是 200ns。

笔记本一句话： 第 22 题＋中断时间占比：处理时间除以请求周期；本题 100÷400＝25%；响应延迟不算处理时间。

【图像】



中断时间占比示意图：400ns 周期中 100ns 处理计入分子，50ns 延迟不计，得 25%

【定位】 计算机组成原理 · 第7章 输入输出系统 · 中断处理时间占比

【易混对比】

- "响应延迟"和"处理时间"是两回事：50ns 是"等"的时间，100ns 是"干活"的时间；占比只算干活的。
- "至少"和"正好"是两回事：100ns 是每周期固定的处理下限，还可能被打断更久；25% 是下限。
- "请求周期"和"处理时间"是两回事：400ns 是两次请求的间隔（分母），100ns 是处理耗时（分子）；别颠倒。

每 400ns 里处理花 100ns，100÷400＝25%；50ns 延迟是等待上限，不进分子，选 B。

【坑位】

别把 50ns 加进分子凑出 37.5%。

(100＋50)÷400＝37.5% 看着"都算上了"，实则错。为什么错：延迟是"请求发出到开始处理"的等待，CPU 在等的时候没在为这个设备干活；占比问的是"CPU 花在 I/O 上的时间"，只算干活。正确做法：分子只放处理耗时 100ns；50ns 是"系统跟得上"的约束条件（100＋50＜400），不是耗时。

【套路】

- 第一步，定分子分母：分子＝处理耗时 100ns，分母＝请求周期 400ns。
- 第二步，相除：100÷400＝25%。
- 第三步，剔干扰项：50ns 延迟是等待上限，不进分子。

中断占比＝处理除以周期，等待延迟不算数。

【类比】

每 400 分钟一个电话打 100 分钟，通话占 1/4；电话响后等 50 分钟才接，等待不算通话。术语贴回去：通话就是"中断处理"，等待接听就是"响应延迟"。

【口诀】 分子干活分母周期，等待时间靠边站。

【考频】 「中断处理」在 2009–2022 年解析中出现 29 次 · 高频

第 23 题 · 不会产生饥饿现象的调度算法（正确答案：A）

题干与选项：

下列调度算法中，不可能导致饥饿现象的是 。

- A. 时间片轮转
- B. 静态优先数调度
- C. 非抢占式短作业优先
- D. 抢占式短作业优先

入手点： 先想饥饿是什么意思：有的进程永远排不上队、永远得不到 CPU；再看哪种算法保证每个进程迟早轮到。

专有名词从零解释：

- 先想“饥饿”：也叫饿死，某个进程因为优先级低、或者作业长，永远被别的进程插队，永远得不到 CPU。
- 再想“时间片轮转”：每个进程轮流用一小段时间（时间片），时间到了就换下一个；不管优先级高低，人人都有份。
- 再想“静态优先数调度”：每个进程的优先级一开始就定死了，运行中不变；优先级高的先跑，优先级低的后跑。
- 再想“短作业优先”：作业越短越先跑；分抢占式（新来的短作业能抢正在跑的长作业的 CPU）和非抢占式（正在跑的作业不被抢，等它跑完再换）。

考查知识点：

- 先想时间片轮转为什么不会饥饿：人人轮流，时间片到了就换下一个；不管优先级高低，每个进程迟早轮到，不会永远等不到。
- 再想静态优先数为什么会饥饿：优先级低的进程，前面永远有优先级高的进程插队；如果高优先级的进程源源不断来，低优先级的就永远轮不到。
- 再想非抢占式短作业优先为什么会饥饿：长作业排在后面，前面永远有短作业插队；短作业源源不断来，长作业就永远轮不到。
- 再想抢占式短作业优先为什么会饥饿：正在跑的长作业，还会被新来的短作业抢走 CPU；短作业不断来，长作业就永远跑不完。
- 拿生活例子验算：时间片轮转像大家轮流发言，每人说一分钟，到点就换人，人人都有机会；优先数和短作业优先像“重要的人先说”“话短的人先说”，不重要、话长的人可能永远说不上话。
- 边界情况：时间片轮转里，时间片太大就接近先来先服务，时间片太小就切换太频繁、开销大；但不管时间片多大或多小，人人轮流的性质不变，不会饥饿。
- 易混对子：不要把“先来先服务”和“时间片轮转”搞混——先来先服务是排队顺序，不会饥饿，但平均等待时间长；时间片轮转是轮流，也不会饥饿；本题四个选项里，只有时间片轮转保证不饥饿。

正确选项为什么对： 时间片轮转人人轮流，每个进程迟早轮到，不会饥饿，选 A。

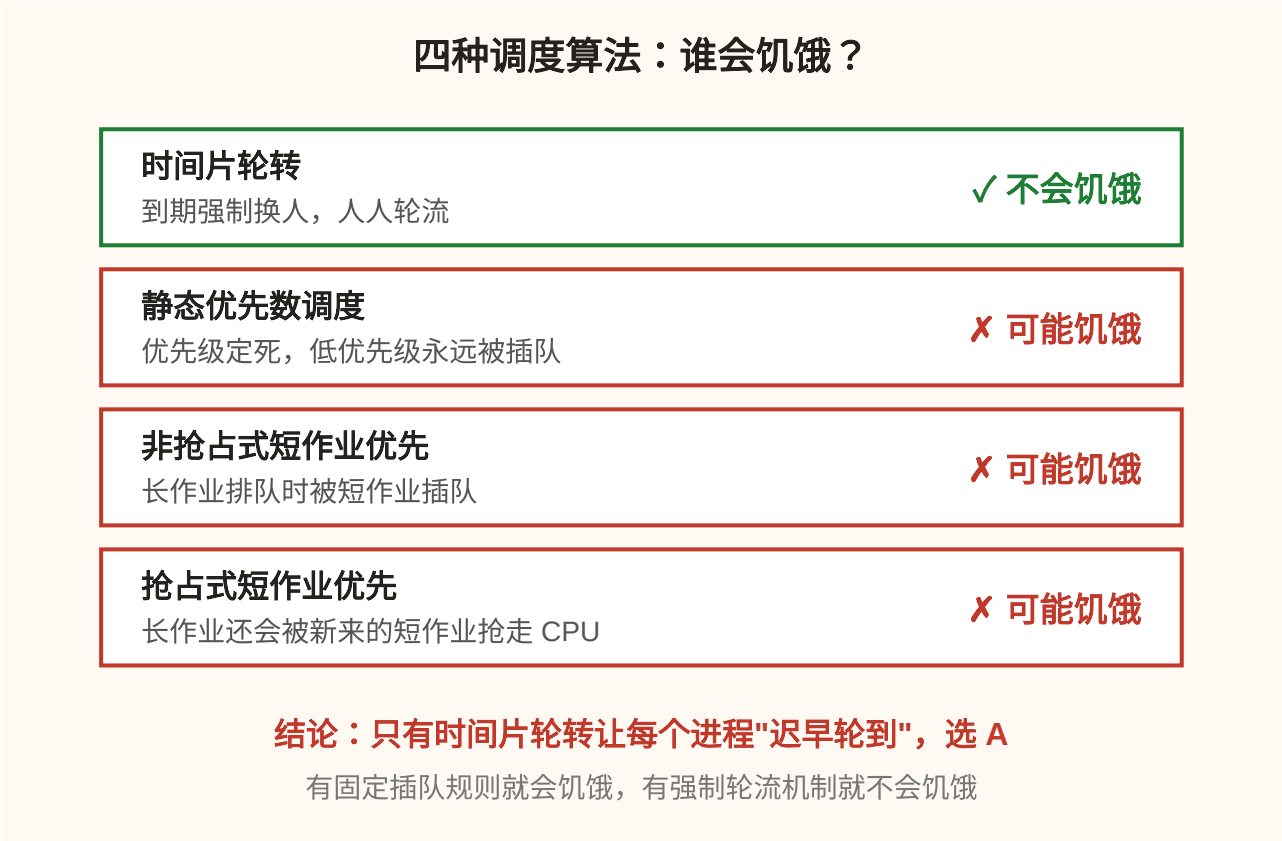
错误选项为什么错＋举例：

- B（静态优先数调度）：优先级低的可能永远被高优先级的插队，会饥饿。教训：优先级固定不变，低优先级的没有出头之日。

- C（非抢占式短作业优先）：长作业可能永远被短作业插队，会饥饿。教训：短作业不断来，长作业永远排不上。
- D（抢占式短作业优先）：长作业不仅排不上，还会被新来的短作业抢走 CPU，更会饥饿。教训：抢占让长作业更难跑完。

笔记本一句话： 第 23 题＋不饥饿的调度：时间片轮转人人轮流，不会饥饿；静态优先数、短作业优先（抢占或非抢占）都可能饥饿。

【图像】



四种调度算法饥饿情况对比：时间片轮转人人轮流不饥饿，其余三者都可能饥饿

【定位】 操作系统 · 第2章 进程管理 · 调度算法与饥饿现象

【易混对比】

- "时间片轮转"和"先来先服务"的区别：两者都不会让进程饿死，但方式不同——轮转靠"到期强制换人"保证轮流，先来先服务靠"先到先得"保证不插队；本题选项里只有轮转。
- "静态优先数调度"和"短作业优先"的区别：前者饿死的是"优先级低"的进程，后者饿死的是"作业长"的进程；一个看优先级、一个看作业长度，但都会被别的进程插队。
- "抢占式短作业优先"和"非抢占式短作业优先"的区别：抢占式里长作业连正在跑都可能被新来的短作业抢走，饥饿更严重；非抢占式里长作业跑上之后不会被抢，但排队时一样可能被短作业插队。

只有"人人轮流"的算法不会饥饿：时间片轮转到期强制换人，每个进程迟早轮到，选 A。

【坑位】

别把"动态优先数（老化）"的结论套到"静态优先数"上。

看到"优先数调度"就想到"优先级会随等待时间提高、不会饥饿"——那是动态优先数加老化机制的结论，本题明确写的是"静态优先数"，优先级定死不变，低优先级的进程永远被高优先级的插队，会饥饿。正确做法：先看清"静态"二字，再判饥饿。

【套路】

- 第一步，找"谁可能永远轮不到"：看算法有没有固定插队规则——固定优先级、按作业长短排队，都有插队风险。
- 第二步，找"谁保证人人轮到"：看算法有没有强制轮流机制——时间片到期强制换人，人人有份。
- 第三步，对照选项：只有时间片轮转满足第二条。

有固定插队规则就会饥饿，有强制轮流机制就不会饥饿。

【类比】

像班级轮流值日：时间片轮转是"每天换一个人值日"，人人迟早轮到；静态优先数是"班干部永远先挑轻松的活"，普通同学可能永远挑不到；短作业优先是"活少的先干"，活多的可能永远排不上。术语贴回去：值日顺序就是"调度算法"，永远挑不到活就是"饥饿"。

【口诀】 轮流值日人人到，固定优先会饿倒，长短排队也挨饿。

【考频】 「饥饿现象」在 2009–2022 年解析中出现 14 次 · 高频

第 24 题 · 避免死锁所需的最少同类设备数（正确答案：B）

题干与选项：

某系统有 n 台互斥使用的同类设备，三个并发进程分别需要 3、4、5 台设备，可确保系统不发生死锁的设备数 n 最小为 。

- A. 9
- B. 10
- C. 11
- D. 12

入手点： 想最危险的情况：每个进程都差 1 台就够，但系统一台都不剩了，那就僵住了；只要系统多留 1 台机动，就能解开。

专有名词从零解释：

- 先想“死锁”：几个进程互相等对方手里的资源，谁也走不下去，僵在那里的状态。
- 再想“互斥使用”：同一台设备同一时间只能给一个进程用，不能两个人同时用。
- 再想“避免死锁”：事先算好，保证系统永远不会走到死锁的状态；办法就是留一手，不把资源全分光。

考查知识点：

- 先想最危险的情况：第一个进程拿了 2 台（差 1 台就够），第二个进程拿了 3 台（差 1 台就够），第三个进程拿了 4 台（差 1 台就够）；三个进程都差 1 台，加起来拿了 9 台。
- 再想这时系统还剩几台：如果系统一共只有 9 台，全被拿走了，一台不剩，三个进程都等着最后 1 台，谁也走不下去，死锁了。
- 再想多 1 台会怎样：如果系统有 10 台，拿了 9 台后还剩 1 台；把这 1 台给任意一个进程（比如给第一个进程），它就够了，跑完释放 3 台；剩下的进程接着跑，链条解开了，不死锁。
- 再想最少是几台：2 加 3 加 4 再加 1，等于 10，就是 B。
- 拿生活例子验算：三个人分别要 3、4、5 把椅子，每个人都差 1 把就够；如果一共只有 9 把椅子，全分光了，三个人都差 1 把，僵住了；多 1 把椅子（10 把），给任意一个人，他坐满了站起来，椅子还回来，其他人接着坐，就解开了。
- 边界情况：这个算法的前提是“每个进程最多需要这么多台”；如果某个进程实际需要的比说的多，算出来的最少台数就不够了。
- 易混对子：不要把“三个进程需要的总数”（3 加 4 加 5 等于 12）当成答案——总数是“每个人都拿满”的情况，避免死锁不需要这么多；也不要把“最危险时已拿的总数”（9 台）当成答案——9 台刚好僵住，还要多 1 台机动。

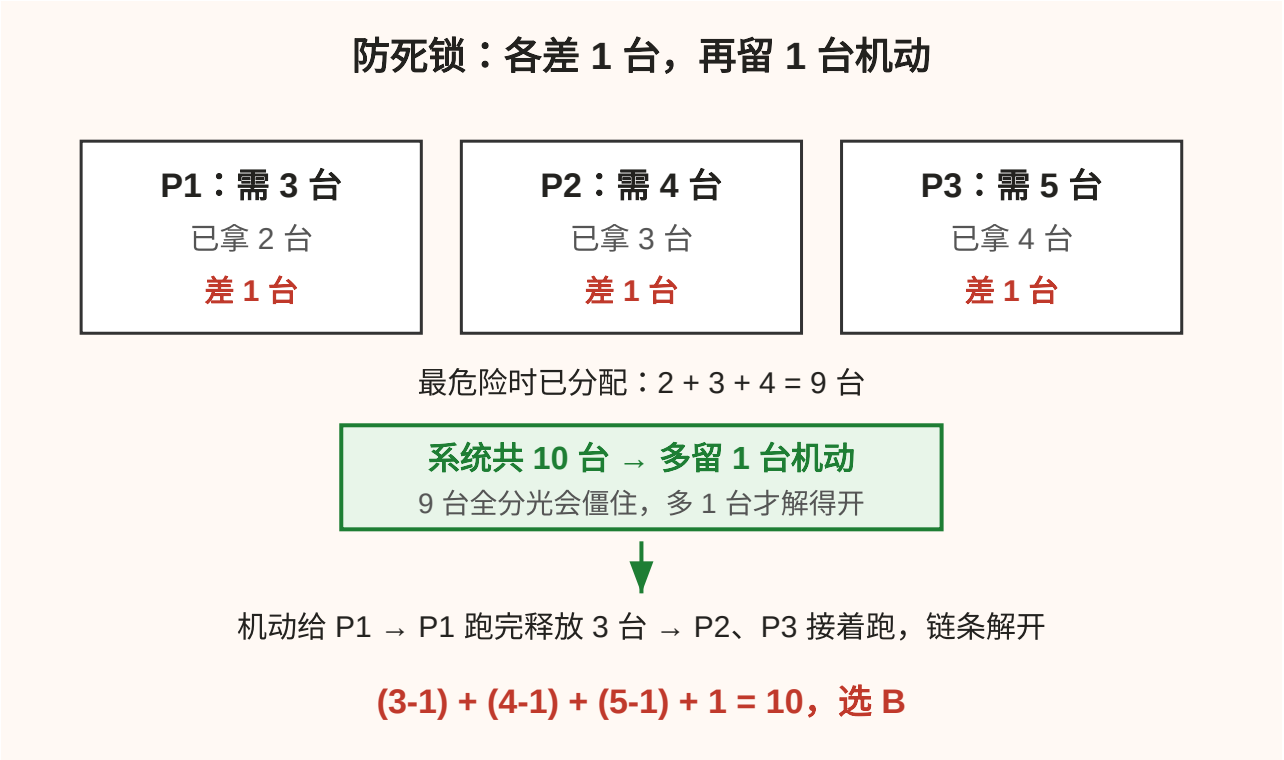
正确选项为什么对： 每个进程差 1 台时已拿 2、3、4 台，再多 1 台机动，一共 10 台，保证不死锁，选 B。

错误选项为什么错＋举例：

- A（9）：9 台刚好是三个进程各差 1 台时已拿的总数，一台不剩，僵住了，死锁；教训：最危险时已拿的总数还不够，要多 1 台机动。
- C（11）：够是够，但不是最少；10 台已经保证不死锁了，多 1 台是浪费。教训：题目问“最少”，够了就行。
- D（12）：那是三个进程需要的总数，每个人都拿满；避免死锁不需要这么多。教训：避免死锁和满足所有人是两回事。

笔记本一句话： 第 24 题＋防死锁最少设备：每进程差 1 台时已拿的总数，再加 1 台机动；本题 (3-1)＋(4-1)＋(5-1)＋1＝10。

【图像】



三个进程各差1台、系统再多留1台机动的死锁避免示意图

【定位】 操作系统 · 第2章 进程管理 · 死锁避免与资源分配

【易混对比】

- "最少设备数"和"设备总数需求"的区别：前者是保证不死锁的下限（本题 10），后者是三个进程同时拿满的总和（本题 12）；题目问的是下限，不是总和。
- "最危险时已拿总数"和"最少设备数"的区别：前者是各差 1 台时已分配出去的 2+3+4=9，后者还要再多 1 台机动才够 10；9 台刚好僵住。
- "死锁"和"饥饿"的区别：死锁是几个进程互相等对方的资源、谁也走不下去；饥饿是一个进程永远轮不到；本题是资源不够分的死锁问题。

每进程差 1 台时已拿 (3-1)+(4-1)+(5-1)=9 台，再多 1 台机动解开链条，最少 10 台，选 B。

【坑位】

别把"三个进程需要的总数 12"当成最少设备数。

12 是"每个人都拿满"的资源总数，避免死锁根本不需要这么多；防死锁的思路是"留一手"：先算出最危险时已分配的 9 台，再多留 1 台给任意一个进程跑完释放，链条就解开了。正确做法：先写 (需求-1) 相加，再 +1。

【套路】

- 第一步，写每个进程"差 1 台"时已拿的数：3-1=2，4-1=3，5-1=4。
- 第二步，相加再 +1 台机动：2+3+4+1=10。
- 第三步，对照选项：A(9) 是已拿总数（会僵住），D(12) 是拿满总数（不是最少）。

差一相加再加一，就是防死锁的最小数。

【类比】

像三个人分椅子，每人分别要 3、4、5 把，且每人都差 1 把就够：如果一共只有 9 把，全分光了，三个人都差 1 把、僵在那里；多 1 把（10 把），把这把给任意一个人，他坐满了站起来，椅子还回来，其他人接着坐，僵局就解开了。术语贴回去：椅子就是"互斥使用的同类设备"，多留的那 1 把就是"机动资源"。

【口诀】 各差一台先相加，再加一台留机动。

【考频】 「死锁」在 2009–2022 年解析中出现 13 次 · 高频

第 25 题 · 不能在用户态执行的指令（正确答案：D）

题干与选项：

下列指令中，不能在用户态执行的是 。

- A. trap 指令
- B. 跳转指令
- C. 压栈指令
- D. 关中断指令

入手点： 先分清哪些指令是特权指令（只有内核态能执行），哪些是普通指令（用户态也能执行）；特权指令就是答案。

专有名词从零解释：

- 先想“用户态”：普通程序跑的状态，权限小；只能执行普通指令，不能碰关键资源。
- 再想“内核态”：操作系统跑的状态，权限大；能执行所有指令，包括特权指令。
- 再想“特权指令”：只有内核态能执行的指令，比如关中断、改页表、停机；用户程序执行特权指令，CPU 会产生异常，进内核处理。
- 再想“trap 指令”：也叫陷入指令，程序在用户态执行它，主动要求进内核；它是用户态进内核的门，不是特权指令本身。

考查知识点：

- 先想关中断指令为什么不能在用户态执行：关中断是特权指令；如果用户程序能随便关中断，它就可以关掉所有中断、独占 CPU，系统就乱套了；所以关中断只能在内核态执行。
- 再想 trap 指令为什么能在用户态执行：trap 指令就是设计给用户程序用的——用户程序想请操作系统帮忙（比如读写文件），就执行 trap 指令，主动进内核；它本身不是特权指令。
- 再想跳转指令为什么能在用户态执行：跳转只是改变程序跑到哪里去，普通程序天天都在跳转，不涉及关键资源。
- 再想压栈指令为什么能在用户态执行：压栈只是把数据压进自己程序的栈里，普通操作，不涉及关键资源。
- 拿生活例子验算：用户态像普通员工，内核态像老板；关中断像“把全公司的电话都掐了”，只有老板能干；trap 像“打电话请老板帮忙”，员工本来就能打；跳转和压栈像“在本子上记笔记”，员工天天干。
- 边界情况：用户程序如果偷偷执行特权指令，CPU 不会真的执行它，而是产生特权指令异常，进内核处理；所以“不能在用户态执行”的意思不是“执行了会死机”，而是“执行了会进内核处理，不会真的执行”。
- 易混对子：不要把“trap 指令进内核”和“trap 是特权指令”搞混——trap 是在用户态执行、用来进内核的指令，不是特权指令；特权指令是在内核态才能执行的指令。

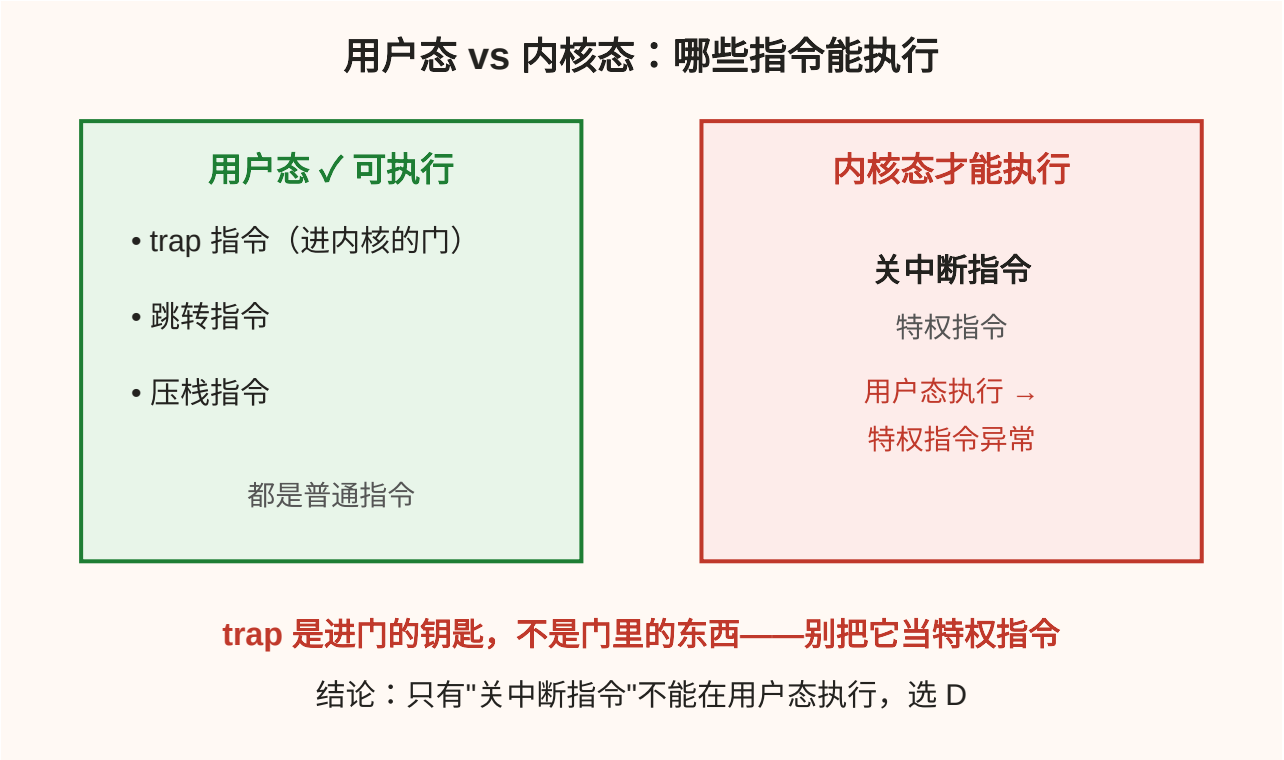
正确选项为什么对： 关中断是特权指令，只能在内核态执行，用户态不能执行，选 D。

错误选项为什么错＋举例：

- A（trap 指令）：trap 就是在用户态执行、主动进内核的指令；教训：trap 是进内核的门，不是特权指令。
- B（跳转指令）：跳转是普通指令，用户程序天天用；教训：跳转不涉及关键资源。
- C（压栈指令）：压栈是普通操作，用户程序天天用；教训：压栈不涉及关键资源。

笔记本一句话： 第 25 题＋用户态禁区：关中断是特权指令，只能在内核态执行；trap 是用户态进内核的门，跳转、压栈是普通指令。

【图像】



用户态与内核态指令权限对照：关中断只能在内核态，trap/跳转/压栈可在用户态

【定位】 操作系统 · 第1章 计算机系统概述 · 特权指令与用户态

【易混对比】

- "trap 指令"和"特权指令"的区别：trap 是在用户态执行、主动敲门进内核的指令；特权指令是只能在内核态执行的指令；一个是进门的钥匙，一个是门里的东西。
- "关中断指令"和"开中断指令"的区别：没有区别——两者都是特权指令，都只能在内核态执行；别以为"关"特权、"开"不特权。
- "跳转指令"和"trap 指令"的区别：跳转只是换个地方继续跑，普通操作；trap 是主动请求操作系统服务，会切换到内核态。

关中断是特权指令，只能在内核态执行，用户态执行会触发特权指令异常；其余三条用户态都能执行，选 D。

【坑位】

别看到 trap 会"进入内核"就把它当成特权指令。

trap 的设计目的就是给用户程序用的——用户程序想请操作系统帮忙（读写文件、申请内存），执行 trap 主动进内核；它本身在用户态执行，不是特权指令。正确做法：判特权看"能不能在用户态执行"，trap 恰恰是在用户态执行的。

【类比】

像公司门禁：普通员工（用户态）不能碰总电闸（关中断），碰了会报警（特权指令异常）；但员工可以打电话请老板帮忙（trap），可以自己在本子上记笔记、翻页（压栈、跳转）。术语贴回去：总电闸就是"特权指令"，打电话就是"访管指令"，报警就是"特权指令异常"。

【口诀】 关中断是总电闸，内核才能碰一下；trap 是电话不是闸，用户态里把门敲。

【考频】 「特权指令」在 2009–2022 年解析中出现 23 次 · 高频

第 26 题 · 磁盘操作完成后进程状态的变化（正确答案：A）

题干与选项：

一个进程的读磁盘操作完成后，操作系统针对该进程必做的是 。

- A. 修改进程状态为就绪态
- B. 降低进程优先级
- C. 给进程分配用户内存空间
- D. 增加进程时间片大小

入手点： 先想进程在等磁盘时是什么状态，再想等的东西到了，状态应该变成什么：等的东西到了，就不用等了，但不一定马上跑。

专有名词从零解释：

- 先想“阻塞态”：也叫等待态，进程在等某个东西（比如磁盘操作完成），东西没到之前不能跑，哪怕 CPU 空着也不能跑。
- 再想“就绪态”：进程已经准备好了、万事俱备，只等 CPU 空出来就能跑的状态。
- 再想“执行态”：进程正在占用 CPU、正在跑的状态。
- 再想“磁盘操作完成”：进程等的磁盘读写做完了，结果已经准备好了。

考查知识点：

- 先想进程等磁盘时是什么状态：进程在等磁盘操作完成，是阻塞态。
- 再想磁盘操作完成后，进程还用等吗：不用等了，结果已经到了；所以进程从阻塞态出来，变成就绪态，等 CPU 调度。
- 再想为什么不是直接变成执行态：就绪态的意思是“准备好了、等 CPU”；CPU 可能正被别的进程用着，这个进程要排队等 CPU 空出来；所以是就绪态，不是执行态。
- 再想为什么其他选项不是“必做”：降低优先级、分配内存、增大时间片，这些都不是磁盘操作完成时一定要做的事；有的系统可能会做，但不是“必做”。
- 拿生活例子验算：等外卖时是阻塞态（外卖没到，不能吃）；外卖到了，就是就绪态（可以吃了，等桌子空出来）；不是外卖一到就马上吃上（桌子可能正被别人占着）。
- 边界情况：如果 CPU 正好空着，进程从就绪态马上就被调度成执行态；但“磁盘操作完成”这个事件本身，只保证进程变成就绪态，不保证马上变成执行态。
- 易混对子：不要把“就绪态”和“执行态”搞混——就绪是“准备好了、等 CPU”，执行是“正在用 CPU”；磁盘操作完成只保证“不用等了”，不保证“马上用 CPU”。

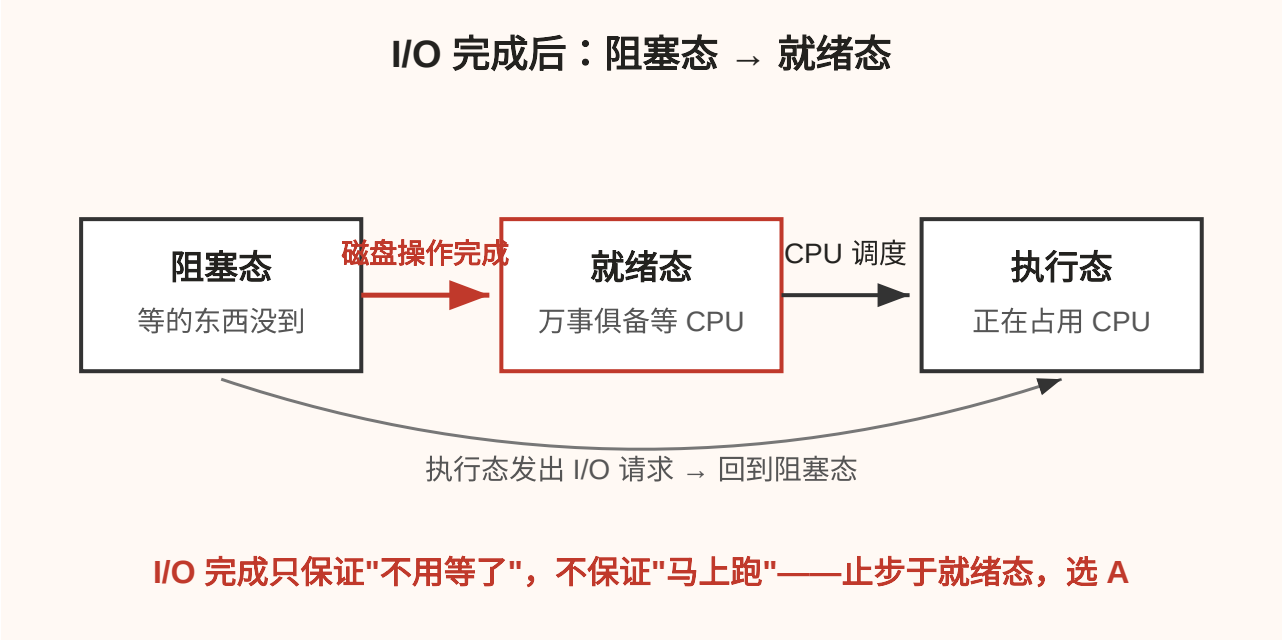
正确选项为什么对： 磁盘操作完成，进程不用等了，从阻塞态变成就绪态，选 A。

错误选项为什么错＋举例：

- B（降低其优先级）：优先级调整不是磁盘操作完成时的必做动作；教训：题目问“必做”，不是“可能做”。
- C（给其分配用户内存空间）：读磁盘完成和分配内存没关系；教训：不要把不相干的事扯进来。
- D（增大其时间片大小）：时间片大小不是磁盘操作完成时必改的；教训：时间片调整和 I/O 完成是两回事。

笔记本一句话： 第 26 题＋I/O 完成状态变：阻塞等东西，东西到了变就绪（等 CPU），不是直接变执行。

【图像】



进程状态转换：阻塞态等I/O完成变为就绪态，再等CPU调度变为执行态

【定位】 操作系统 · 第2章 进程管理 · 进程状态转换

【易混对比】

- "就绪态"和"执行态"的区别：就绪是"万事俱备、只等 CPU"，执行是"正在占用 CPU 跑"；I/O 完成只解决"等的东西到了"，不解决"CPU 轮到我"。
- "必做"和"可能做"的区别：题目问"必做"，降低优先级、调时间片某些系统可能做，但不是每次 I/O 完成都发生；只有"阻塞变就绪"是必然的。
- "阻塞态"和"就绪态"的区别：阻塞是"等的东西没到、CPU 空着也不能跑"，就绪是"东西齐了、只等 CPU"；I/O 完成就是把"等"字去掉。

等的磁盘操作完成了，进程从阻塞态变为就绪态（等 CPU 调度），而不是直接执行，选 A。

【坑位】

别把"I/O 完成"理解成"马上轮到它跑"。

I/O 完成只保证"不用再等了"，CPU 可能正被别的进程占着，这个进程要排队等调度；直接跳到执行态需要"CPU 正好空着"这个额外条件，题目没给。正确做法：I/O 完成 → 就绪态，止步，不再往前推。

【类比】

像等外卖：外卖没到时是"干等"（阻塞态），外卖到了就是"可以吃了"（就绪态）；但能不能马上吃，还要看桌子空没空（CPU 空没空），桌子被别人占着就得排队。术语贴回去：外卖就是"磁盘操作的结果"，桌子就是"CPU"。

【口诀】 东西到了不用等，阻塞变就绪排队等 CPU。

【考频】 「进程状态」在 2009–2022 年解析中出现 11 次 · 高频

第 27 题 · 用位图法管理磁盘空间所需的簇数（正确答案：A）

题干与选项：

现有一个容量为 10GB 的磁盘分区，磁盘空间以簇(Cluster)为单位进行分配，簇的 大小为 4KB，若采用位图法管理该分区的空闲空间，即用一位(bit)标识一个簇是否被分配， 则存放该位图所需簇的个数为 。

- A. 80
- B. 320
- C. 80K
- D. 320K

入手点： 三步算：先算一共多少簇，再算位图占多少字节，最后把位图字节数换算成簇数。

专有名词从零解释：

- 先想“簇”：磁盘分区里分配空间的最小单位；文件占空间，都是按簇算的。
- 再想“位图法”：用一位（0 或者 1）表示一个簇是空闲还是已分配；所有位排成一张表，就是位图；1 位管一个簇。
- 再想“GB、KB”：1GB 是 2 的 30 次方字节，1KB 是 2 的 10 次方字节；算的时候要用 1024 进制，不能用 1000。

考查知识点：

- 先想一共多少簇：10GB 除以 4KB，等于 10 乘 2 的 30 次方除以 4 乘 2 的 10 次方，等于 2.5 乘 2 的 20 次方，等于 2621440 个簇。
- 再想位图占多少字节：一位管一个簇，2621440 位，除以 8（一位变一字节要除以 8），等于 327680 字节，等于 320KB。
- 再想位图占几个簇：320KB 除以 4KB，等于 80 簇，就是 A。
- 拿生活例子验算：2621440 个座位，每个座位用 1 位表示空闲还是已占，位图表就是 2621440 位，合 320KB；320KB 的表放在 4KB 一间的房间里，要 80 间房。
- 边界情况：如果位图字节数除以簇大小有余数，要向上取整（多出来的半间房也要算一间）；本题 320KB 除以 4KB 正好是 80，没有余数。
- 易混对子：不要把“位”和“字节”搞混——一位管一个簇，位图字节数是簇数除以 8；也不要把“位图占多少 KB”和“位图占几个簇”搞混——320KB 是字节数，80 簇是簇数，中间还要除以 4KB。

正确选项为什么对： 簇数 2621440，位图 320KB，320KB 除以 4KB 等于 80 簇，选 A。

错误选项为什么错＋举例：

- B（320）：那是位图的 KB 数（320KB），不是簇数；忘了最后除以 4KB。教训：字节数和簇数中间还要除一次簇大小。
- C（80K）：把 80 簇后面的单位搞错了，80K 簇是 80 乘 1024 簇；教训：簇数就是 80，单位不能乱加 K。
- D（320K）：把位图的字节数（327680 字节，约等于 320K）直接当成了簇数；教训：字节数要除以簇大小（4KB），才是簇数。

笔记本一句话： 第 27 题＋位图占几个簇：簇数除以 8 得字节数，再除以簇大小；本题 2621440 位→320KB→80 簇。

【图像】



位图占簇数三步计算链：总簇数→位图字节数→位图簇数

【定位】 操作系统 · 第4章 文件管理 · 空闲空间管理（位图法）

【易混对比】

- "位（bit）"和"字节（Byte）"的区别：1 位管 1 个簇，位图总位数 = 簇总数；换算成字节要除以 8，漏了这一步就会把 2621440 直接当字节。
- "位图占多少 KB"和"位图占几个簇"的区别：前者是 320KB（字节数），后者是 80 簇；中间还要除以簇大小 4KB，别拿字节数直接当簇数。
- "80 簇"和"80K 簇"的区别：答案就是 80 个簇，单位不能乱加 K；80K 簇是 80×1024 个簇，差了 1024 倍。

三步：总簇数 10GB/4KB=2621440；位图字节 2621440/8=320KB；位图簇数 320KB/4KB=80，选 A。

【坑位】

单位换算全程用 1024，中途别混进 1000。

10GB=10×2^30 字节，4KB=4×2^10 字节；如果按 1000 进制算（10×10^9），簇数就变成 2500000，位图 312.5KB，最后 78.125 簇——四个选项里没有，全错。正确做法：看到 GB、KB 先换成 2 的幂，再除。

【套路】

- 第一步，算总簇数：分区大小 ÷ 簇大小 = 10×2^30 ÷ (4×2^10) = 2621440。
- 第二步，算位图字节数：1 位管 1 簇，总位数 ÷ 8 = 2621440 ÷ 8 = 327680 字节 = 320KB。
- 第三步，算位图占几个簇：位图字节数 ÷ 簇大小 = 320KB ÷ 4KB = 80。

簇数除以 8 得字节，字节除以簇大小得簇数。

【类比】

像给停车场每个车位配一把小钥匙：2621440 个车位，就要 2621440 把钥匙（位）；钥匙装箱，8 把一盒（除以 8），共 320KB 的箱子；箱子放进 4KB 一间的储物间，要 80 间。术语贴回去：钥匙就是"位图中的位"，车位就是"簇"，储物间就是"存放位图的簇"。

【口诀】 簇数除八得字节，字节再除簇大小；单位全程一零二四。

【考频】 「位图法」在 2009–2022 年解析中出现 4 次 · 中频

第 28 题 · 加快虚实地址转换的措施（正确答案：C）

题干与选项：

下列措施中，能加快虚实地址转换的是 。 Ⅰ．增大快表(TLB)容量 Ⅱ．让页表常驻内存 Ⅲ．增大交换区(swap)

- A. 仅 Ⅰ
- B. 仅 Ⅱ
- C. 仅 Ⅰ、Ⅱ
- D. 仅 Ⅱ、Ⅲ

入手点： 先想虚实地址转换慢在哪：查页表要访存；再看每条措施是不是对着这个慢的地方下手。

专有名词从零解释：

- 先想“虚地址、实地址”：程序用的是虚地址，主存里真正的位置是实地址；CPU 给出虚地址，要先换成实地址，才能访问主存。
- 再想“页表”：虚地址和实地址的对照表；查页表就是查“这个虚页对应哪个实页框”。
- 再想“TLB”：也叫快表，是页表的一份小而快的副本，放在 CPU 里；查 TLB 比查页表快得多。
- 再想“页表常驻内存”：页表一直放在主存里，不用每次查之前先从外存调进来。
- 再想“交换区”：外存里放被换出页面的地方；页面不够用时，把不用的页面换到交换区去。

考查知识点：

- 先想 Ⅰ 为什么对：增大 TLB 容量，TLB 里能装的对照关系多了，命中率提高了；命中了就不用去主存查页表，转换就快了。
- 再想 Ⅱ 为什么对：页表常驻内存，查页表时不用先从外存把页表调进来，少了一次外存访问，转换就快了。
- 再想Ⅲ为什么错：交换区是放换出页面的地方，交换区大一点小一点，和“查页表快不快”没关系；地址转换慢，是因为查页表要访存，不是交换区不够大。
- 再想组合： Ⅰ 对、 Ⅱ 对、Ⅲ错，就是“仅 Ⅰ 、 Ⅱ”，就是 C。
- 拿生活例子验算：查字典时，手边有一本小抄（TLB），小抄越大，查到的概率越高；字典本身放在桌上（常驻内存），不用每次去书架上拿；书架上的旧书区（交换区）大不大，和查字典快不快没关系。
- 边界情况：TLB 容量增大后，命中率提高，但 TLB 本身变大也可能让查 TLB 稍微慢一点；不过总体上还是快的，所以 Ⅰ 算对。
- 易混对子：不要把“交换区”和“页表”搞混——交换区放的是被换出的页面，页表放的是地址对照关系；增大交换区不加快地址转换。

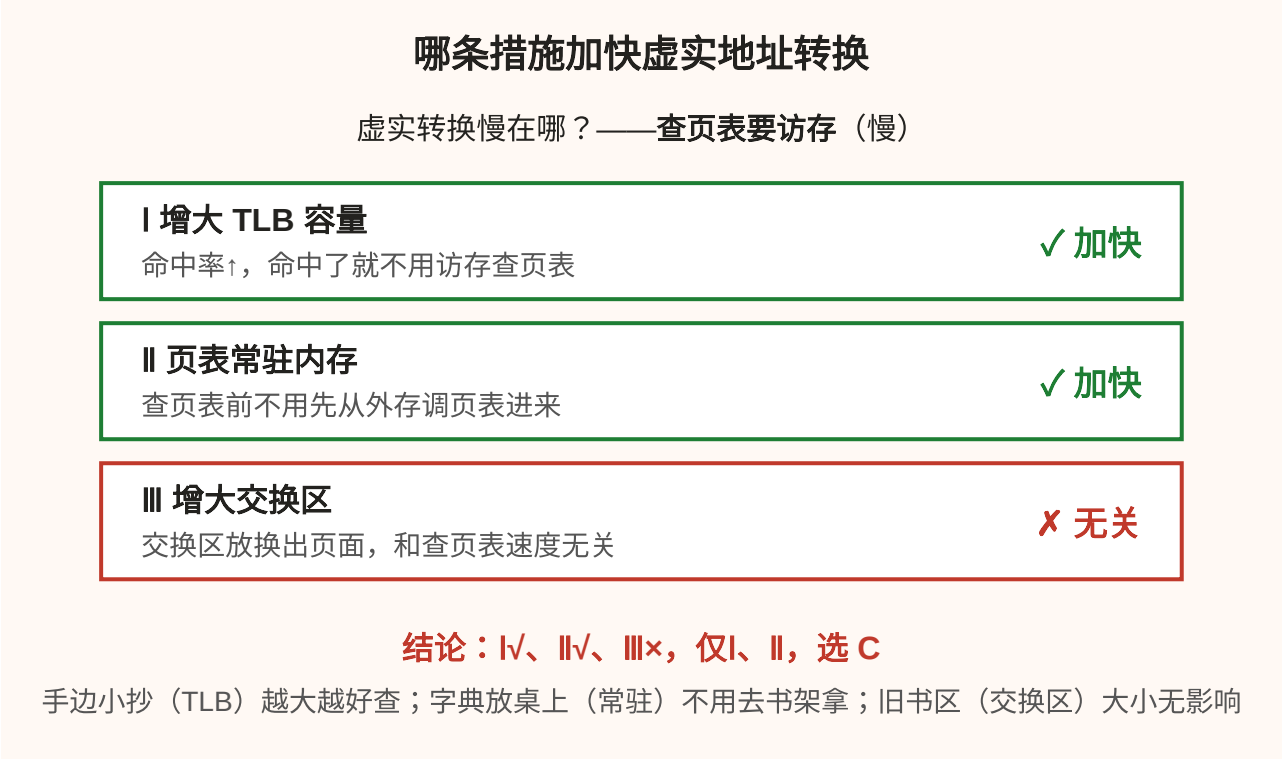
正确选项为什么对： Ⅰ 和 Ⅱ 都对着“查页表要访存”这个慢的地方下手，Ⅲ无关，所以是仅 Ⅰ 、 Ⅱ，选 C。

错误选项为什么错＋举例：

- A（仅 Ⅰ）：漏了 Ⅱ；页表常驻内存也能加快转换。教训：组合题要一条一条验，不能漏。
- B（仅 Ⅱ）：漏了 Ⅰ；增大 TLB 容量也能加快转换。教训：TLB 命中率高了，转换就快了。
- D（仅 Ⅱ 、Ⅲ）：把无关的Ⅲ算进来了，漏了 Ⅰ；教训：交换区大小和地址转换速度没关系。

笔记本一句话： 第 28 题＋加快虚实转换：增大 TLB、页表常驻内存都对；增大交换区无关；仅 Ⅰ 、 Ⅱ。

【图像】



三条措施对虚实地址转换速度的影响：增大TLB与页表常驻有效，增大交换区无关

【定位】 操作系统 · 第3章 内存管理 · 虚实地址转换与 TLB

【易混对比】

- "TLB（快表）"和"页表"的区别：TLB 是页表的小而快的副本、放在 CPU 里；页表在主存里。增大 TLB 提高命中率，命中就不用访存查页表。
- "页表常驻内存"和"增大交换区"的区别：前者让查页表少一次外存访问（页表不用先从外存调进来）；后者只是让被换出的页面有地方放，和查页表快不快无关。
- "加快地址转换"和"减少缺页"的区别：本题问的是转换速度，不是缺页次数；交换区大可能让换出更顺畅，但不加快单次地址转换。

Ⅰ（TLB 变大、命中率高）和 Ⅱ（页表常驻、少一次外存访问）都对着"查页表要访存"下手，Ⅲ无关，仅Ⅰ、Ⅱ，选 C。

【坑位】

组合题别只看对了一条就急着选"仅Ⅰ"。

Ⅰ 对了不等于只有 Ⅰ 对；Ⅱ"页表常驻内存"同样加快转换——页表不在内存时，查页表之前还要先把页表从外存调进来，多一次慢速访问。正确做法：三条逐条打勾，Ⅰ√、Ⅱ√、Ⅲ×，再拼组合。

【套路】

- 第一步，找慢的地方：虚实转换慢在"查页表要访存"。
- 第二步，逐条看是不是对慢处下手：Ⅰ（TLB 命中免访存）√；Ⅱ（页表常驻少一次外存）√；Ⅲ（交换区大小）×。
- 第三步，拼组合：仅Ⅰ、Ⅱ。

先找慢在哪，再看每条措施治没治到慢处。

【类比】

像查字典：手边的小抄（TLB）越大，查到的概率越高；字典本身放在桌上（页表常驻内存），不用每次去书架上拿；书架上的旧书区（交换区）大不大，和查字典快不快没关系。术语贴回去：查字典就是"虚实地址转换"，小抄就是"快表"。

【口诀】 快表大了命中多，页表驻留少一访；交换区大帮不上。

【考频】 「虚实地址转换」在 2009–2022 年解析中出现 21 次 · 高频

第 29 题 · 文件首次被用户程序打开时操作系统的工作（正确答案：B）

题干与选项：

在一个文件被用户进程首次打开的过程中，操作系统需做的是 。

- A. 将文件内容读到内存中
- B. 将文件控制块读到内存中
- C. 修改文件控制块中的读写权限
- D. 将文件的数据缓冲区首指针返回给用户进程

入手点： 先想“打开文件”是干什么：是“登记”不是“读内容”；登记什么？登记文件的档案。

专有名词从零解释：

- 先想“文件控制块”：也叫 FCB，是文件的档案，里面写着文件名、文件位置、大小、权限这些信息；文件存在外存上时，档案也在外存上。
- 再想“打开文件”：用户程序想用一个文件时，先请操作系统把这个文件的档案调进内存、登记一下；登记完了，后面读写文件就按这个登记找文件，不用每次都去外存翻档案。
- 再想“文件内容”：文件里真正的数据；打开文件时不用读内容，读内容是以后读操作的事。

考查知识点：

- 先想为什么是把 FCB 读到内存：打开文件就是要把文件的档案从外存调进内存，建好打开文件表项；后面读写文件，都按这个表项找文件。
- 再想为什么不是把文件内容读到内存：打开只是登记，内容还没开始读；内容是以后用户程序发读操作时才读进来的。
- 再想为什么不是修改读写权限：打开文件不改文件的权限；权限是文件本身的属性，打开时只检查权限，不修改权限。
- 再想为什么不是返回数据缓冲区首指针：打开文件返回给用户的是文件打开的标识（比如文件描述符），不是数据缓冲区的指针；数据缓冲区是读写时才用的。
- 拿生活例子验算：去图书馆借书，先登记借书卡（把档案调进内存），不是把整本书复印一遍（读内容）；登记不改书的借阅权限，登记完给的是借书凭证，不是书的内容。
- 边界情况：如果文件已经被别的程序打开过，FCB 可能已经在内存里了，这次打开就不用再从外存读了；但“首次打开”时，FCB 一定要从外存读进内存。
- 易混对子：不要把“打开文件”和“读文件”搞混——打开是登记，读是拿内容；也不要把“文件描述符”和“数据缓冲区指针”搞混——描述符是打开的标识，缓冲区指针是读写时数据放的地方。

正确选项为什么对： 首次打开要把 FCB 从外存读到内存、登记好，选 B。

错误选项为什么错＋举例：

- A（将文件内容读到内存）：那是读操作的事，不是打开的事；教训：打开是登记，不是读内容。
- C（修改文件控制块中的读写权限）：打开只检查权限，不修改权限；教训：打开不改档案。
- D（将文件数据缓冲区首指针返回给用户进程）：返回的是文件打开的标识，不是数据缓冲区的指针；教训：标识和缓冲区指针是两回事。

笔记本一句话： 第 29 题＋首次打开文件：把 FCB 读到内存、登记好；不是读内容、不是改权限、不是返回缓冲区指针。

【图像】



打开文件=把FCB调入内存登记，与读文件=把内容读入内存的对比

【定位】 操作系统 · 第4章 文件管理 · 文件的打开操作

【易混对比】

- "打开文件"和"读文件"的区别：打开是"登记"（把 FCB 从外存调进内存、建打开文件表项），读是"拿内容"（把文件数据读进内存）；打开时不读内容。
- "检查权限"和"修改权限"的区别：打开时只检查有没有权限，不改 FCB 里的权限位；权限是文件的属性，打开动作不改它。
- "文件描述符"和"数据缓冲区指针"的区别：打开返回给用户的是打开的标识（文件描述符），用来指代这次打开；数据缓冲区的指针是读写数据时才用的。

首次打开就是把文件的档案（FCB）从外存调进内存、登记好，后面读写按登记找文件，选 B。

【坑位】

别被"首次"二字带偏，以为首次打开要多做一步"读内容"。

"首次"真正的意思是：FCB 还没在内存里，所以必须从外存读进来；如果文件之前被别的进程打开过，FCB 已在内存，这次打开连读 FCB 都省了。正确做法：看到"打开"先想"登记"，看到"首次"想"FCB 不在内存、要从外存读"。

【类比】

像去图书馆借书：先登记借书卡（把档案调进内存），不是把整本书复印一遍（读内容）；登记不改书的借阅规则（不改权限），登记完给的是借书凭证（文件描述符），不是书的内容。术语贴回去：借书卡就是"文件控制块 FCB"，借书凭证就是"文件描述符"。

【口诀】 打开只把档案调，内容等到读时要；权限只查不修改。

【考频】 「打开文件」在 2009–2022 年解析中出现 10 次 · 高频

第 30 题 · 会出现 Belady 异常的页面置换算法（正确答案：A）

题干与选项：

在页式虚拟存储管理系统中，采用某些页面置换算法，会出现 Belady 异常现象， 即进程的缺页次数会随着分配给该进程的页框个数的增加而增加。下列算法中，可能出现 Belady 异常现象的是 。 Ⅰ．LRU 算法 Ⅱ．FIFO 算法 Ⅲ．OPT 算法

- A. 仅 Ⅱ
- B. 仅 Ⅰ、Ⅱ
- C. 仅 Ⅰ、Ⅲ
- D. 仅 Ⅱ、Ⅲ

入手点： 先想 Belady 异常是什么意思：分配的页框多了，缺页反而多了；再想哪种算法有这个毛病。

专有名词从零解释：

- 先想“页面置换算法”：主存里的页框满了、新页要进来时，挑一个旧页换出去的规矩。
- 再想“Belady 异常”：分配的页框数增加了，缺页次数反而增加了；正常直觉是页框越多、缺页越少，Belady 异常反过来了。
- 再想“FIFO”：先进先出，最先进来的页最先换出去。
- 再想“LRU”：最近最久未使用，最久没用过的页最先换出去。
- 再想“OPT”：最佳置换，换出去以后最久不用的页（要预知未来，实际做不到，只用来比较）。

考查知识点：

- 先想 FIFO 为什么会出现：FIFO 只看进来的先后，不看用没用过；页框多了，进来的顺序和换出去的顺序可能错开，导致多换几次，缺页反而多了；经典例子里，3 个页框缺页 9 次，4 个页框缺页 10 次。
- 再想 LRU 为什么不会出现：LRU 有“栈性质”——页框多了，缺页次数只会少、不会多；因为最近用过的页，在页框多的情况下一定还在，不会被多换出去。
- 再想 OPT 为什么不会出现：OPT 是最佳的，页框多了，缺页次数只会少、不会多；它本身就是按“以后最久不用”挑的，不会出现越换越多的情况。
- 再想组合：只有 FIFO 会出现，就是“仅 Ⅱ”，就是 A。
- 拿生活例子验算：FIFO 像排队买票，先来的人先走；队伍变长了，反而有人多排了一次队；LRU 和 OPT 像“最近用过的先留着”，队伍变长了，留着的人只会更多，不会更少。
- 边界情况：Belady 异常只在某些访问序列下出现，不是每次都出现；题目问“会出现”，只要有一种序列能出现，就算会。
- 易混对子：不要把“FIFO 简单”和“FIFO 不会异常”划等号——FIFO 简单，但正是它会出现 Belady 异常；也不要以为 LRU 也会——LRU 有栈性质，不会出现。

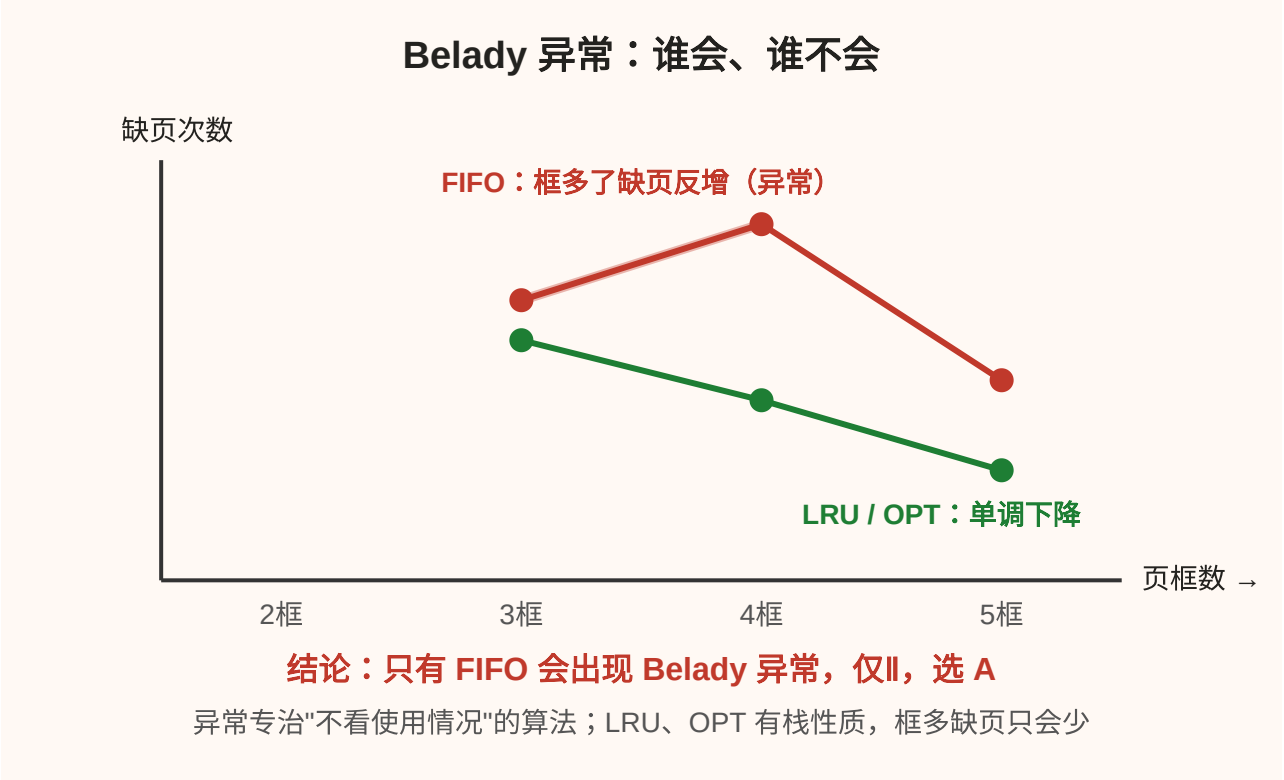
正确选项为什么对： 只有 FIFO 会出现 Belady 异常，LRU 和 OPT 不会，所以是仅 Ⅱ，选 A。

错误选项为什么错＋举例：

- B（仅 Ⅰ、Ⅱ）：把 LRU 也算进来了；LRU 有栈性质，不会出现。教训：LRU 页框越多缺页越少，不会反过来。
- C（仅 Ⅰ、Ⅲ）：把 LRU 和 OPT 都算进来了；两个都不会出现。教训：最佳和最近最久未用都有栈性质。
- D（仅 Ⅱ、Ⅲ）：把 OPT 也算进来了；OPT 是最佳的，不会出现。教训：最佳置换不可能越换越多。

笔记本一句话： 第 30 题＋Belady 异常：只有 FIFO 会出现；LRU、OPT 有栈性质，不会出现。

【图像】



Belady异常示意：FIFO页框数增加时缺页数可能上升，LRU与OPT单调下降

【定位】 操作系统 · 第3章 内存管理 · 页面置换算法与 Belady 异常

【易混对比】

- "FIFO"和"LRU"的区别：FIFO 只看"先进来先出去"，不管用没用过；LRU 看"最近最久没用过"，有栈性质——页框多了缺页只会少、不会多。
- "会出现"和"每次都会"的区别：题目问"可能出现"，只要存在某个访问序列让缺页数随页框数增加而增加，就算会；不是要求每次都这样。
- "OPT"和"FIFO"的区别：OPT 换出"以后最久不用"的页，是理论最优，不可能越换越多；FIFO 不看未来使用情况，才会出异常。

只有 FIFO 会出现 Belady 异常；LRU 和 OPT 都有栈性质，页框越多缺页越少，仅 II，选 A。

【坑位】

别因为 LRU 比 FIFO"高级"就默认它也有异常。

恰恰相反：Belady 异常专治"不看使用情况"的算法。FIFO 只看进来顺序，页框多了、进出顺序错开，可能多换几次；LRU 有栈性质，页框数增加时缺页次数不会增加。正确做法：记住"FIFO 会，LRU/OPT 不会"，别记反了。

【类比】

像排队买票只看先来后到（FIFO）：队伍变长了，反而有人多排了一次队——这就是异常；如果按"最近买过的人先留着"（LRU），队伍变长，留着的人只会更多、不会更少。术语贴回去：队伍长度就是"分配的页框数"，多排一次队就是"缺页次数增加"。

【口诀】 FIFO 不看用不用，框多反而缺页增；LRU 最优有栈性，框多缺页只会降。

【考频】 「Belady 异常」在 2009–2022 年解析中出现 11 次 · 高频

第 31 题 · 管道通信相关叙述的正误，问正确的一项（正确答案：C）

题干与选项：

下列关于管道(Pipe)通信的叙述中，正确 的是 。

- A. 一个管道可实现双向数据传输
- B. 管道的容量仅受磁盘容量大小限制
- C. 进程对管道进行读操作和写操作都可能被阻塞
- D. 一个管道只能有一个读进程或一个写进程对其操作

入手点： 先把管道的两个基本性质记牢：单向的、字节流；再看每条说法和这两个性质合不合。

专有名词从零解释：

- 先想“管道”：也叫 Pipe，进程之间通信的一种办法；像一根单向的水管，数据从一头流进去、从另一头流出来。
- 再想“单向”：管道只能一个方向传数据；要双向传，就要建两根管道。
- 再想“阻塞”：进程读写管道时，如果条件不满足（比如读的时候管道是空的、写的时候管道是满的），进程就停下来等，等条件满足了再继续。
- 再想“管道容量”：管道在内存里占一块缓冲区，大小是固定的（比如 64KB），不是无限大的。

考查知识点：

- 先想 A 为什么错：一个管道只能单向传数据，不能双向传；要双向，就要建两根管道，一根各管一个方向。
- 再想 B 为什么错：管道容量受内存缓冲区大小限制，是定长的；不是受磁盘容量限制；磁盘多大和管道容量没关系。
- 再想 C 为什么对：读管道时，如果管道是空的，读进程就阻塞起来等数据来；写管道时，如果管道是满的，写进程就阻塞起来等地方空出来；所以读、写操作都可能被阻塞。
- 再想 D 为什么错：一个管道可以有多个读进程，也可以有多个写进程（比如父进程和子进程都往同一根管道里写）；不是只能有一个读进程或一个写进程。
- 拿生活例子验算：管道像一根单向的水管，水只能从一头流到另一头；水管空了，接水的人要等水来；水管满了，倒水的人要等水流走；一根水管可以好几个人接水、好几个人倒水。
- 边界情况：管道分无名管道和有名管道；无名管道只能在有亲缘关系的进程之间用，有名管道可以在没有亲缘关系的进程之间用；但单向、阻塞这些性质都是一样的。
- 易混对子：不要把“管道”和“双向通信”划等号——管道是单向的；也不要把“管道容量”和“磁盘容量”划等号——管道在内存里，容量是内存缓冲区的大小。

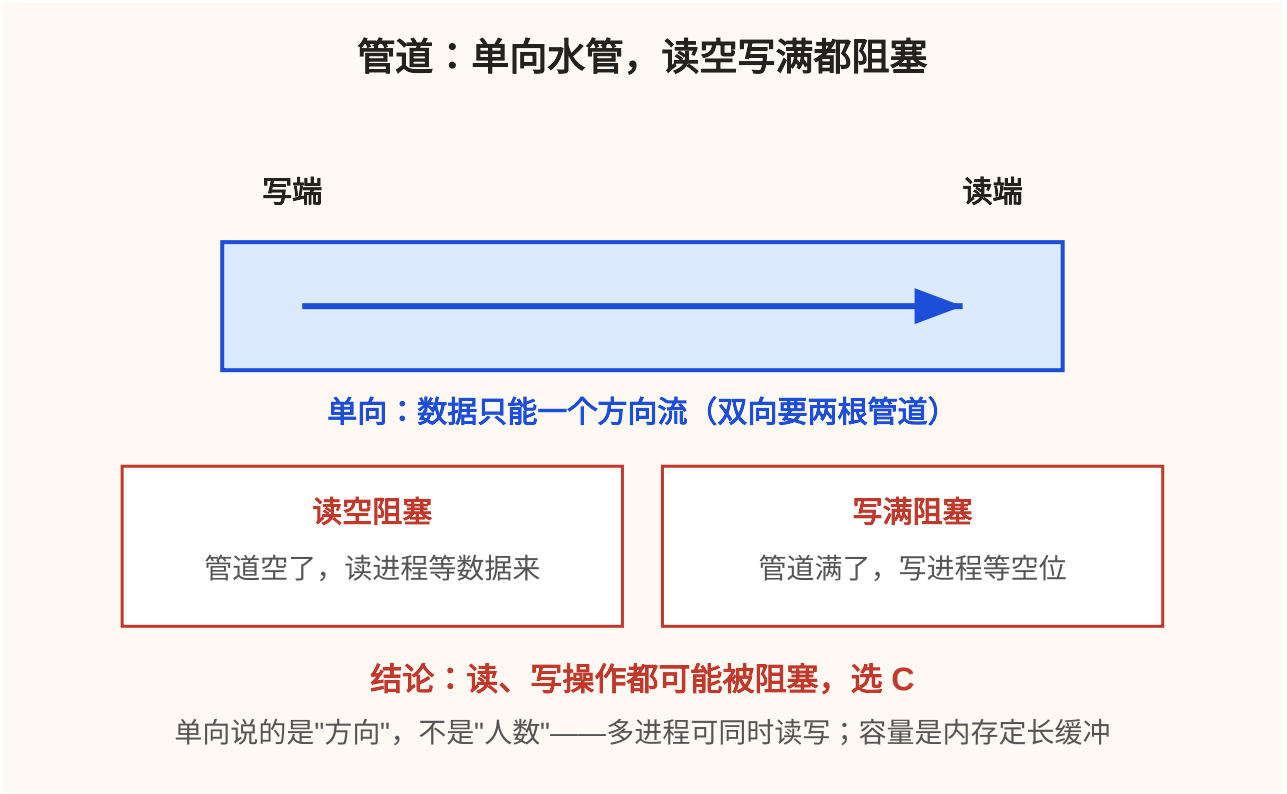
正确选项为什么对： 读空阻塞、写满阻塞，读、写操作都可能被阻塞，选 C。

错误选项为什么错＋举例：

- A（一个管道可实现双向数据传输）：管道是单向的，双向要两根管道；教训：单向是管道的基本性质。
- B（管道容量仅受磁盘容量大小限制）：管道在内存里，容量是内存缓冲区定长；教训：管道不在磁盘上。
- D（一个管道只能有一个读进程或一个写进程对其操作）：多个进程可以同时读写同一根管道；教训：读进程和写进程的个数没有只能一个的限制。

笔记本一句话： 第 31 题＋管道：单向字节流，内存缓冲区定长；读空阻塞、写满阻塞；可多进程读写。

【图像】



单向管道示意：读空阻塞与写满阻塞，多个进程可同时读写

【定位】 操作系统 · 第2章 进程管理 · 进程通信（管道）

【易混对比】

- "单向"和"双向"的区别：一条管道只能一个方向传数据；要双向传就建两根管道，一根管一个方向。
- "管道容量"和"磁盘容量"的区别：管道在内存里，容量是内存缓冲区的定长；磁盘多大和它没关系。
- "读空阻塞"和"写满阻塞"的区别：读的时候没数据→读进程等；写的时候没地方→写进程等；两种方向都会阻塞。

管道单向、内存定长缓冲；读空了读进程等数据来，写满了写进程等地方空出来，读写都可能阻塞，选 C。

【坑位】

别把"只能有一个读进程"和"读端只能读"混为一谈。

管道是单向的（一端只读、一端只写），但读端和写端都可以被多个进程同时用——父子进程可以同时往同一根管道里写，也可以同时从同一根管道里读。正确做法：单向说的是"方向"，不是"人数"。

【类比】

像一根单向的水管：水只能从一头流到另一头；水管空了，接水的人要等水来（读空阻塞）；水管满了，倒水的人要等水流走（写满阻塞）；一根水管可以好几个人接水、好几个人倒水。术语贴回去：水管就是"管道"，接水等水来就是"读阻塞"，倒水等水流走就是"写阻塞"。

【口诀】 管道单向像水管，读空写满会等干；人数不限方向限。

【考频】 「管道通信」在 2009–2022 年解析中出现 1 次 · 低频

第 32 题 · 多级页表的优点（正确答案：D）

题干与选项：

下列选项中，属于多级页表优点的是 。

- A. 加快地址变换速度
- B. 减少缺页中断次数
- C. 减少页表项所占字节数
- D. 减少页表所占的连续内存空间

入手点： 先想单级页表有什么毛病：页表本身太大，要占一大块连续内存；再看多级页表治的是不是这个毛病。

专有名词从零解释：

- 先想“页表”：虚地址和实地址的对照表；虚页号对应实页框号，查页表就是查这个对照关系。
- 再想“单级页表”：整张对照表放在一起，一级查完；地址空间越大，页表越大。
- 再想“多级页表”：页表再分页，只给用到的虚地址空间建页表，不用的部分不建；各级页表可以分散存放，不用连在一起。
- 再想“连续内存空间”：一段地址挨着的内存；单级页表要一大块挨着的内存，多级页表不用。

考查知识点：

- 先想单级页表的毛病：地址空间大时，页表本身就很大；单级页表要一整块连续内存放下，找这么大一块连续内存不容易。
- 再想多级页表的好处：只给用到的部分建页表，不用的部分不建；各级页表可以分散放在不连续的地方；所以不用一整块连续内存了，就是 D。
- 再想为什么不是加快地址变换：多级页表要查好几级（先查一级页表、再查二级页表），比单级查一次更慢；所以不是加快，是变慢。
- 再想为什么不是减少缺页中断：缺页中断的次数由程序局部性和页框数决定，和页表是几级没关系。
- 再想为什么不是减少页表字节数：多级页表多了各级页表本身的开销，总字节数不一定少；少的是“连续”的要求，不是字节数。
- 拿生活例子验算：单级页表像一本很厚的电话簿，必须放在一个大抽屉里；多级页表像把电话簿按姓氏拆成好几本，哪本用到了再拿哪本，几本可以放在不同的抽屉里，不用一个大抽屉。
- 边界情况：如果程序把整个地址空间都用满了，多级页表也要建满各级，字节数反而比单级多；多级页表的好处只在“地址空间稀疏”（只用一小部分）时明显。
- 易混对子：不要把“减少连续内存空间”和“减少字节数”搞混——多级页表省的是“连续”，不是“字节”；也不要以为“多级”就“更快”——多级要多查几级，更慢。

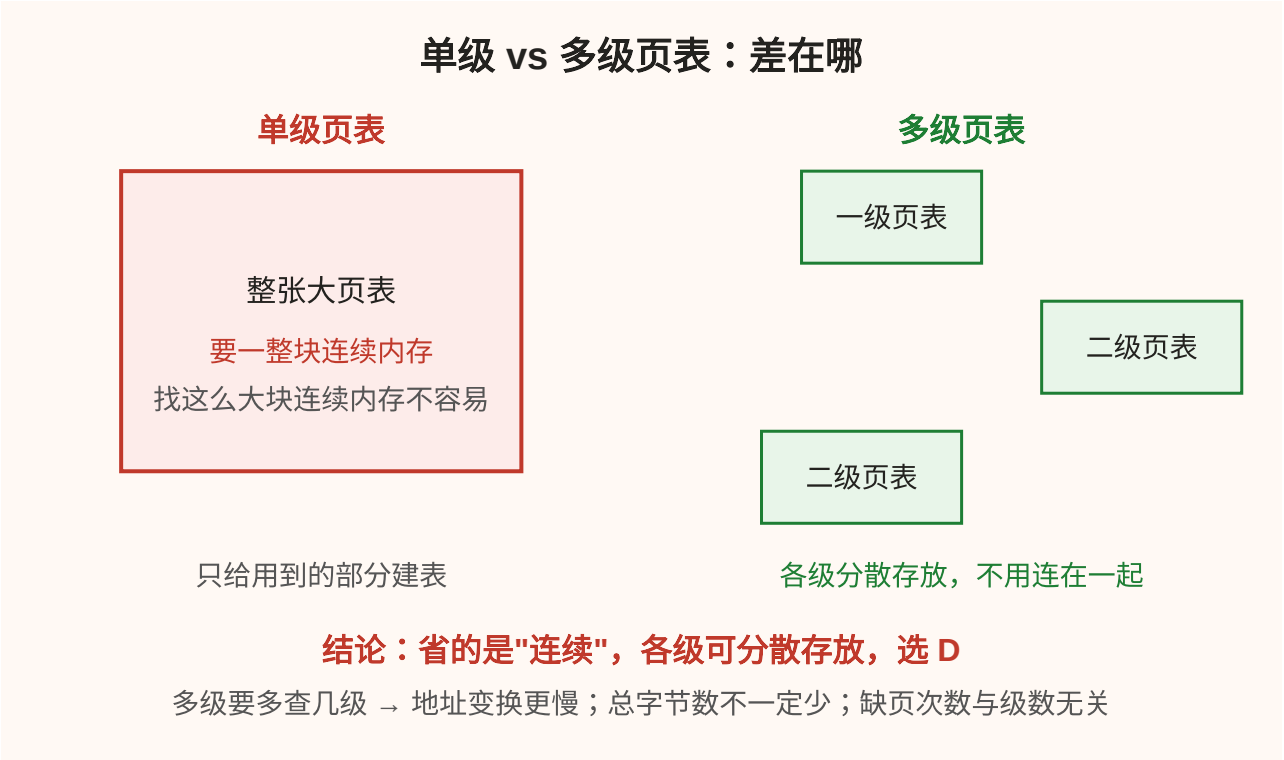
正确选项为什么对： 多级页表不用一整块连续内存，各级可以分散存放，选 D。

错误选项为什么错＋举例：

- A（加快地址变换速度）：多级要查好几级，比单级慢；教训：级数多了，查的次数也多了。
- B（减少缺页中断次数）：缺页次数看局部性和页框数，和几级页表没关系；教训：缺页和页表级数是两回事。
- C（减少页表所占字节数）：总字节数不一定少，多了各级页表本身的开销；教训：省的是连续，不是字节。

笔记本一句话： 第 32 题＋多级页表优点：不用一整块连续内存，各级可分散存放；不加快变换、不减缺页、不一定减字节。

【图像】



单级页表需整块连续内存 vs 多级页表各级可分散存放的对比

【定位】 操作系统 · 第3章 内存管理 · 多级页表

【易混对比】

- "减少连续内存空间"和"减少页表字节数"的区别：多级页表省的是"连续"——各级页表可以分散放，不用一整块挨着的内存；总字节数因为多了各级页表本身的开销，不一定少。
- "多级"和"更快"的区别：多级要查好几级（先一级、再二级），比单级查一次更慢；"多级"治的是"放不下"，不是"查得慢"。
- "缺页中断次数"和"页表级数"的区别：缺页次数看程序局部性和页框数，和页表是几级没关系。

单级页表的毛病是"整张表太大、要一整块连续内存"；多级页表只给用到的部分建表、各级分散存放，治的就是这个毛病，选 D。

【坑位】

别看到"多级"就默认"各方面都更好"。

多级页表只解决"连续内存难找"这一个问题，代价是地址变换变慢（多查几级）、总字节数可能变多；如果程序把整个地址空间都用满了，多级页表的好处就没了。正确做法：先想单级页表的毛病是什么，再看多级治的是不是这个毛病。

【类比】

像电话簿：单级页表是一本很厚的电话簿，必须放在一个大抽屉里；多级页表是按姓氏拆成好几本，哪本用到了再拿哪本，几本可以放在不同的小抽屉里，不用一个大抽屉。术语贴回去：大抽屉就是"连续内存空间"，拆成几本就是"多级"。

【口诀】 多级省的是连续位，变换变慢字节未必减。

【考频】 「多级页表」在 2009–2022 年解析中出现 21 次 · 高频

第 33 题 · OSI 参考模型中直接为会话层提供服务的层次（正确答案：C）

题干与选项：

在 OSI 参考模型中，直接为会话层提供服务的是 。

- A. 应用层
- B. 表示层
- C. 传输层
- D. 网络层

入手点： 先把 OSI 七层的顺序排好，再记住一条规矩：下层为上层提供服务；会话层的下层是谁，谁就是答案。

专有名词从零解释：

- 先想“OSI 参考模型”：把网络通信分成七层的标准；从上到下依次是应用层、表示层、会话层、传输层、网络层、数据链路层、物理层。
- 再想“服务”：下层为上层干活；上层要用下层的功能，就通过服务接口调用下层；比如应用层要用表示层的数据格式功能，表示层就为应用层提供服务。
- 再想“会话层”：管会话的建立、维持、断开；比如两个应用之间的一次对话，从开始到结束，由会话层管。
- 再想“传输层”：管端到端的可靠传送；比如数据从一台机器到另一台机器，丢了重传、乱了排队，由传输层管。
- 再想“表示层”：管数据格式、加密、压缩；比如数据怎么表示、怎么加密，由表示层管。

考查知识点：

- 先想七层的顺序：从上到下是应用层（第 7 层）、表示层（第 6 层）、会话层（第 5 层）、传输层（第 4 层）、网络层（第 3 层）、数据链路层（第 2 层）、物理层（第 1 层）。
- 再想服务方向：下层为上层提供服务；第 4 层为第 5 层提供服务，第 5 层为第 6 层提供服务，以此类推。
- 再想会话层的下层是谁：会话层是第 5 层，它的下层是第 4 层传输层；所以直接为会话层提供服务的，就是传输层，就是 C。
- 再想为什么不是表示层：表示层是第 6 层，在会话层上面；是会话层为表示层提供服务，不是表示层为会话层提供服务；方向反了。
- 拿生活例子验算：七层楼，第 5 层是会话层，第 4 层是传输层；楼下的第 4 层为楼上的第 5 层服务（比如送水、送电）；第 6 层表示层在第 5 层楼上，是第 5 层为它服务，不是它为第 5 层服务。
- 边界情况：每一层都只直接为它上面紧挨着的那一层提供服务；传输层直接为会话层服务，也间接为表示层、应用层服务；但题目问“直接”，所以是传输层，不是更下面的网络层。
- 易混对子：不要把“会话层为表示层服务”和“表示层为会话层服务”搞反——服务方向是下层到上层，会话层在表示层下面，所以是会话层为表示层服务；题目问的是反方向，是传输层为会话层服务。

正确选项为什么对： 会话层的下层是传输层，下层为上层提供服务，所以传输层直接为会话层提供服务，选 C。

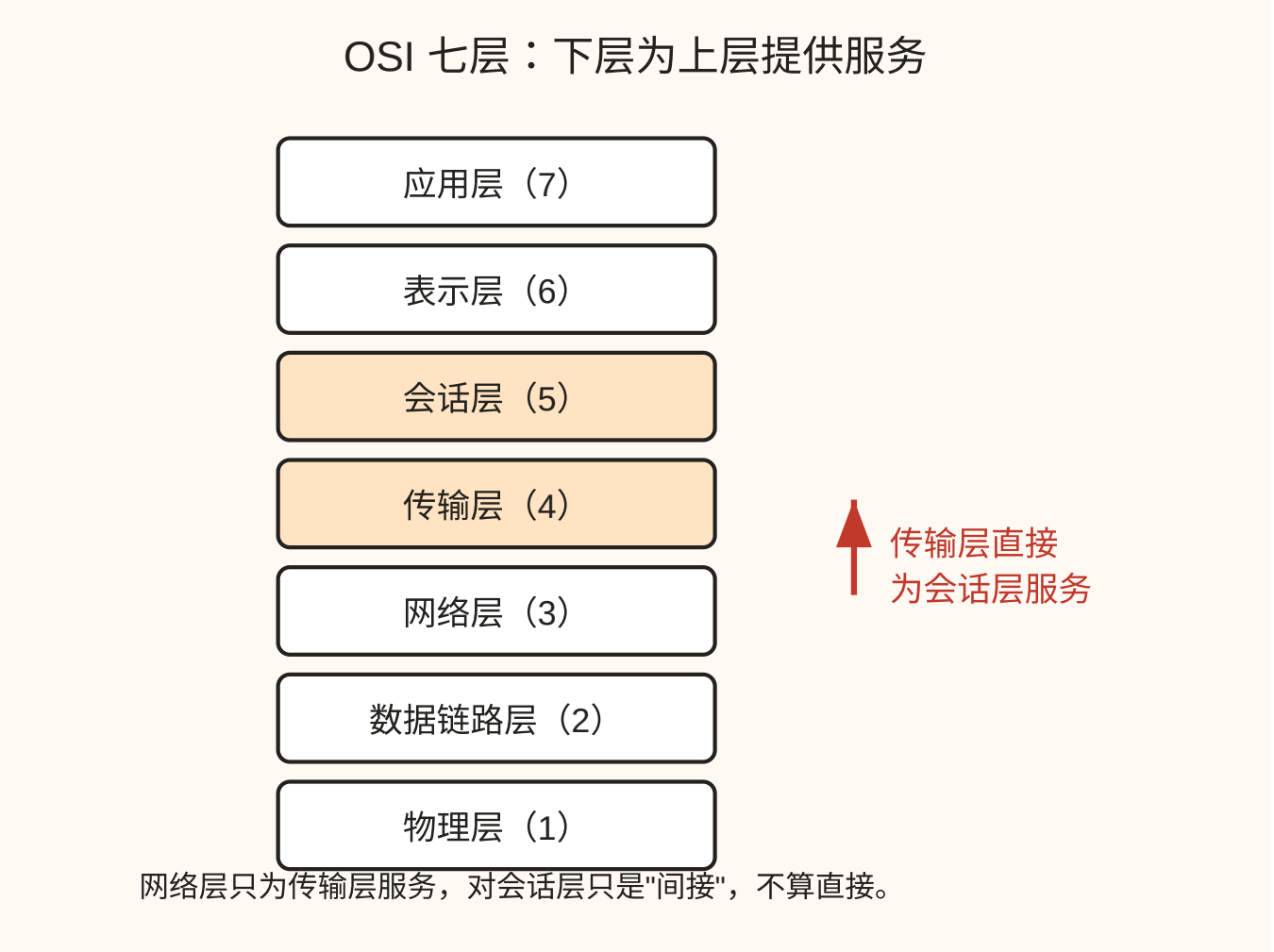
错误选项为什么错＋举例：

- A（应用层）：应用层是第 7 层，在最上面；它用下面各层的服务，不是为会话层提供服务。教训：应用层是服务的使用者，不是会话层的服务者。
- B（表示层）：表示层是第 6 层，在会话层上面；是会话层为表示层提供服务，方向反了。教训：服务方向是下层到上层，不能反过来。

- D（网络层）：网络层是第 3 层，它直接为传输层（第 4 层）提供服务，不是直接为会话层（第 5 层）提供服务；中间隔着传输层。教训：题目问“直接”，隔了一层就不算直接。

笔记本一句话： 第 33 题＋OSI 服务方向：下层为上层服务；会话层（第 5 层）的下层是传输层（第 4 层），所以传输层直接为会话层服务。

【图像】



OSI 七层模型中传输层向上为会话层直接提供服务的箭头示意图

【定位】 计算机网络 · 第1章 计算机网络体系结构 · OSI 参考模型服务方向

【易混对比】

- "直接为会话层服务"和"间接为会话层服务"的区别：直接只认紧邻的下层，会话层（第 5 层）的紧邻下层只有传输层（第 4 层）；网络层（第 3 层）只能算间接服务，题目问"直接"时不算。
- "为会话层服务"和"会话层为它服务"的区别：前者是传输层（在会话层下面），后者是表示层（在会话层上面）；服务方向永远下层→上层，把谁在谁上面排反了就选反。

会话层是第 5 层，紧邻下层是传输层（第 4 层）；下层为上层服务，所以传输层直接为会话层服务。

【坑位】

别把"表示层为会话层服务"当成对的。

表示层是第 6 层，在会话层上面。题目问"为会话层提供服务"，找的是下层；表示层是上层，是会话层为表示层服务，方向反了。正确做法：先按 7→1 从上往下把层排好，会话层是第 5 层，直接看它下面一格是谁，就是传输

层。

【类比】

像七层楼：住在 5 楼（会话层）的住户，水电维修都由 4 楼（传输层）送上来——楼下给楼上服务。6 楼（表示层）住在上面，是 5 楼给它服务，不是它给 5 楼服务。术语贴回去：楼下的层就是"直接为上层提供服务的层"，中间隔着一层就不算直接。

【口诀】 下层为上层服，紧邻才算直接；会话层是五，传输层是四。

【考频】 「OSI 服务方向」在 2009–2022 年解析中出现 9 次 · 中频

第 34 题 · 交换机对数据帧与确认帧的转发端口判定（正确答案：B）

题干与选项：

某以太网拓扑及交换机当前转发表如下图所示，主机 00-e1-d5-00-23-a1 向主机 00-e1-d5-00-23-c1 发送 1 个数据帧，主机 00-e1-d5-00-23-c1 收到该帧后，向主机 00-e1-d5-00-23-a1 发送 1 个确认帧，交换机对这两个帧的转发端口分别是（）。

> 注：原卷拓扑图略。转发表：端口 1 接主机 00-e1-d5-00-23-a1，端口 2 接主机 00-e1-d5-00-23-b1，端口 3 接主机 00-e1-d5-00-23-c1；当前转发表仅 1 行：目的地址 00-e1-d5-00-23-b1 → 端口 2。插图以原卷为准

- A. {3}和{1}
- B. {2,3}和{1}
- C. {2,3}和{1,2}
- D. {1,2,3}和{1}

入手点： 交换机收到帧先做两件事：先学源地址（把源地址和接收端口记下来），再查目的地址（查到了按表转发，查不到就泛洪）。

专有名词从零解释：

- 先想“交换机”：按 MAC 地址转发帧的设备；每个端口连一台主机或者一个网段。
- 再想“MAC 地址”：写在网卡里的物理地址；本题三台主机的地址尾巴分别是 a1、b1、c1，接在端口 1、2、3 上。
- 再想“转发表”：交换机里的一张表，写着“目的地址对应哪个端口”；收到帧时查这张表，决定从哪个端口转发。
- 再想“学习”：交换机收到帧时，把帧的源地址和接收端口记到转发表里；下次再有发给这个源地址的帧，就知道从哪个端口转发了。
- 再想“泛洪”：目的地址在转发表里查不到时，交换机就把帧从除接收端口外的所有端口都发出去，总有一个端口能到目的地。

考查知识点：

- 先想初始转发表：只有一行，目的地址 b1 对应端口 2；a1 和 c1 都不在表里。
- 再想数据帧 a1→c1：从端口 1 进来；先学源地址，记下 a1 对应端口 1；再查目的地址 c1，表里没有；查不到就泛洪，从除接收端口 1 外的端口 2、3 发出去。
- 再想确认帧 c1→a1：从端口 3 进来；先学源地址，记下 c1 对应端口 3；再查目的地址 a1，表里已经有了（数据帧时刚学的），对应端口 1；所以只从端口 1 发出去。
- 再想组合：数据帧从端口 2 和 3 转发，确认帧只从端口 1 转发，就是 B。
- 拿生活例子验算：交换机像电话总机；a1 打电话找 c1，总机不知道 c1 在哪个房间，就把电话接到除了 a1 房间外的所有房间（泛洪），同时记下 a1 在 1 号房；c1 回电话找 a1，总机已经知道 a1 在 1 号房，就只接到 1 号房。
- 边界情况：泛洪时不包括接收端口——帧就是从接收端口进来的，再从接收端口发回去没有意义；所以数据帧泛洪是端口 2、3，不是端口 1、2、3。
- 易混对子：不要以为“确认帧也要泛洪”——确认帧来的时候，a1 已经在表里了（数据帧时学的），所以不用泛洪；也不要以为“数据帧只从端口 3 发”——c1 不在表里，交换机不知道 c1 在端口 3，只能泛洪。

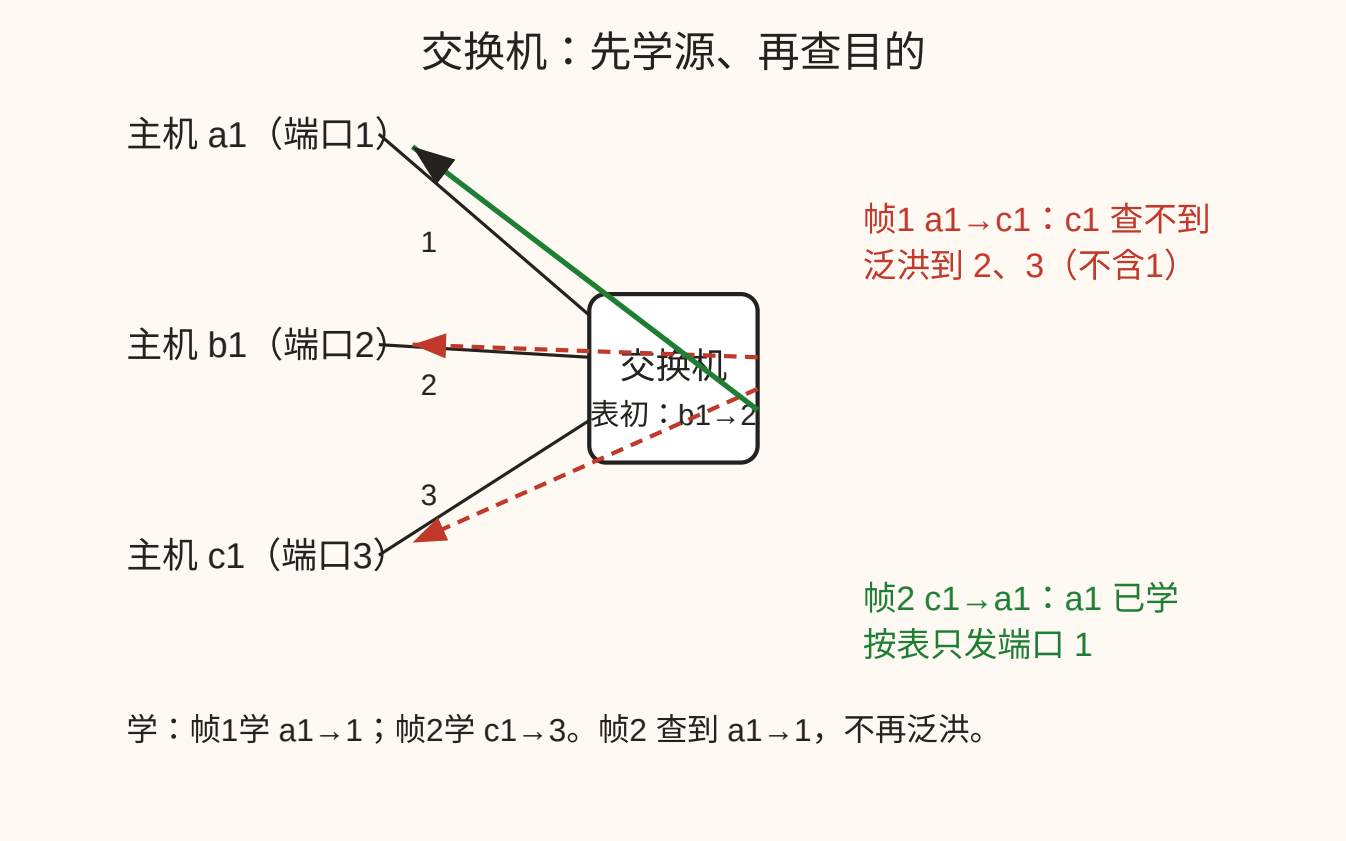
正确选项为什么对： 数据帧目的 c1 不在表里，泛洪到端口 2、3；确认帧目的 a1 已在表里，只发端口 1，选 B。

错误选项为什么错＋举例：

- A（数据帧仅从端口 3 转发，确认帧仅从端口 1 转发）：确认帧对了，但数据帧错了；c1 不在表里，交换机不知道 c1 在端口 3，不能只发端口 3，只能泛洪。教训：查不到目的地址时要泛洪，不能猜一个端口。
- C（数据帧从端口 2 和 3 转发，确认帧从端口 1 和 2 转发）：数据帧对了，但确认帧错了；a1 已经在表里对应端口 1，只用发端口 1，不用发端口 2。教训：查到了目的地址，就按表转发，不用泛洪。
- D（数据帧从端口 1、2、3 转发，确认帧仅从端口 1 转发）：确认帧对了，但数据帧错了；泛洪不包括接收端口 1，帧就是从端口 1 进来的，再从端口 1 发回去没有意义。教训：泛洪是除接收端口外的所有端口。

笔记本一句话： 第 34 题＋交换机转发：先学源地址、再查目的地址；查不到泛洪（不含接收端口），查到了按表转发。

【图像】



交换机收到 a1 发往 c1 的帧先学后泛洪到端口 2、3，收到确认帧后按表只发端口 1 的示意图

【定位】 计算机网络 · 第3章 数据链路层 · 交换机的学习与转发

【易混对比】

- "学源地址"和"查目的地址"的区别：学是把"源地址↔接收端口"记进表（为以后铺路），查是拿"目的地址"找转发表（决定这次从哪发）；学错对象或顺序颠倒，转发就全错。
- "泛洪"和"按表转发"的区别：泛洪是目的地址查不到时的办法，从除接收端口外的所有端口都发；按表转发是查到了，就只发一个端口；查到了还泛洪，或泛洪时含接收端口，都错。

先学源、再查目的：查到了按表只发一口，查不到从除接收端口外所有口泛洪；第一帧学完 a1，第二帧的确认就不用再泛洪了。

【坑位】

别以为"数据帧只发往端口 3"——c1 还没进过表，交换机不知道它在 3 号口。

交换机转发只认表，不认"目的地就在某个口上"这种上帝视角。数据帧到来时表里只有 b1→2，c1 查不到，只能泛洪。正确做法：每帧都按"先学后查"走两步，查不到就老实泛洪（去掉接收端口），不猜端口。

【套路】

- 第一步，学源地址：把"源地址↔接收端口"写入转发表（数据帧学 a1↔1；确认帧学 c1↔3）。
- 第二步，查目的地址：查到→只从表中端口发；查不到→从除接收端口外的所有端口泛洪。
- 第三步，注意顺序带来的变化：第二帧到来时，第一帧学的 a1 已在表里，确认帧按表转发，不再泛洪。

每帧两步走：先学源、再查目的；查到单发，查不到泛洪（不含来口）。

【类比】

像电话总机：a1 打来找 c1，总机不认识 c1，就把电话转到除 a1 房间外的所有房间（泛洪），同时记下"a1 住 1 号房"；c1 回电找 a1，总机翻记录知道 a1 在 1 号房，就只转 1 号房。术语贴回去：记房间就是"学习"，翻记录就是"查表"，群呼就是"泛洪"。

【口诀】 先学源来再查目，查到单发、查不到泛洪不含来口。

【考频】 「交换机转发」在 2009–2022 年解析中出现 14 次 · 高频

第 35 题 · 不影响信道数据传输速率的因素（正确答案：D）

题干与选项：

下列因素中，不会影响信道数据传输速率的是 。

- A. 信噪比
- B. 频率宽带
- C. 调制速率
- D. 信号传播速度

入手点： 先想数据传输速率由什么决定：带宽、信噪比、调制方式；再看哪个因素只影响延迟、不影响速率。

专有名词从零解释：

- 先想“信道数据传输速率”：每秒能传多少比特，单位是比特每秒；速率越高，传同样的数据花的时间越短。
- 再想“信噪比”：信号强度和噪声强度的比值；信噪比越高，信号越清楚，能传的速率越高。
- 再想“频率带宽”：信道能用的频率范围有多宽；带宽越宽，能传的速率越高。
- 再想“调制速率”：也叫波特率，每秒信号变化几次；调制速率越高，携带的信息越多，速率越高。
- 再想“信号传播速度”：信号在介质里跑多快；比如光纤里光速、电线里接近光速；传播速度决定信号多久到达，不决定每秒传多少比特。

考查知识点：

- 先想信噪比为什么影响速率：香农公式里，速率上限和带宽、信噪比都有关系；信噪比越高，速率上限越高。
- 再想频率带宽为什么影响速率：奈奎斯特公式里，速率上限和带宽成正比；带宽越宽，速率越高。
- 再想调制速率为什么影响速率：调制速率是每秒信号变化的次数，变化越快，携带的信息越多；而且每个信号状态能带好几位比特，调制速率和每状态比特数一起决定速率。
- 再想信号传播速度为什么不影响速率：传播速度只决定信号从一端到另一端要花多久（延迟），不决定每秒能发出多少比特；就像水管里的水流速度决定水多久流到，不决定每秒流出多少水（那是水管粗细决定的）。
- 拿生活例子验算：高速公路上，车道越多（带宽）、车况越好（信噪比）、车速变化越灵活（调制），每小时通过的车越多；但车从起点到终点要开多久（传播速度），不影响每小时通过多少车。
- 边界情况：传播速度影响的是“第一个比特多久到达”，也就是延迟；传输速率影响的是“传完所有比特要多久”；两者是不同的指标。
- 易混对子：不要把“延迟”和“速率”搞混——延迟是等多久，速率是每秒多少；传播速度管延迟，不管速率。

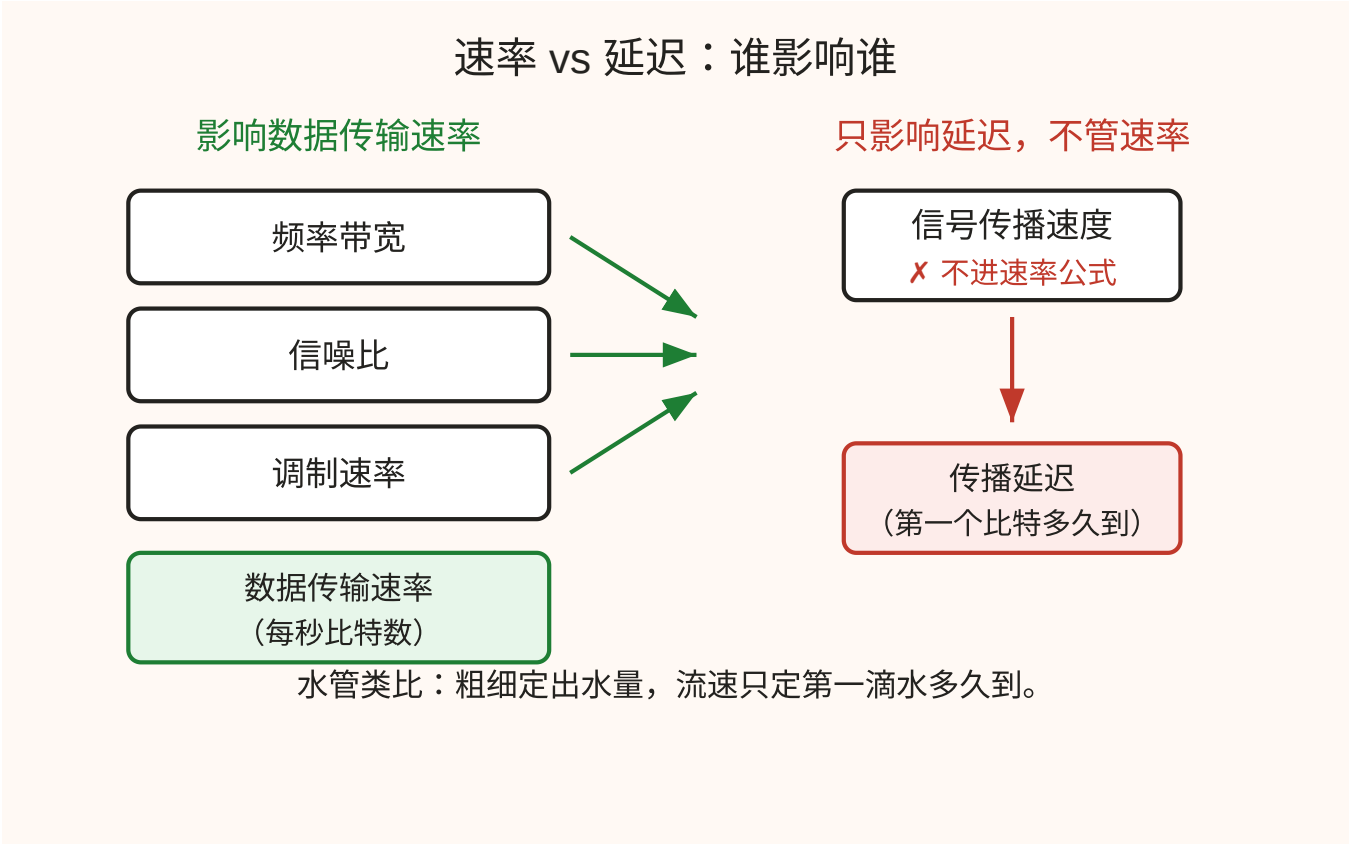
正确选项为什么对： 信号传播速度只影响延迟，不影响每秒传多少比特，选 D。

错误选项为什么错＋举例：

- A（信噪比）：香农公式里信噪比决定速率上限，信噪比越高，速率越高；所以它影响速率，不是答案。教训：信噪比是香农公式里的关键量。
- B（频率带宽）：奈奎斯特和香农公式里带宽都决定速率上限，带宽越宽，速率越高；所以它影响速率，不是答案。教训：带宽是速率的第一决定因素。
- C（调制速率）：调制速率是每秒信号变化几次，直接决定速率；所以它影响速率，不是答案。教训：调制速率和速率是直接挂钩的。

笔记本一句话： 第 35 题＋速率和延迟：带宽、信噪比、调制速率都影响速率；传播速度只影响延迟，不影响速率。

【图像】



信道数据传输速率由带宽、信噪比、调制速率决定，信号传播速度只决定延迟的对比示意图

【定位】 计算机网络 · 第2章 物理层 · 信道数据传输速率的影响因素

【易混对比】

- "传输速率"和"延迟"的区别：速率是每秒发多少比特（水管粗细），延迟是第一个比特多久到达（水流多久到）；传播速度只改变后者，不改变前者。
- "影响速率"和"不进速率公式"的区别：带宽（奈奎斯特）、信噪比（香农）、调制速率都直接抬升速率；传播速度连速率公式都不进，根本不参与。

速率看每秒发多少比特：带宽、信噪比、调制速率都影响它；传播速度只影响"多久到"，不影响"每秒多少"。

【坑位】

别把"信号在介质里跑得快"当成"每秒传的比特多"。

传播速度快，第一个比特到得早，但发送方每秒还是只能发出那么多比特——发比特的速度看带宽和调制，不看传播。正确做法：看到"影响速率"先问"它进速率公式吗"：带宽、信噪比、调制速率都进公式，传播速度只进延迟公式。

【类比】

像水管：水管的粗细（带宽）、水压和水质（信噪比）、阀门开关的频率（调制速率）决定每秒流出多少水；水在管里流多快（传播速度）只决定第一滴水多久到你家，不决定每秒出多少水。术语贴回去：出水量就是"数据传输速率"，第一滴水到家的时间就是"传播延迟"。

【口诀】 速率看粗细，延迟看快慢；传播速度不管速率，只管延迟。

【考频】 「传输速率影响因素」在 2009–2022 年解析中出现 20 次 · 高频

第 36 题 · 后退 N 帧协议的最大平均数据传输速率（正确答案：C）

题干与选项：

主机甲与主机乙之间使用后退 N 帧协议(GBN)传输数据，甲的发送窗口尺寸为 1000， 数据帧长为 1000 字节，信道带宽为 100Mbps，乙每收到一个数据帧立即利用一个短帧(忽略 其传输延迟)进行确认，若甲乙之间的单向传播延迟是 50ms，则甲可以达到的最大平均数据 传输速率约为 。

- A. 10Mbps
- B. 20Mbps
- C. 80Mbps
- D. 100Mbps

入手点： 先算发完一窗口要多久，再算确认多久回来；窗口是瓶颈，速率就是一窗口的数据除以一轮的时间。

专有名词从零解释：

- 先想“后退 N 帧协议”：也叫 GBN，发送方一次能连续发好几帧；如果有一帧出错，发送方就把这帧和它后面的帧全部重发。
- 再想“发送窗口”：发送方一次能连续发几帧、不用等确认；本题窗口是 1000 帧。
- 再想“单向传播延迟”：信号从甲到乙要花多久，本题是 50ms；往返就是 100ms。
- 再想“最大平均数据传输速率”：长时间平均下来，每秒能传多少比特；窗口限制下，实际速率可能达不到信道带宽。

考查知识点：

- 先想发一帧要多久：帧长 1000 字节，等于 8000 比特；带宽 100Mbps，就是每秒 1 亿比特；8000 除以 1 亿，等于 80 微秒。
- 再想发完一窗口要多久：1000 帧，每帧 80 微秒，一共 80ms。
- 再想确认多久回来：第一帧发出后，确认要经过往返时间才回来；往返时间是单向 50ms 的两倍，等于 100ms。
- 再想一轮的时间和速率：发完 1000 帧用了 80ms，但第一帧的确认要 100ms 才到；所以每 100ms 才能发 1000 帧；1000 帧是 800 万比特，除以 0.1 秒，等于 80Mbps，就是 C。
- 拿生活例子验算：一次寄 1000 封信，寄完要 80 分钟；第一封信的回执要 100 分钟才到；所以每 100 分钟寄 1000 封信，平均每小时寄 600 封；回执没到之前，不能再寄下一批。
- 边界情况：如果窗口足够大（大到发完一窗口的时间超过往返时间），发送方就不用等确认，可以一直发，速率就能达到带宽 100Mbps；本题窗口 1000 帧，发完只要 80ms，小于往返 100ms，所以窗口是瓶颈。
- 易混对子：不要把“发完一窗口的时间”（80ms）当成“一轮的时间”——一轮的时间是确认回来的时间（100ms），不是发完的时间；也不要把“信道带宽”（100Mbps）当成答案——带宽是上限，窗口限制下实际只能到 80Mbps。

正确选项为什么对： 每 100ms 发 1000 帧，1000×8000÷0.1＝80Mbps，选 C。

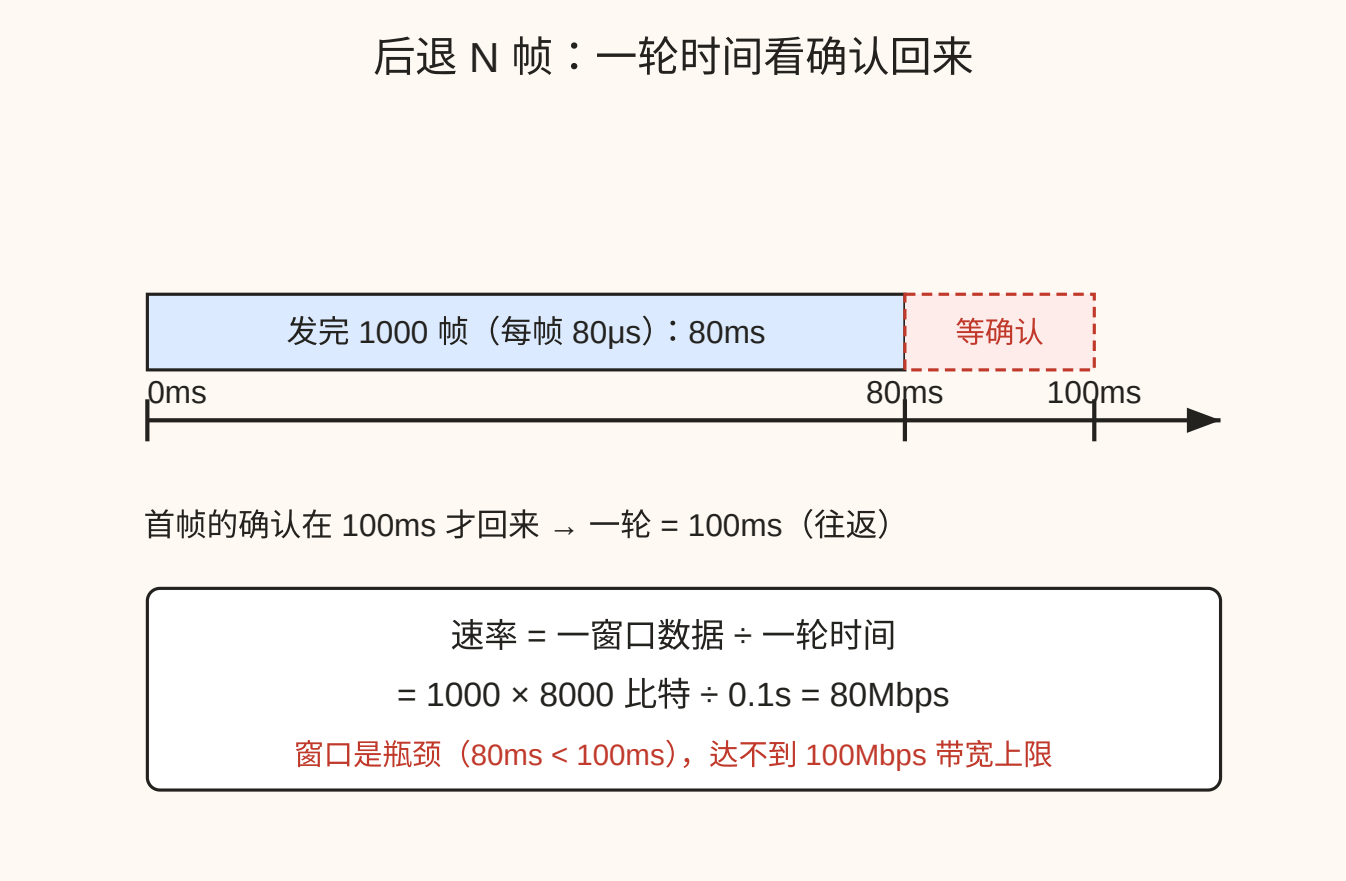
错误选项为什么错＋举例：

- A（10Mbps）：把窗口或时间算错了一个数量级；教训：帧长是 1000 字节（8000 比特），不是 1000 比特。
- B（20Mbps）：也是窗口或时间算错了；教训：往返时间是单向的两倍（100ms），不是单向的 50ms，也不是 400ms。

- D（100Mbps）：那是信道带宽上限，没算窗口限制；教训：窗口满了要等确认，实际速率达不到带宽。

笔记本一句话： 第 36 题＋窗口限制速率：一轮时间是往返时间（100ms），不是发完时间；1000 帧÷100ms＝80Mbps。

【图像】



后退 N 帧下每 100ms 往返才能发出一窗口 1000 帧的时序示意图

【定位】 计算机网络 · 第3章 数据链路层 · 滑动窗口协议的平均速率

【易混对比】

- "发完一窗口的时间"和"一轮的时间"的区别：发完 1000 帧只要 80ms，但首帧的确认要 100ms（往返）才回来；确认没回来前窗口卡死不动，一轮是 100ms，不是 80ms。
- "信道带宽"和"实际平均速率"的区别：带宽 100Mbps 是天花板；窗口太小导致发送方大量时间在等确认，实际只能到 80Mbps；窗口大到发不停时才能摸到天花板。

窗口是瓶颈时：一轮 = 往返时间 RTT，速率 = 一窗口的数据量 ÷ RTT。

【坑位】

别把"发完窗口的 80ms"当成分母。

算出每帧 80μs、1000 帧 80ms 就顺手除，漏了"确认要 100ms 才回来"这一环。窗口满了就得等，等到确认回来的 100ms 才是真实的一轮时间。正确做法：先比"发完窗口时间"和"往返时间"，哪个长一轮就是哪个；本题 100ms > 80ms，窗口是瓶颈。

【套路】

- 第一步，算一帧的发送时间：1000 字节 = 8000 比特，÷ 100Mbps = 80μs。

- 第二步，算发完窗口的时间：1000 × 80μs = 80ms。
- 第三步，算往返时间：单向 50ms × 2 = 100ms；取两者中大的作为一轮时间。
- 第四步，算速率：一窗口数据 1000 × 8000 比特 ÷ 0.1s = 80Mbps。

一帧时间 → 一窗口时间 → 与往返比大小 → 窗口数据 ÷ 较长者。

【类比】

像一次寄 1000 封信：寄完信花 80 分钟，但第一封信的回执要 100 分钟才到；回执没到就不能寄下一批，所以每 100 分钟才寄出 1000 封。术语贴回去：寄信就是"发一窗口"，等回执就是"往返时间"，寄出速度就是"平均数据传输速率"。

【口诀】 一轮看长的：窗口慢用窗口，确认慢用确认；窗口数据除以一轮时间。

【考频】 「滑动窗口速率」在 2009–2022 年解析中出现 15 次 · 高频

第 37 题 · CDMA 码片序列的解码（正确答案：B）

题干与选项：

站点 A、B、C 通过 CDMA 共享链路，A、B、C 的码片序列(chipping sequence)分别是(1,1,1,1)、(1,-1,1,-1)和(1,1,-1,-1)。若 C 从链路上收到的序列是(2,0,2,0,0,-2,0,-2,0,2,0,2)， 则 C 收到 A 发送的数据是 。

- A. 000
- B. 101
- C. 110
- D. 111

入手点： 用 A 的码片序列和接收序列做内积，分组算：每组对应 1 比特，内积是正数就是 1，内积是负数就是 0。

专有名词从零解释：

- 先想“CDMA”：码分多址，好几个用户同时在一条链路上发数据，每人用不同的码片序列区分。
- 再想“码片序列”：一串 1 和负 1；比如 A 的码片是 (1,1,1,1)，B 的是 (1,-1,1,-1)，C 的是 (1,1,-1,-1)。
- 再想“内积”：两个序列对应位置相乘，再全部加起来；比如 (2,0,2,0) 和 (1,1,1,1) 的内积是 2 乘 1 加 0 乘 1 加 2 乘 1 加 0 乘 1，等于 4。
- 再想“正交”：不同用户的码片序列内积是 0；所以用 A 的码片去算接收序列，B 和 C 的部分会被抵消掉，只剩下 A 的部分。

考查知识点：

- 先想接收序列怎么分组：接收序列一共 12 位，A 的码片是 4 位，所以分成 3 组，每组 4 位，对应 A 发的 3 比特。
- 再想第 1 组：(2,0,2,0) 和 (1,1,1,1) 做内积，等于 2 加 0 加 2 加 0，等于 4；正数，对应比特 1。
- 再想第 2 组：(0,-2,0,-2) 和 (1,1,1,1) 做内积，等于 0 减 2 加 0 减 2，等于负 4；负数，对应比特 0。
- 再想第 3 组：(0,2,0,2) 和 (1,1,1,1) 做内积，等于 0 加 2 加 0 加 2，等于 4；正数，对应比特 1。
- 再想合起来：1、0、1，就是 101，就是 B。
- 拿生活例子验算：三个人同时说话，每人说不同的语言；听的人只懂 A 的语言，把听到的话按 A 的语言翻译，B 和 C 的话就被过滤掉了，只剩下 A 的话。
- 边界情况：内积是 0 时，说明这个用户没发数据（或者发的是 0 比特的另一种表示）；本题三组内积都不为 0，所以三比特都有值。
- 易混对子：不要把“内积正数”和“比特 0”搞反——正数对应 1，负数对应 0；也不要把分组数错——12 位除以 4 位码片，是 3 组，不是 4 组。

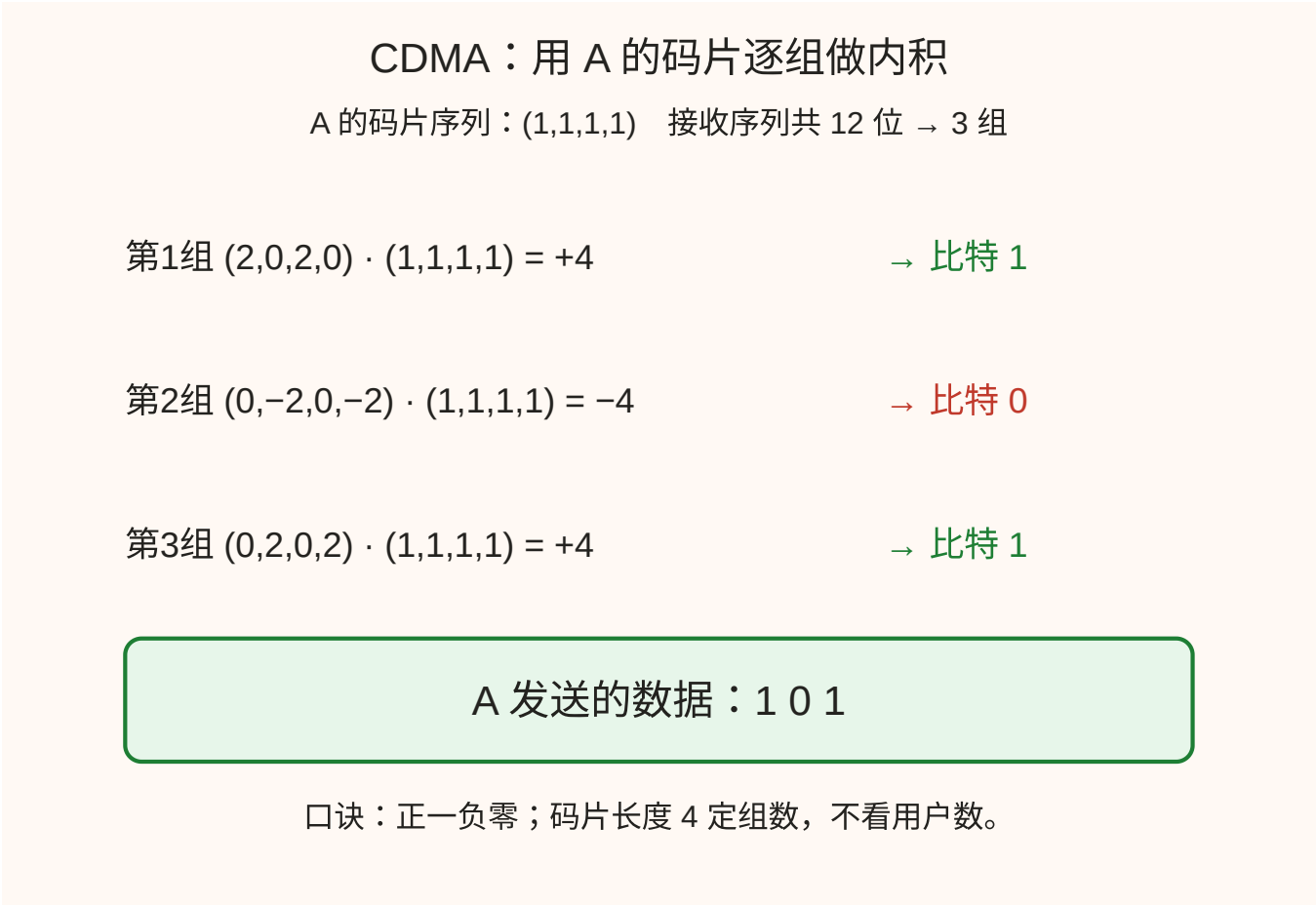
正确选项为什么对： 三组内积分别是正、负、正，对应 1、0、1，就是 101，选 B。

错误选项为什么错＋举例：

- A（000）：三组内积都看成负数了；教训：正数对应 1，不是 0。
- C（110）：第 3 组算错了；(0,2,0,2) 的内积是正数 4，对应 1，不是 0。教训：分组内积要一位一位乘了再加。
- D（111）：第 2 组算错了；(0,-2,0,-2) 的内积是负数，对应 0，不是 1。教训：负数对应 0，不是 1。

笔记本一句话： 第 37 题＋CDMA 解码：用接收方的码片分组做内积，正数是 1、负数是 0；本题是 101。

【图像】



用 A 的码片序列对接收序列分组做内积得到 1、0、1 的解码示意图

【定位】 计算机网络 · 第2章 物理层 · CDMA 码片序列解码

【易混对比】

- "内积为正"和"比特 1"的区别：内积为正（+4）对应发送的比特是 1，内积为负（−4）对应比特是 0；符号和比特值一一映射，不能反。
- "用谁的码片解"的区别：要解 A 发的数据，必须用 A 的码片序列做内积；用 B 或 C 的码片，算出来的是 B 或 C 的数据。

用目标用户的码片分组做内积：正数读 1，负数读 0；不同用户的码片正交，干扰相互抵消。

【坑位】

别把"12 位接收序列"按用户数 3 分组——分组数看的是码片长度。

分组依据是码片序列长度 4 位，不是用户数 3 个；12 ÷ 4 = 3 组，对应 A 发的 3 个比特。正确做法：先数码片有几位，接收序列除以码片长度才是组数；每组用码片做一次内积，读一个比特。

【套路】

- 第一步，定分组：码片 4 位，接收序列 12 位 → 3 组，每组对应 1 比特。
- 第二步，逐组做内积：组内对应位相乘再相加（(2,0,2,0)·(1,1,1,1)=4；(0,−2,0,−2)·(1,1,1,1)=−4；(0,2,0,2)·(1,1,1,1)=4）。
- 第三步，看符号读比特：正→1，负→0，得 101。

码片长度定组数，逐组内积看正负，正一负零。

【类比】

像三个人同时说三种语言：只懂 A 的语言的人，把听到的话按 A 的语言翻译，B 和 C 的话就被自动过滤掉。术语贴回去：语言就是"码片序列"，按语言翻译就是"做内积"，听懂的正反意思就是"比特 1 或 0"。

【口诀】 谁的码片解谁的数，分组内积看正负，正一负零。

【考频】 「CDMA」在 2009–2022 年解析中出现 3 次 · 中频

第 38 题 · TCP 拥塞窗口变化后的发送窗口（正确答案：A）

题干与选项：

主机甲和主机乙已建立了 TCP 连接，甲始终以 MSS=1KB 大小的段发送数据，并 一直有数据发送；乙每收到一个数据段都会发出一个接收窗口为 10KB 的确认段。若甲在 t 时刻发生超时时拥塞窗口为 8KB，则从 t 时刻起，不再发生超时的情况下，经过 10 个 RTT 后，甲的发送窗口是 。

- A. 10KB
- B. 12KB
- C. 14KB
- D. 15KB

入手点： 先算超时后的起点，再分两个阶段走：慢启动阶段翻倍，拥塞避免阶段每次加 1；最后发送窗口取拥塞窗口和接收窗口里小的那个。

专有名词从零解释：

- 先想“拥塞窗口”：也叫 cwnd，发送方根据网络拥塞程度自己定的窗口；窗口越大，一次能发的数据越多。
- 再想“慢启动”：TCP 刚开始或者超时后，拥塞窗口从 1 个 MSS 开始，每经过一个 RTT 翻一倍；像汽车起步时慢慢加速。
- 再想“慢启动阈值”：也叫 ssthresh，是慢启动和拥塞避免的分界线；拥塞窗口小于阈值时，每 RTT 翻倍；大于阈值时，每 RTT 只加 1 个 MSS。
- 再想“拥塞避免”：超过阈值后，每 RTT 只加 1 个 MSS，增速变慢；目的是试探网络，不一下子发太多。
- 再想“接收窗口”：也叫 rwnd，接收方告诉发送方的窗口；等于接收缓存还剩多少空间；本题接收窗口是 10KB。
- 再想“发送窗口”：也叫 swnd，发送方实际能发的窗口；等于拥塞窗口和接收窗口里较小的那一个。
- 再想“MSS”：最大报文段长度，一次能发的数据段的最大字节数；本题 MSS 是 1KB。
- 再想“RTT”：往返时间，发一帧、等确认回来的时间。

考查知识点：

- 先想超时后的起点：超时时拥塞窗口是 8KB，超时后拥塞窗口变成 1KB，慢启动阈值变成原来的一半，也就是 4KB。
- 再想慢启动阶段（拥塞窗口小于阈值 4KB）：第 1 个 RTT，拥塞窗口从 1KB 翻倍到 2KB；第 2 个 RTT，从 2KB 翻倍到 4KB，到达阈值；慢启动阶段用了 2 个 RTT。
- 再想拥塞避免阶段（拥塞窗口达到阈值后，每 RTT 加 1KB）：第 3 个 RTT，4KB 加到 5KB；第 4 个 RTT，5KB 加到 6KB；第 5 个 RTT，6KB 加到 7KB；第 6 个 RTT，7KB 加到 8KB；第 7 个 RTT，8KB 加到 9KB；第 8 个 RTT，9KB 加到 10KB；第 9 个 RTT，10KB 加到 11KB；第 10 个 RTT，11KB 加到 12KB；拥塞避免阶段用了 8 个 RTT。
- 再想 10 个 RTT 后的拥塞窗口：慢启动 2 个 RTT 加拥塞避免 8 个 RTT，一共 10 个 RTT，拥塞窗口是 12KB。
- 再想发送窗口：拥塞窗口 12KB，接收窗口 10KB，取较小的，就是 10KB，就是 A。
- 拿生活例子验算：水桶（接收缓存）一共 10 升，水龙头（拥塞窗口）能放 12 升；一次最多只能接 10 升，不然桶就满了；发送窗口就是“桶还剩多少”，也就是 10 升。
- 边界情况：如果接收窗口比拥塞窗口大，发送窗口就是拥塞窗口；本题接收窗口 10KB 比拥塞窗口 12KB 小，所以发送窗口是接收窗口。

- 易混对子：不要把“拥塞窗口”和“发送窗口”搞混——发送窗口是两个窗口里较小的那个；也不要把“慢启动阈值”当成“超时后的拥塞窗口”——超时后拥塞窗口是 1KB，阈值是 4KB。

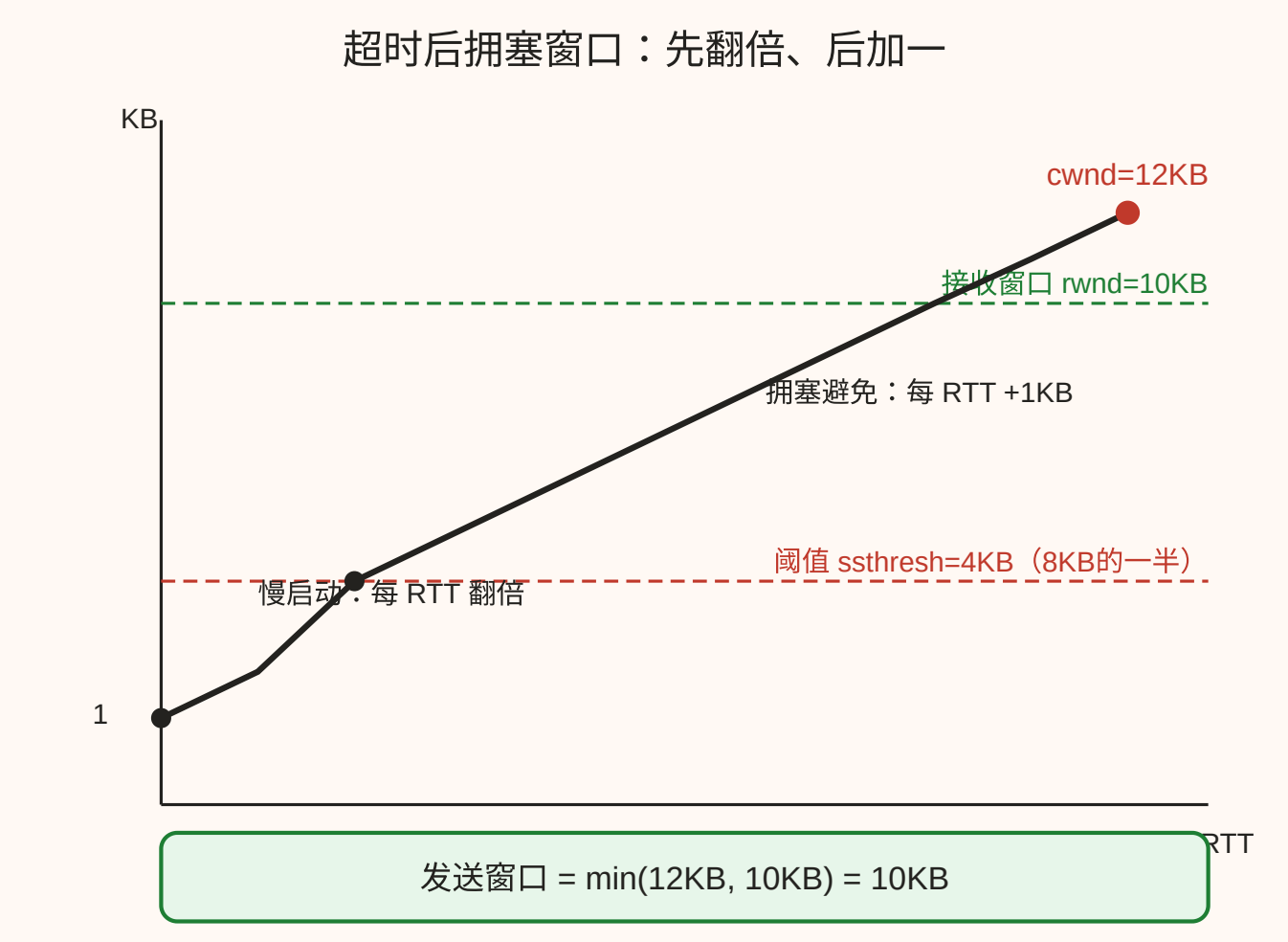
正确选项为什么对： 10 个 RTT 后拥塞窗口 12KB、接收窗口 10KB，取较小值，发送窗口是 10KB，选 A。

错误选项为什么错＋举例：

- B（12KB）：那是拥塞窗口，不是发送窗口；发送窗口要取拥塞窗口和接收窗口里较小的那个，接收窗口只有 10KB。教训：发送窗口取两者最小值。
- C（14KB）：RTT 数算错了，多算了 2 个 RTT 的增长；教训：慢启动 2 个 RTT 加拥塞避免 8 个 RTT，一共 10 个 RTT，不是 12 个。
- D（15KB）：把阈值或增长算错了；教训：超时后阈值是原来的一半（4KB），不是 8KB；拥塞避免每 RTT 只加 1KB。

笔记本一句话： 第 38 题＋TCP 发送窗口：超时后 cwnd＝1KB、阈值＝4KB；慢启动 2 个 RTT 到 4KB，拥塞避免 8 个 RTT 到 12KB；发送窗口取 min(12,10)＝10KB。

【图像】



超时后拥塞窗口从 1KB 慢启动翻倍到阈值 4KB，再每 RTT 加 1KB 到 12KB，取与接收窗口 10KB 较小者得发送窗口 10KB 的曲线图

【定位】 计算机网络 · 第5章 传输层 · TCP 拥塞控制与发送窗口

【易混对比】

- "拥塞窗口"和"发送窗口"的区别：拥塞窗口（cwnd）是发送方按拥塞情况自己估的，发送窗口是 min（拥塞窗口，接收窗口）；题目问发送窗口，必须再和接收窗口 10KB 比一次。
- "超时后阈值"和"超时时拥塞窗口"的区别：超时后阈值 = 超时时拥塞窗口的一半（8KB → 4KB），拥塞窗口本身重置为 1KB；阈值不是 8KB，起点也不是 8KB。

超时后：cwnd 从 1KB 起，阈值 = 原 cwnd 一半；慢启动翻倍到阈值，之后每 RTT 加 1；发送窗口 = min（拥塞窗口，接收窗口）。

【坑位】

别从 8KB 继续涨——超时把起点打回 1KB 了。

看到"超时时拥塞窗口 8KB"就从 8KB 往上加，是最常见的坑。超时的惩罚是：阈值砍半（4KB）、窗口归 1KB，然后慢启动重新爬。正确做法：先写"新起点 1KB、新阈值 4KB"，再分两段数 RTT：翻倍段（1→2→4，用 2 个 RTT）+ 线性段（4→12，用 8 个 RTT）。

【套路】

- 第一步，定超时后的新起点：cwnd = 1KB，ssthresh = 8KB ÷ 2 = 4KB。
- 第二步，数慢启动 RTT：cwnd < 阈值时每 RTT 翻倍：1→2→4，用 2 个 RTT 到阈值。
- 第三步，数拥塞避免 RTT：剩余 8 个 RTT，每 RTT 加 1KB：4→12KB。
- 第四步，取发送窗口：min（12KB，10KB）= 10KB。

超时重置 1KB、阈值减半；先翻倍后加一，最后和接收窗口取小。

【类比】

像水龙头接水桶：超时相当于把水龙头拧到最小（1KB）重新开；开到一半（阈值 4KB）后改成每次只拧大一点；但水桶（接收窗口）只有 10 升，水龙头再大也只能接 10 升。术语贴回去：水龙头开度就是"拥塞窗口"，水桶容量就是"接收窗口"，实际接水量就是"发送窗口"。

【口诀】 超时归一阈减半，先翻倍来后加一，发送窗口取较小。

【考频】 「TCP 拥塞控制」在 2009–2022 年解析中出现 12 次 · 高频

第 39 题 · UDP 协议相关叙述的正误（正确答案：B）

题干与选项：

下列关于 UDP 协议的叙述中，正确 的是 。 Ⅰ．提供无连接服务 Ⅱ．提供复用/分用服务 Ⅲ．通过差错校验，保障可靠数据传输

- A. 仅 Ⅰ
- B. 仅 Ⅰ、Ⅱ
- C. 仅 Ⅱ、Ⅲ
- D. Ⅰ、Ⅱ、Ⅲ

入手点： 先把 UDP 的三个特点记牢：无连接、复用分用、只检错不保证可靠；再看每条说法和这三个特点合不合。

专有名词从零解释：

- 先想“UDP”：用户数据报协议，传输层的一种协议；特点是简单、快，但不保证可靠。
- 再想“无连接服务”：传数据前不用建立连接，直接发；像寄明信片，写好地址就扔进邮筒。
- 再想“复用、分用”：多个应用共用一条路，靠端口号区分；发送时叫复用（好几个应用的数据合起来走一条路），接收时叫分用（按端口号分给不同的应用）。
- 再想“差错校验”：UDP 算一个校验和，接收方收到后验算；验算不对，说明数据错了，就把数据报丢掉；但丢了不重传，不保证可靠。

考查知识点：

- 先想 Ⅰ 为什么对：UDP 提供无连接服务，传数据前不建立连接，直接发；所以 Ⅰ 对。
- 再想 Ⅱ 为什么对：UDP 用端口号区分不同应用，就是复用和分用服务；所以 Ⅱ 对。
- 再想Ⅲ为什么错：UDP 的差错校验只能发现错误，不能保证可靠传输；发现错了就把数据报丢掉，不重传；保障可靠传输的是 TCP，不是 UDP。
- 再想组合： Ⅰ 对、 Ⅱ 对、Ⅲ错，就是“仅 Ⅰ 、 Ⅱ”，就是 B。
- 拿生活例子验算：UDP 像寄明信片，不用先打电话约好（无连接），信封上写收件人（端口号，分用），字写错了收件人看出来就扔掉（检错），但扔了不会再寄一封（不保证可靠）。
- 边界情况：UDP 的校验和是可选的（IPv4 里可以不算），但就算算了，也只能检错，不能纠错、不能重传；所以Ⅲ还是错的。
- 易混对子：不要把“有差错校验”当成“可靠传输”——校验只能发现错误，不能保证数据一定到、一定对；可靠传输要确认、重传、排序，这些 UDP 都没有。

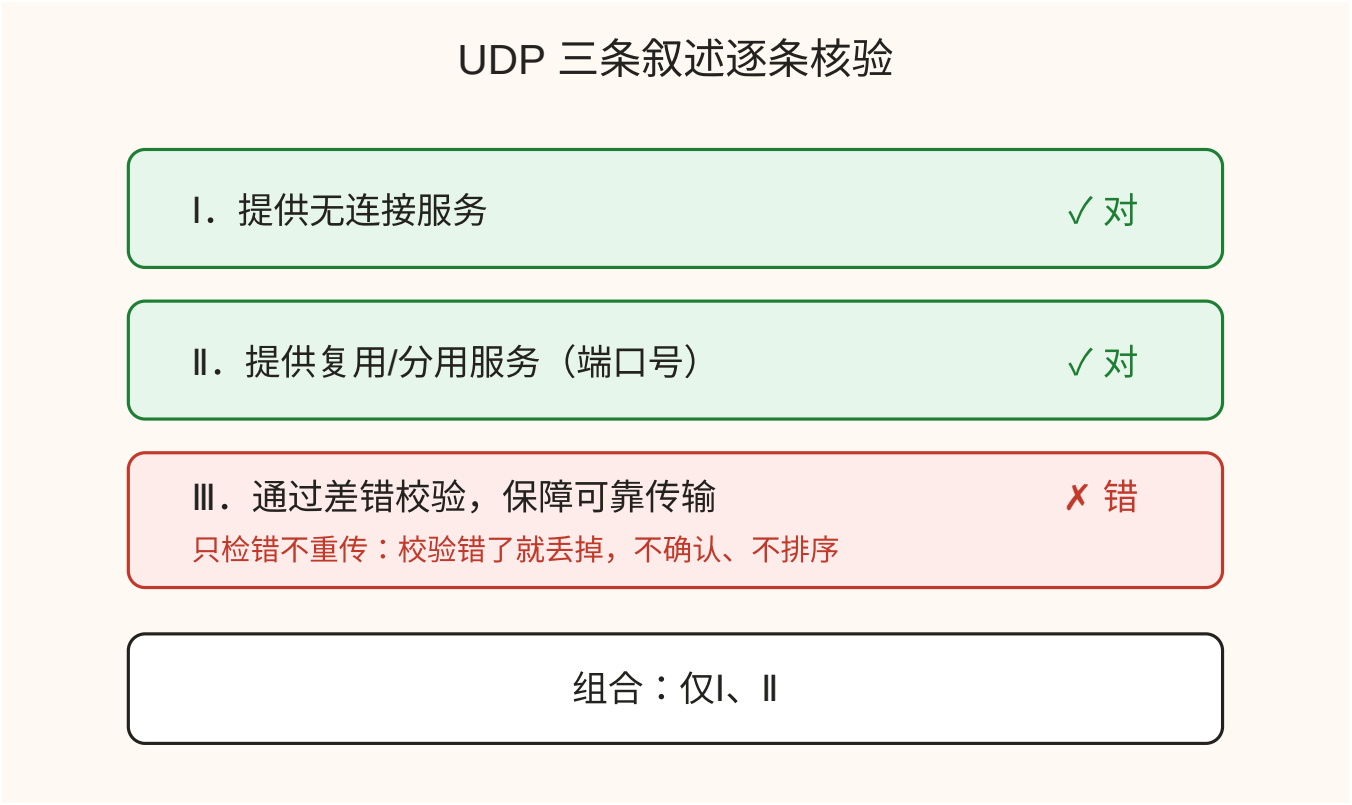
正确选项为什么对： Ⅰ 和 Ⅱ 都对，Ⅲ错，所以是仅 Ⅰ 、 Ⅱ，选 B。

错误选项为什么错＋举例：

- A（仅 Ⅰ）：漏了 Ⅱ；UDP 用端口号复用分用， Ⅱ 也是对的。教训：组合题要一条一条验，不能漏。
- C（仅 Ⅱ 、Ⅲ）：把错的Ⅲ算进来了，漏了 Ⅰ ；教训：Ⅲ的“保障可靠”是错的，UDP 不保障可靠。
- D（ Ⅰ 、 Ⅱ 、Ⅲ）：把错的Ⅲ也算进来了；教训：三条不可能全对，Ⅲ是错的。

笔记本一句话： 第 39 题＋UDP 三特点：无连接、复用分用、只检错不保证可靠；仅 Ⅰ 、 Ⅱ 。

【图像】



UDP 三条叙述逐条核验：无连接对、复用分用对、保障可靠错的打勾打叉示意图

【定位】 计算机网络 · 第5章 传输层 · UDP 协议的特点

【易混对比】

- "有差错校验"和"保障可靠"的区别：校验只能发现错误（发现错了就丢掉），保障可靠还要确认、重传、排序；UDP 只有前者，没有后者，所以第 III 条错。
- "复用/分用"和"可靠"的区别：复用/分用是靠端口号把数据分给不同应用的机制，和数据丢不丢、错不错没关系；别因为 UDP 有端口号就觉得它可靠。

UDP = 无连接 + 端口号复用分用 + 只检错不重传；"保障可靠"四个字一出现，这条就错。

【坑位】

别看到"差错校验"就勾"可靠"。

UDP 确实算校验和，但校验和错了只是把数据报扔掉，不重发、不排序、不确认——可靠传输的活儿一件不干。正确做法：逐条核验时，把第 III 条拆成两问："有校验吗？有。能保障可靠吗？不能。"两问都对才算对。

【类比】

像寄明信片：不用先打电话约（无连接），信封写收件人名字（端口号，分用），字写错了收件人看出来就扔掉（检错）；但扔了不会再寄一张（不保障可靠）。术语贴回去：明信片就是"UDP 数据报"，收件人名字就是"端口号"，扔掉不重寄就是"不可靠"。

【口诀】 UDP 三件套：无连接、端口分用、检错不重传；见"可靠"就判错。

【考频】 「UDP」在 2009–2022 年解析中出现 15 次 · 高频

第 40 题 · 浏览大学 Web 网站主页时不可能用到的协议（正确答案：D）

题干与选项：

使用浏览器访问某大学 Web 网站主页时，不可能 . 使用到的协议是 。

- A. PPP
- B. ARP
- C. UDP
- D. SMTP 二、综合应用题：41—47 小题，共 70 分。

入手点： 先想打开一个网页要经过哪些步骤：接入网络、查域名、查 MAC 地址、取网页；再看每个协议是哪个步骤用的。

专有名词从零解释：

- 先想“PPP”：点对点协议，拨号上网时用的协议；电脑通过电话线或者宽带拨号接入网络时，要用 PPP。
- 再想“ARP”：地址解析协议，IP地址查 MAC 地址；电脑要给网关发数据时，要先查网关的 MAC 地址，就用 ARP。
- 再想“UDP”：用户数据报协议；查域名用的 DNS，平时就是走 UDP 的。
- 再想“SMTP”：简单邮件传送协议，发邮件用的协议；发邮件时，邮件客户端把邮件发给邮件服务器，就用 SMTP。
- 再想“HTTP”：超文本传送协议，取网页用的协议；浏览器向 Web 服务器要网页，就用 HTTP。

考查知识点：

- 先想 PPP 为什么可能用到：电脑要上网，可能要先拨号接入；拨号接入就用 PPP。
- 再想 ARP 为什么可能用到：电脑要给网关发数据，要先知道网关的 MAC 地址；查 MAC 地址就用 ARP。
- 再想 UDP 为什么可能用到：浏览器要先查大学网站的域名对应哪个 IP 地址；查域名用的 DNS，平时走 UDP。
- 再想 SMTP 为什么不可能用到：SMTP 是发邮件的协议；打开网页主页不用发邮件，所以用不上 SMTP。
- 拿生活例子验算：去大学网站首页，像去一栋楼里找一间办公室；进楼要刷卡（PPP），要问门牌号对应的房间号（ARP），要查楼层索引（DNS 走 UDP）；发邮件（SMTP）是另一件事，找办公室用不上。
- 边界情况：如果大学网站首页里嵌了邮件订阅功能，用户点了订阅才可能用到 SMTP；但题目问“浏览主页时”，只是打开主页，不点订阅，所以 SMTP 用不上。
- 易混对子：不要把“DNS 走 UDP”和“打开网页走 TCP”搞混——DNS 查域名走 UDP，取网页走 TCP（HTTP）；题目问 UDP 可不可能用到，DNS 就用 UDP，所以 UDP 可能用到。

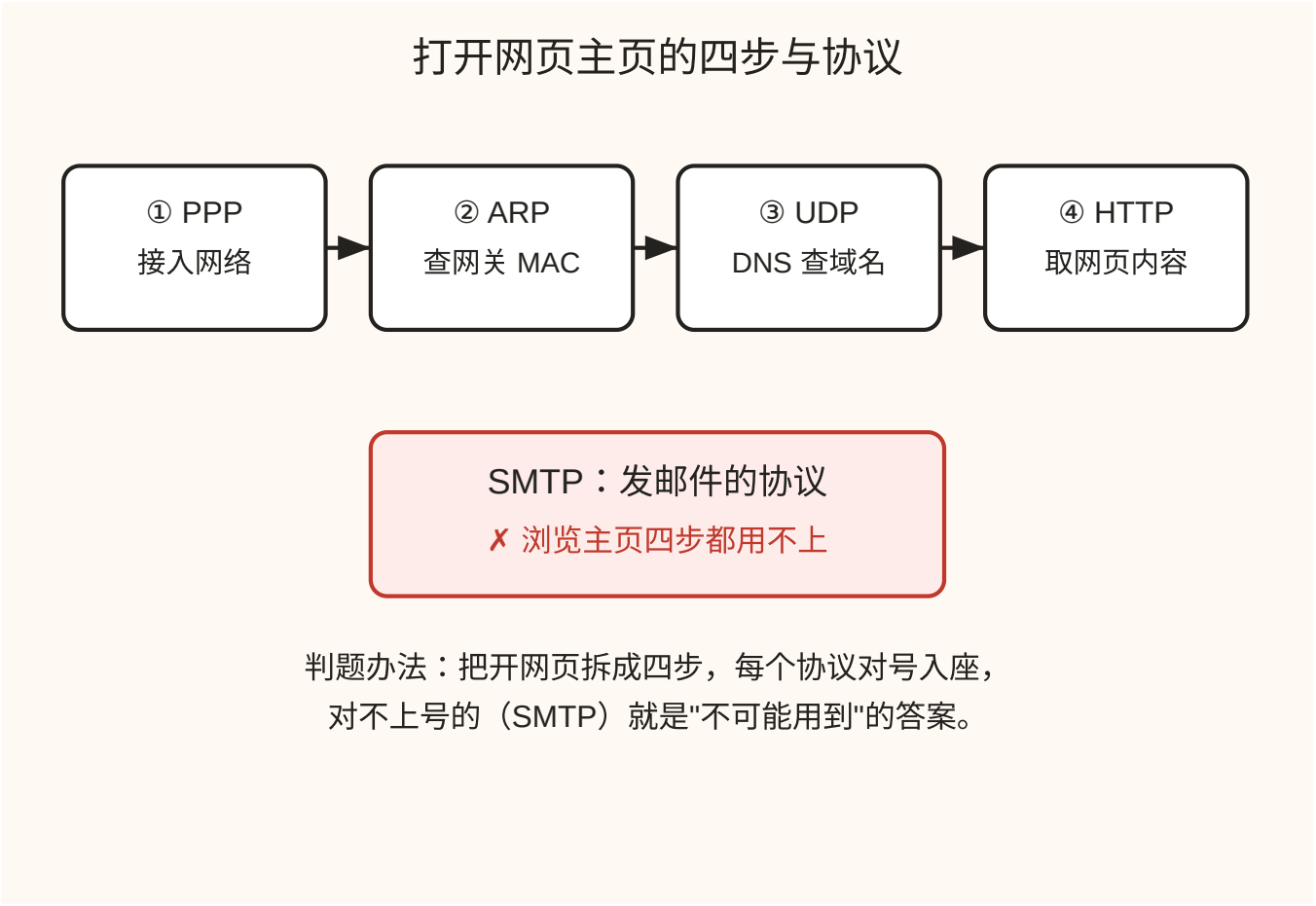
正确选项为什么对： SMTP 是发邮件的协议，打开网页主页用不上，选 D。

错误选项为什么错＋举例：

- A（PPP）：拨号接入可能用到 PPP；教训：上网接入这一步可能用 PPP。
- B（ARP）：查网关 MAC 地址可能用到 ARP；教训：发数据前要先查 MAC 地址。
- C（UDP）：查域名用的 DNS 走 UDP，可能用到；教训：DNS 平时走 UDP。

笔记本一句话： 第 40 题＋打开网页用到的协议：PPP 接入、ARP 查 MAC、UDP 查域名都可能；SMTP 发邮件用不上。

【图像】



打开网页四步用到的协议：PPP 接入、ARP 查 MAC、DNS 走 UDP 查域名、HTTP 取网页，SMTP 发邮件用不上的流程图

【定位】 计算机网络 · 第6章 应用层 · 浏览网页涉及的协议

【易混对比】

- "可能用到"和"一定用到"的区别：题目问"不可能用到"，PPP 拨号接入、ARP 查网关 MAC、UDP 查域名都"可能"出现在开网页的某一步，只有 SMTP（发邮件）哪一步都插不进去。
- "DNS 走 UDP"和"取网页走 TCP"的区别：查域名是 DNS 的活、走 UDP；拿网页内容是 HTTP 的活、走 TCP；题目问 UDP 可不可能用到，看的是 DNS 那一步，不是取网页那一步。

开网页四步：PPP 接入、ARP 查 MAC、UDP（DNS）查域名、HTTP 取网页；SMTP 只管发邮件，主页里用不上。

【坑位】

别因为"DNS 走 UDP、取网页走 TCP"就把 UDP 判成用不上。

题目问的是整个"浏览主页"过程用不用 UDP，不是"取网页这一步"用不用 UDP；查域名那一步 DNS 默认走 UDP，所以 UDP 可能用到。正确做法：把开网页拆成接入→查 MAC→查域名→取网页四步，每个协议往四步里对号，对不上号的才是答案。

【类比】

像去一栋楼找办公室：进楼刷卡（PPP 接入），问门牌对应的房间号（ARP 查 MAC），查楼层索引（DNS 走 UDP），进屋拿文件（HTTP 取网页）；发邮件（SMTP）是去邮局办另一件事，找办公室用不上。术语贴回去：刷卡就是"接入认证"，房间号就是"MAC 地址"，楼层索引就是"域名解析"。

【口诀】 开网页四步走：接入查域取网页；SMTP 发邮件，主页用不上。

【考频】 「应用层协议」在 2009–2022 年解析中出现 15 次 · 高频